

PL/SQL

Sanae MAZOUZ

PL/SQL

- ***PL/SQL (Procedural Language / Structured Query Language)*** est un langage de programmation procédural développé par Oracle.
- Il permet d'étendre le langage SQL en y ajoutant des structures de programmation telles que les variables, les conditions, les boucles et la gestion des exceptions.

Oracle

- Oracle Database est un système de gestion de base de données relationnelle (**SGBDR**) développé par la société **Oracle Corporation**.
- Il permet de stocker, organiser, sécuriser et gérer de grandes quantités de données, tout en offrant des mécanismes avancés de performance, de sécurité et de disponibilité.
- Oracle Database supporte le langage SQL pour l'interrogation des données et PL/SQL pour la programmation côté serveur

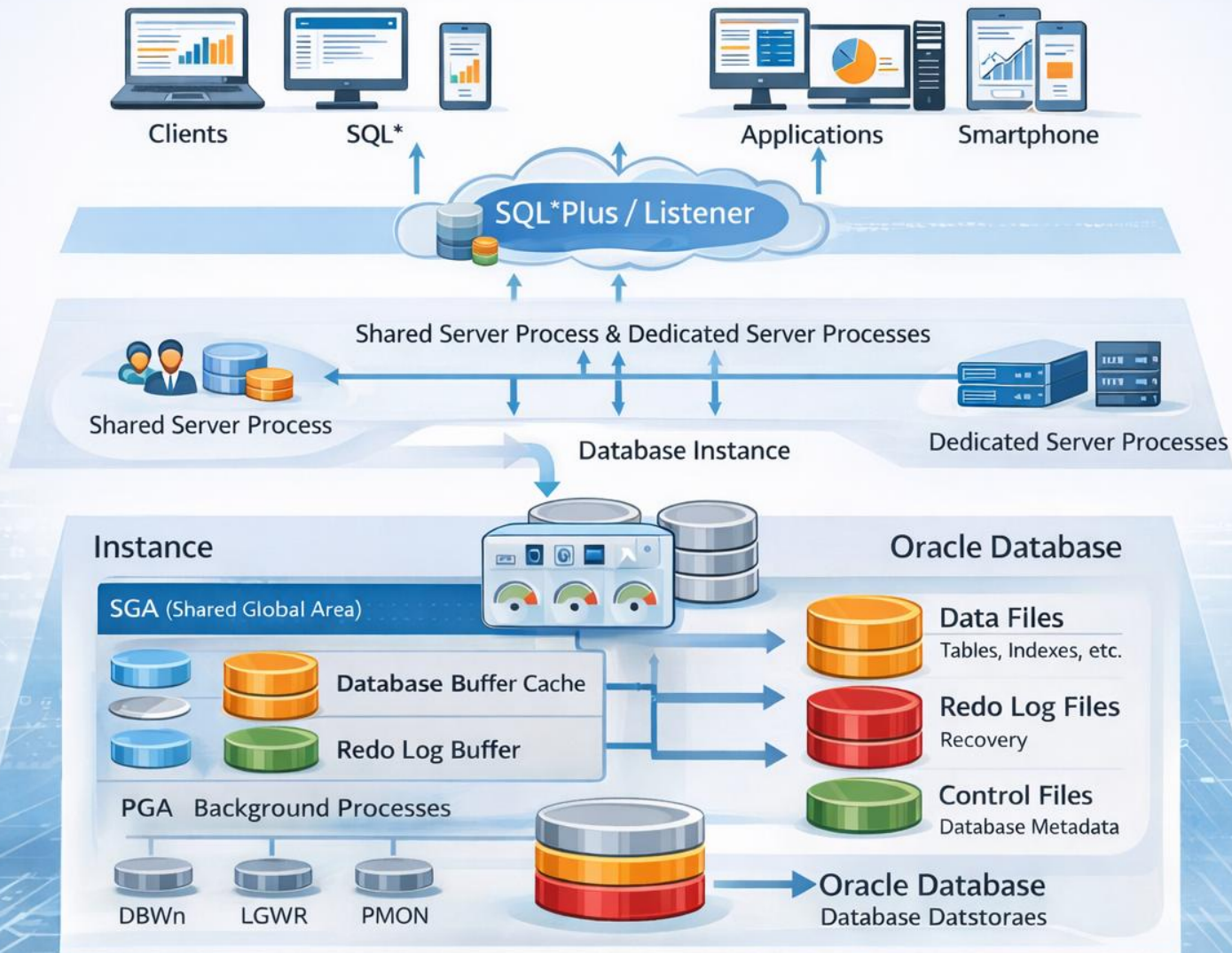
Évolution d'Oracle Database

Principales versions d'Oracle Database et dates de sortie

Version Oracle	Date de sortie	Remarques principales
Oracle 2	1979	Première version commerciale
Oracle 3	1983	Support des bases relationnelles
Oracle 4	1984	Transactions, SQL amélioré
Oracle 5	1985	Multi-utilisateurs, PL/SQL introduit
Oracle 6	1988	Gestion des transactions améliorée
Oracle 7	1992	Triggers, vues matérialisées
Oracle 8	1997	Support objet-relationnel
Oracle 8i	1999	"i" pour Internet, Java intégré
Oracle 9i	2001	Real Application Clusters (RAC)
Oracle 10g	2004	"g" pour Grid Computing
Oracle 11g	2007	Performance et automatisation
Oracle 12c	2013	"c" pour Cloud, multitenant
Oracle 18c	2018	Release annuelle, Cloud-ready
Oracle 19c	2019	Version LTS (Long Term Support)
Oracle 21c	2021	Innovations Big Data et AI

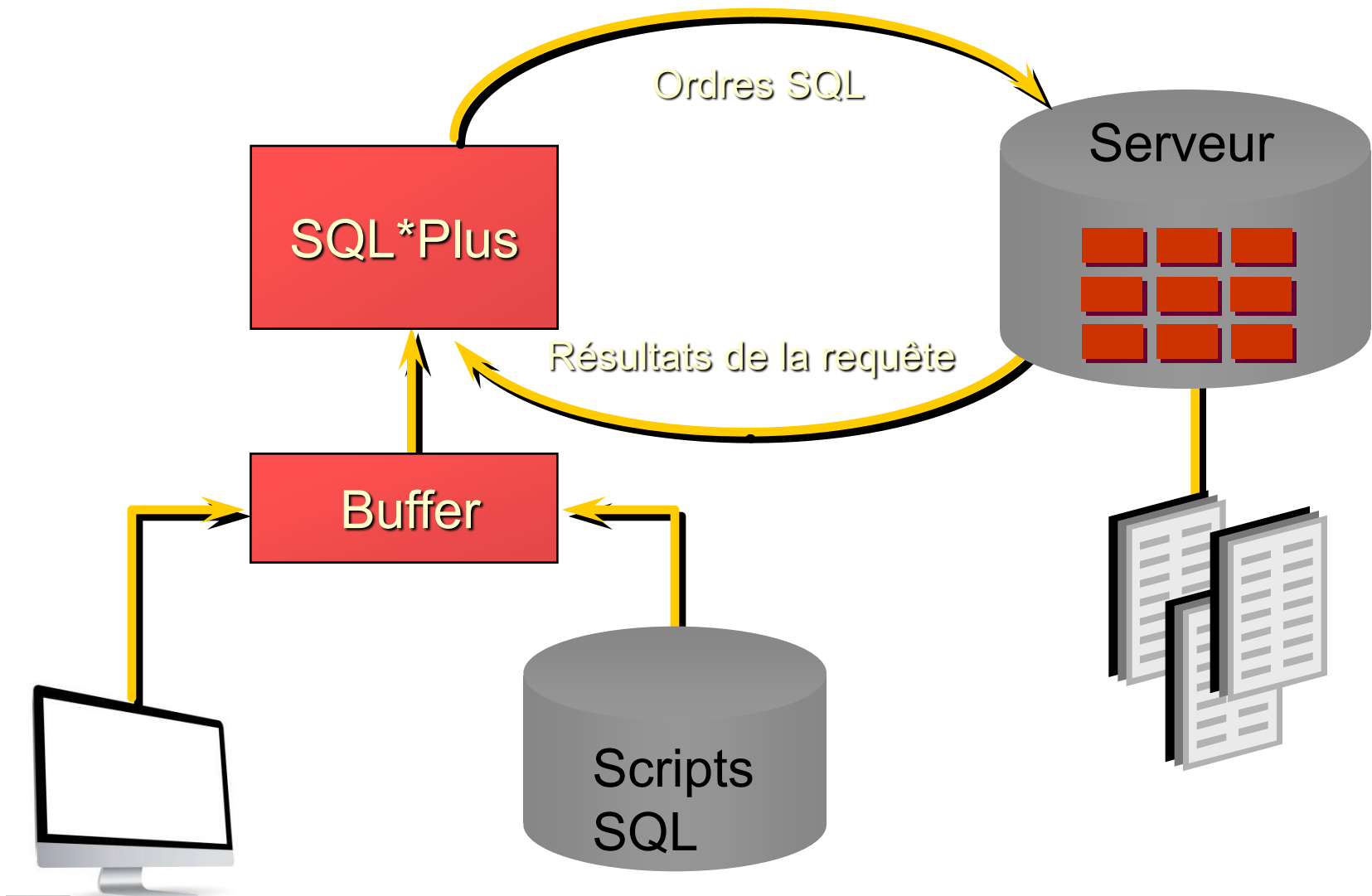


Architecture Oracle Database



SQL*PLUS

Interaction entre SQL et SQL*Plus



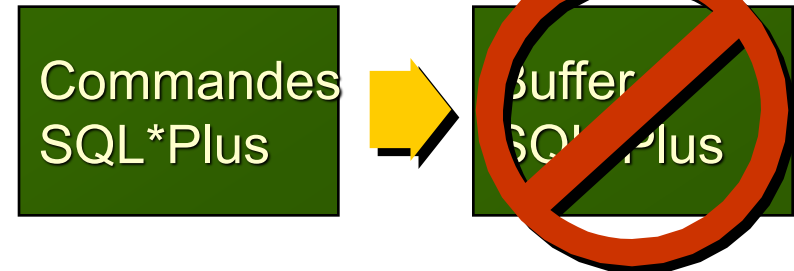
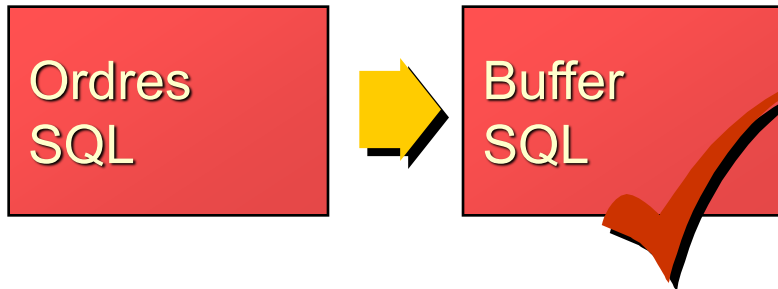
Comparatif entre Ordres SQL et Commandes SQL*Plus

SQL

- Un langage
- Standard ANSI
- Abréviation des mots-clés impossible
- Les ordres agissent sur le contenu et la définition des tables de la base de données

SQL*Plus

- Un environnement
- Produit propriétaire Oracle
- Abréviation des mots-clés possible
- Les commandes ne permettent d'agir ni sur le contenu, ni sur la définition des tables.



Buffer

- **A la saisie d'un ordre SQL, il est stocké dans un espace mémoire appelé buffer SQL où il reste jusqu'à la saisie d'un nouvel ordre.**

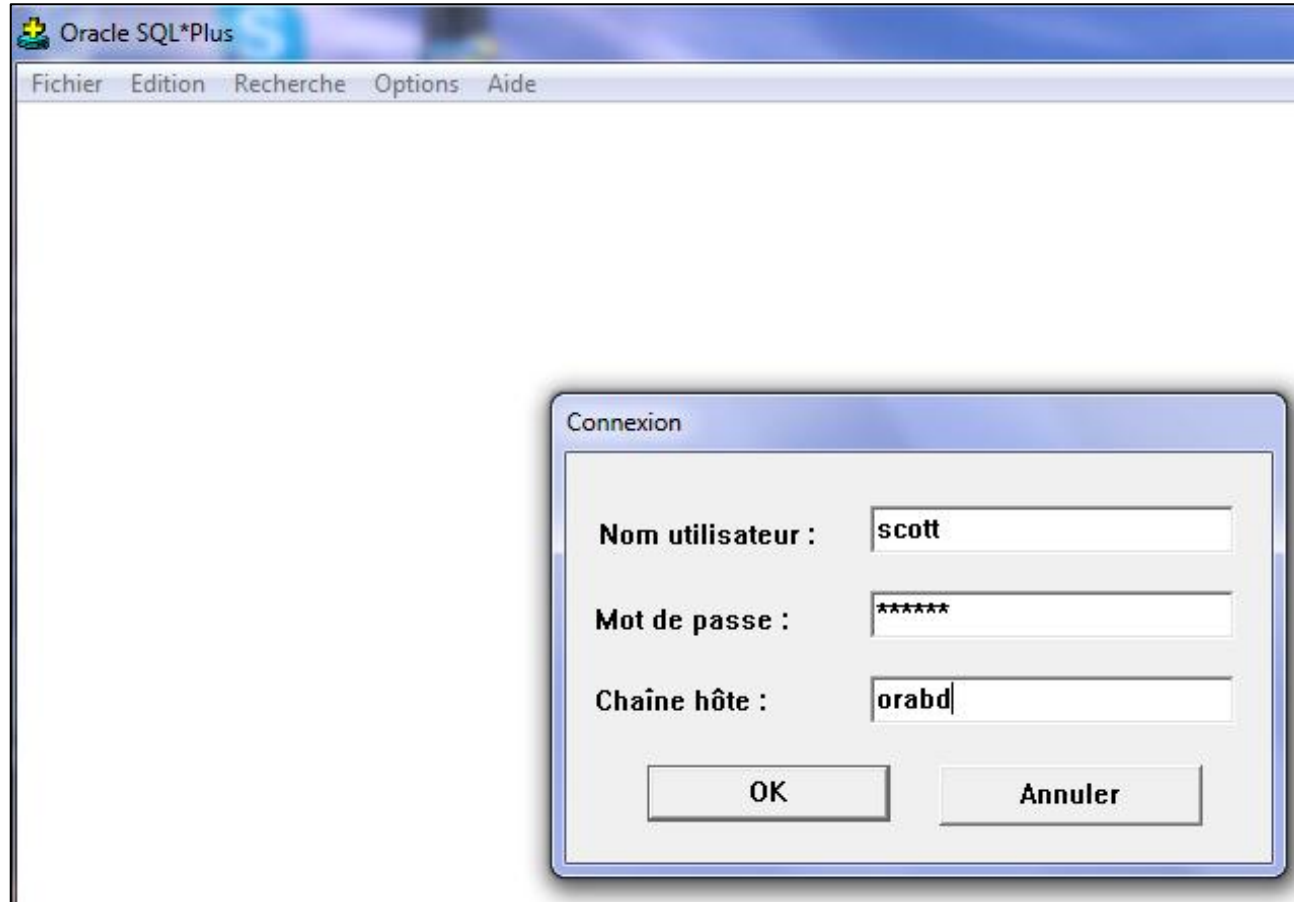
Présentation de SQL*Plus

L'environnement SQL*Plus permet :

- **Extraction, modification, ajout et suppression de données au moyen d'ordre SQL.**
- **Le formatage, le calcul, le stockage et l'impression de requête sous forme d'états.**
- **Création de fichiers scripts.**

Connexion à SQL*Plus

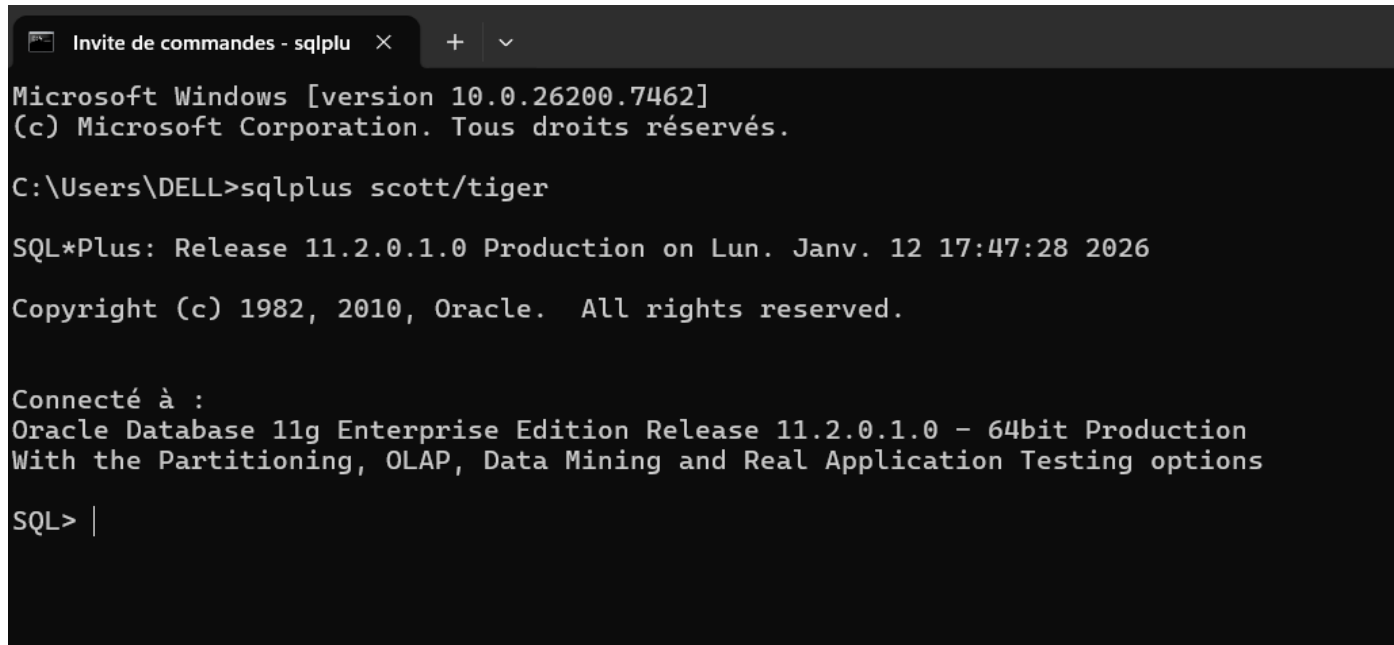
- Depuis l'environnement Windows :



Connexion à SQL*Plus

- Depuis une ligne de commande :

```
sqlplus [username[/password  
[@database]]]
```



```
Microsoft Windows [version 10.0.26200.7462]  
(c) Microsoft Corporation. Tous droits réservés.  
  
C:\Users\DELL>sqlplus scott/tiger  
  
SQL*Plus: Release 11.2.0.1.0 Production on Lun. Janv. 12 17:47:28 2026  
  
Copyright (c) 1982, 2010, Oracle. All rights reserved.  
  
Connecté à :  
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production  
With the Partitioning, OLAP, Data Mining and Real Application Testing options  
  
SQL> |
```


Connexion à SQL*Plus

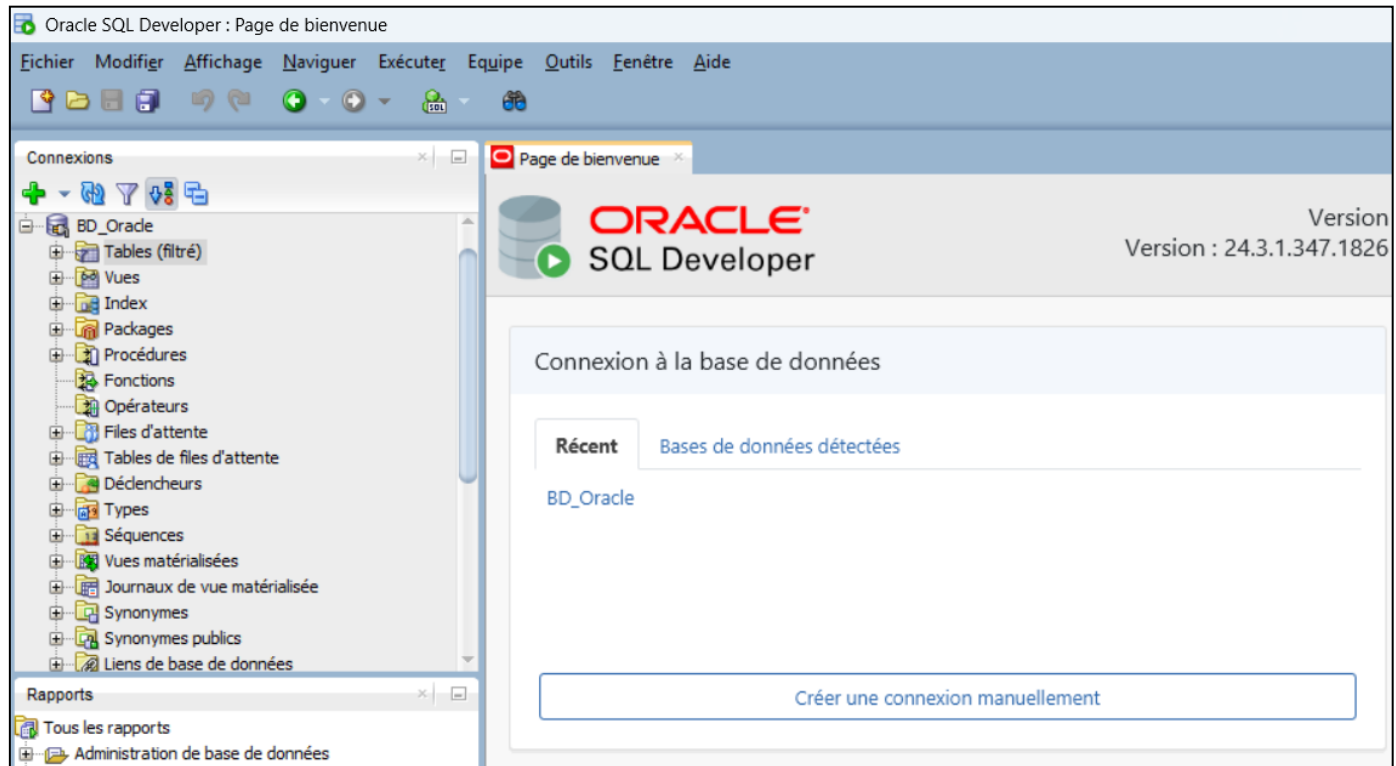
- *isqlplus* est une application en mode web qui permet de travailler avec une interface HTML de type sqlplus afin de faire des requêtes sur la base de données Oracle.



The screenshot shows the Oracle iSQL*Plus web interface. At the top left is the Oracle logo and the text "iSQL*Plus". At the top right is a circular help icon with a question mark and the word "Help" below it. Below the header is a "Login" link. The main form contains three input fields: "Username:", "Password:", and "Connection Identifier:". Below these fields is a "Login" button. The entire interface is enclosed in a thin black border.

Connexion à SQL*Plus

- **SQL Developer** est développé par Oracle permet de manipuler les requêtes SQL et les objets de la BD.



Commandes de base de SQL*Plus

- les commandes de SQL*Plus sont insensibles à la casse.
- Noter tous en majuscules pour respecter les conventions d'Oracle

Commandes d'Édition SQL*Plus

Commandes de session

- **CONN[ECT] user/password**
→ Se connecter à la base Oracle.
- **DISC[ONNECT]**
→ Se déconnecter de la base.
- **SHOW USE[R]**
→ Affiche l'utilisateur connecté.
- **SHOW ERR[ORS]**
→ Affiche les erreurs PL/SQL.
- **EXIT ou QUIT**
→ Quitte SQL*Plus.

Commandes d'Édition SQL*Plus

```
C:\Users\DELL>SQLPLUS /NOLOG
```

```
SQL*Plus: Release 11.2.0.1.0 Production on Ven. Janv. 16 18:05:25 2026
```

```
Copyright (c) 1982, 2010, Oracle. All rights reserved.
```

```
SQL> CONN HR/hr
```

```
Connecté.
```

```
SQL> SHOW USER
```

```
USER est "HR"
```

```
SQL> CONN SCOTT/tiger
```

```
Connecté.
```

```
SQL> SHOW USER
```

```
USER est "SCOTT"
```

```
SQL> EXIT
```

```
Déconnecté de Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production  
With the Partitioning, OLAP, Data Mining and Real Application Testing options
```

```
C:\Users\DELL>|
```

Commandes d'Édition SQL*Plus

Commandes d'édition du buffer SQL

- **A[PPEND] text**
→ Ajoute du texte à la fin de la ligne courante.
- **L[IST]**
→ Affiche le contenu du buffer SQL.
- **L[IST] n**
→ Affiche la ligne *n*.
- **L[IST] m n**
→ Affiche les lignes de *m* à *n*.
- **DEL**
→ Supprime la ligne courante du buffer.
- **DEL n**
→ Supprime la ligne numéro *n*.

Commandes d'Édition SQL*Plus

```
Invite de commandes - sqlplu  ×  +  ▾
SQL> select empno,ename,sal
  2  from emp
  3  where sal>1000
  4  ;
```

EMPNO	ENAME	SAL
7499	ALLEN	1600
7521	WARD	1250
7566	JONES	2975
7654	MARTIN	1250
7698	BLAKE	2850
7782	CLARK	2450
7788	SCOTT	3000
7839	KING	5000
7844	TURNER	1500
7876	ADAMS	1100
7902	FORD	3000

EMPNO	ENAME	SAL
7934	MILLER	1300

```
12 ligne(s) sélectionnée(s).

SQL> 3
  3* where sal>1000
SQL> a and sal<3000
  3* where sal>1000and sal<3000
SQL>
```

```
SQL> del 4
SQL> l
  1  select empno,ename,sal,deptno
  2  from emp
  3* where sal >1000 and sal<3000
```

Commandes d'Édition SQL*Plus

Commandes d'édition du buffer SQL

- **DEL m n**
→ Supprime les lignes de *m* à *n*.
- **I[INPUT]**
→ Insère une ou plusieurs lignes dans le buffer.
- **CL[EAR] BUFF[ER]**
→ Efface complètement le buffer SQL.

Commandes d'Édition SQL*Plus

```
SQL> l
  1  select empno,ename,sal,deptno
  2*  from emp
SQL> I
  3  order by ename;
```

EMPNO	ENAME	SAL	DEPTNO
7876	ADAMS	1100	20
7499	ALLEN	1600	30
7698	BLAKE	2850	30
7782	CLARK	2450	10
7902	FORD	3000	20
7900	JAMES	950	30
7566	JONES	2975	20
7839	KING	5000	10
7654	MARTIN	1250	30
7934	MILLER	1300	10
7788	SCOTT	3000	20

EMPNO	ENAME	SAL	DEPTNO
7369	SMITH	800	20
120	test		
7844	TURNER	1500	30
7521	WARD	1250	30

15 ligne(s) sélectionnée(s).

```
SQL> l
  1  select empno,ename,sal,deptno
  2  from emp
  3* order by ename
SQL> |
```

```
SQL> l
  1  select empno,ename,sal,deptno
  2  from emp
  3* order by ename
SQL> CL BUFF
buffer effacé
SQL> l
SP2-0223: Pas de lignes dans la mémoire tampon SQL.
SQL> |
```

Commandes d'Édition SQL*Plus

Commandes d'exécution

- **R[UN]**
→ Exécute le contenu du buffer SQL.
- **/**
→ Exécute la dernière commande SQL ou PL/SQL.
- **EXEC[UTE] procédure**
→ Exécute une procédure ou fonction PL/SQL.

Commandes d'Édition SQL*Plus

Commandes d'affichage et formatage

- **SET LINESI[ZE] n**

→ Définit la largeur d'affichage des lignes.

- **SET PAGE[SIZE] n**

→ Définit le nombre de lignes par page.

Commandes d'information

- **DESC[RIBE] table**

→ Affiche la structure d'une table

Commandes d'Édition SQL*Plus

```
SQL> desc emp
```

Nom	NULL ?	Type
EMPNO	NOT NULL	NUMBER(4)
ENAME		VARCHAR2(10)
JOB		VARCHAR2(9)
MGR		NUMBER(4)
HIREDATE		DATE
SAL		NUMBER(7,2)
COMM		NUMBER(7,2)
DEPTNO		NUMBER(2)

Commandes d'Édition SQL*Plus

Commandes de fichiers

- **SPO[OL] fichier**
→ Enregistre la sortie dans un fichier.
- **SPO[OL] OFF**
→ Arrête l'enregistrement.
- **SAV[E] fichier**
→ Sauvegarde le buffer SQL dans un fichier.
- **STA[RT] fichier**
→ Exécute un script SQL.
- **@fichier**
→ Exécute un script SQL.

Commandes d'Édition SQL*Plus

```
Invite de commandes - sqlplu x + v
Microsoft Windows [version 10.0.26200.7623]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\DELL>sqlplus /nolog

SQL*Plus: Release 11.2.0.1.0 Production on Ven. Janv. 16 17:36:20 2026

Copyright (c) 1982, 2010, Oracle. All rights reserved.

SQL> conn scott/tiger
Connecté.
SQL> spool F:\test.sql
SQL> desc dept
  Nom                                     NULL ?   Type
-----
DEPTNO                                NOT NULL NUMBER(2)
DNAME                                VARCHAR2(14)
LOC                                  VARCHAR2(13)

SQL> desc emp
  Nom                                     NULL ?   Type
-----
EMPNO                                NOT NULL NUMBER(4)
ENAME                                VARCHAR2(10)
JOB                                  VARCHAR2(9)
MGR                                  NUMBER(4)
HIREDATE                             DATE
SAL                                  NUMBER(7,2)
COMM                                NUMBER(7,2)
DEPTNO                               NUMBER(2)
```

Fichier	Modifier	Affichage
SQL> desc dept		
Nom	NULL ?	Type

DEPTNO	NOT NULL	NUMBER(2)
DNAME		VARCHAR2(14)
LOC		VARCHAR2(13)
SQL> desc emp		
Nom	NULL ?	Type

EMPNO	NOT NULL	NUMBER(4)
ENAME		VARCHAR2(10)
JOB		VARCHAR2(9)
MGR		NUMBER(4)
HIREDATE		DATE
SAL		NUMBER(7,2)
COMM		NUMBER(7,2)
DEPTNO		NUMBER(2)
SQL> select *		
2		from emp
3		;

```
7934 MILLER      test      7782 23/01/82    1300
10

15 ligne(s) sélectionnée(s).

SQL> spool off
```

test.sql

Commandes d'Édition SQL*Plus

```
SQL> save F:\essai.sql  
Création de file F:\essai.sql  
SQL> |
```

Fichier	Modifier	Affichage
select * from emp /		

essai.sql

```
SQL> @F:\essai.sql
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
120	test					
7369 20	SMITH	CLERK	7902	17/12/80	800	
7499 30	ALLEN	SALESMAN	7698	20/02/81	1600	300

Variables d'environnement

- **Les variables d'environnement permettent de paramétrer une session SQL*Plus. L'affectation d'une variable s'opère à l'aide de la commande SET.**
- **L'état d'une variable d'environnement est donné par la commande SHOW.**

Schéma de SCOTT

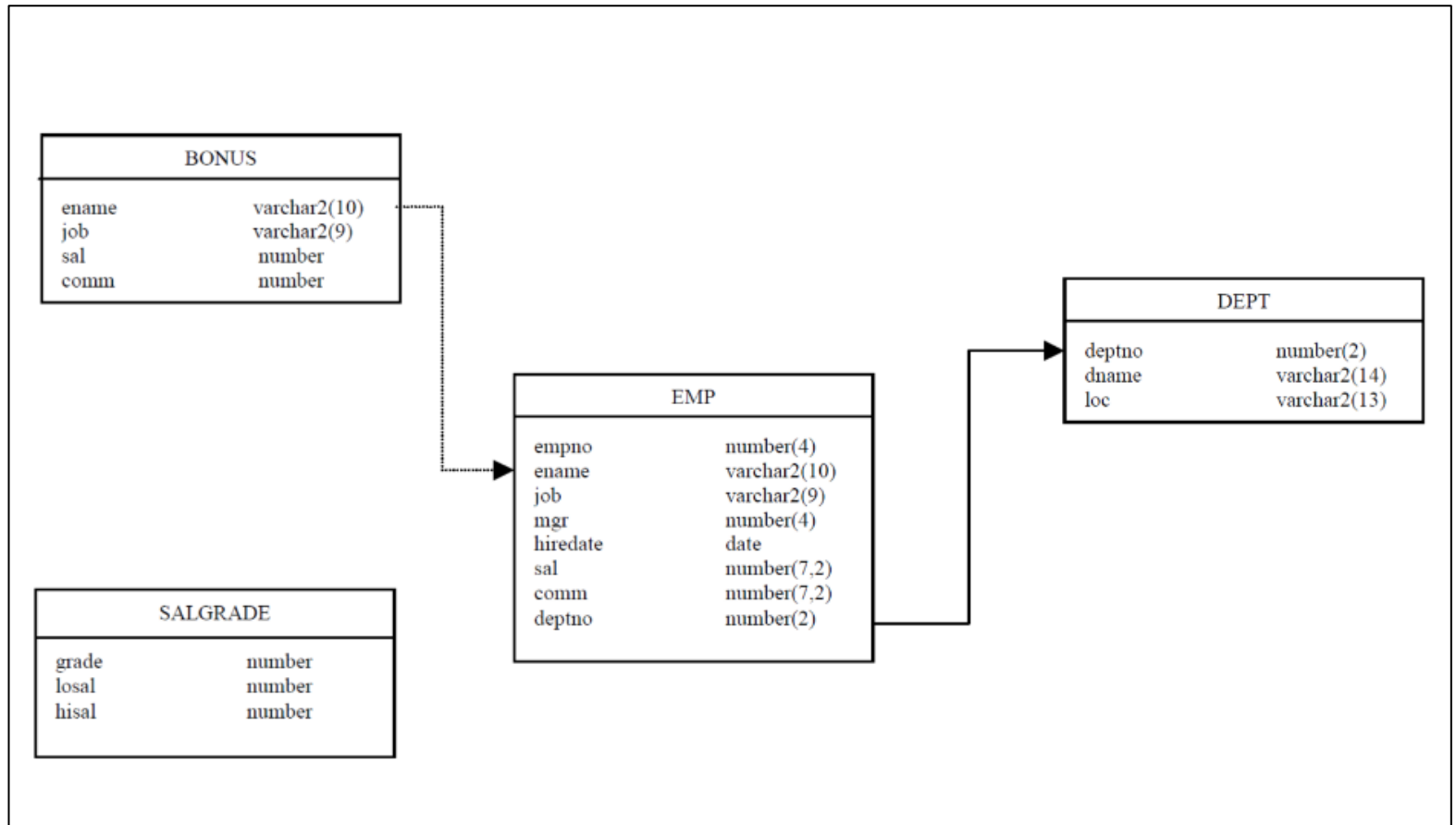
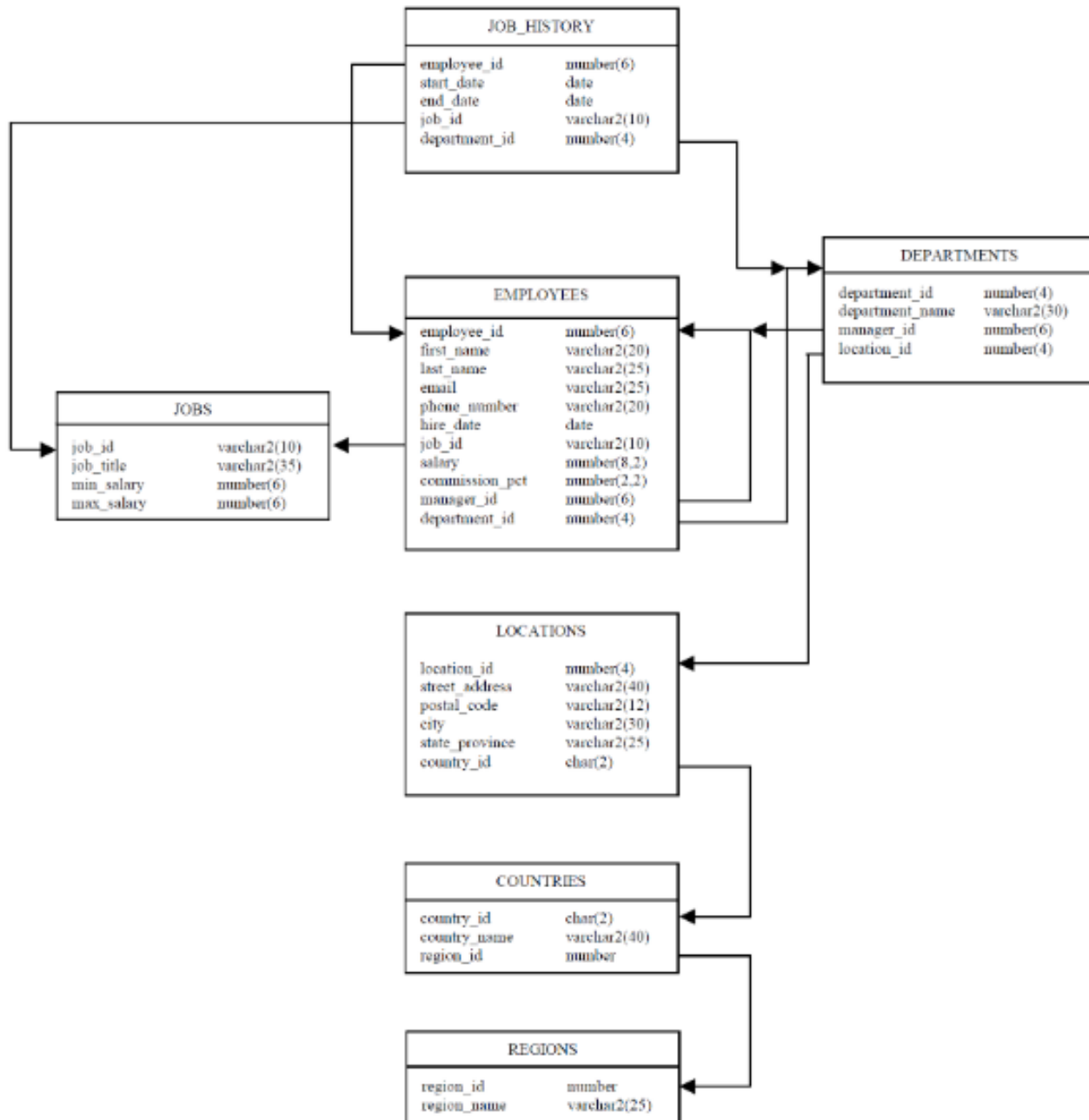


Schéma de HR



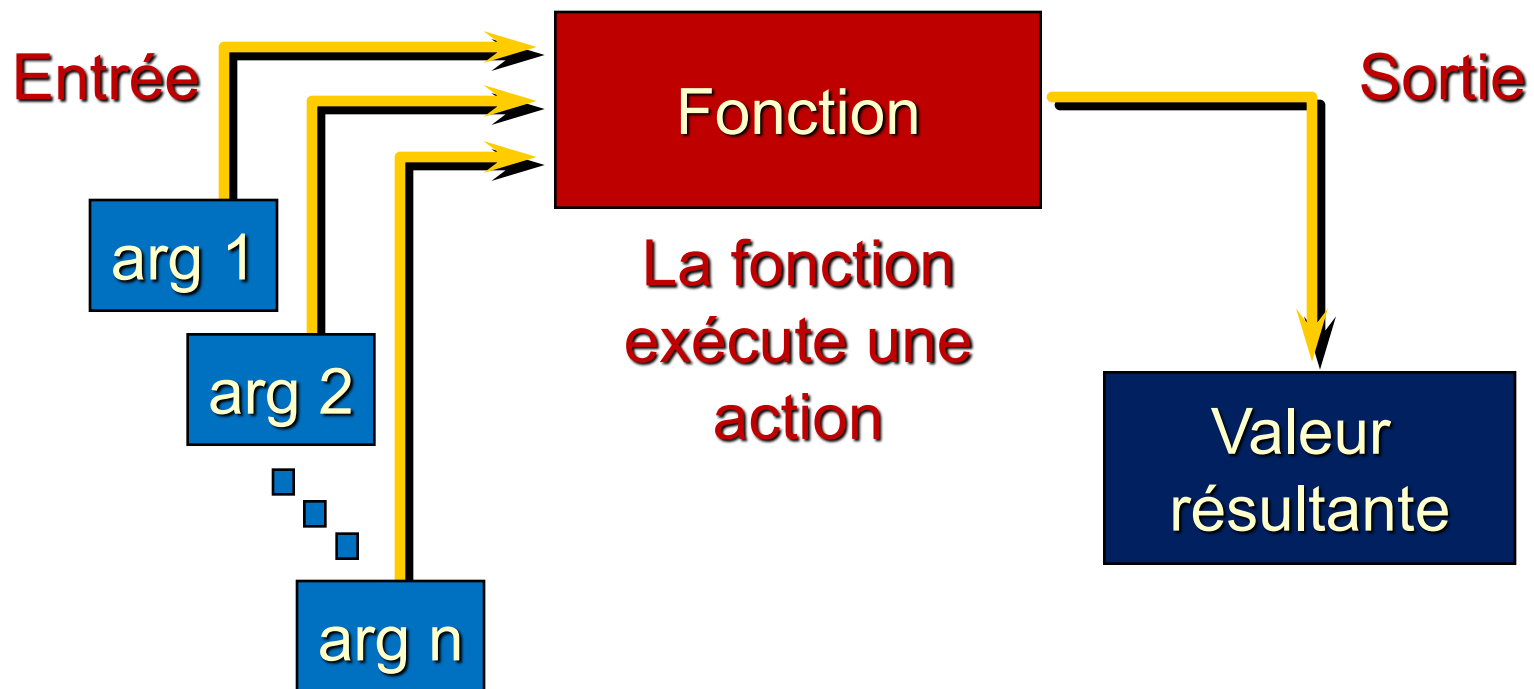
Fonctions Mono-Ligne

Sanae MAZOUZ

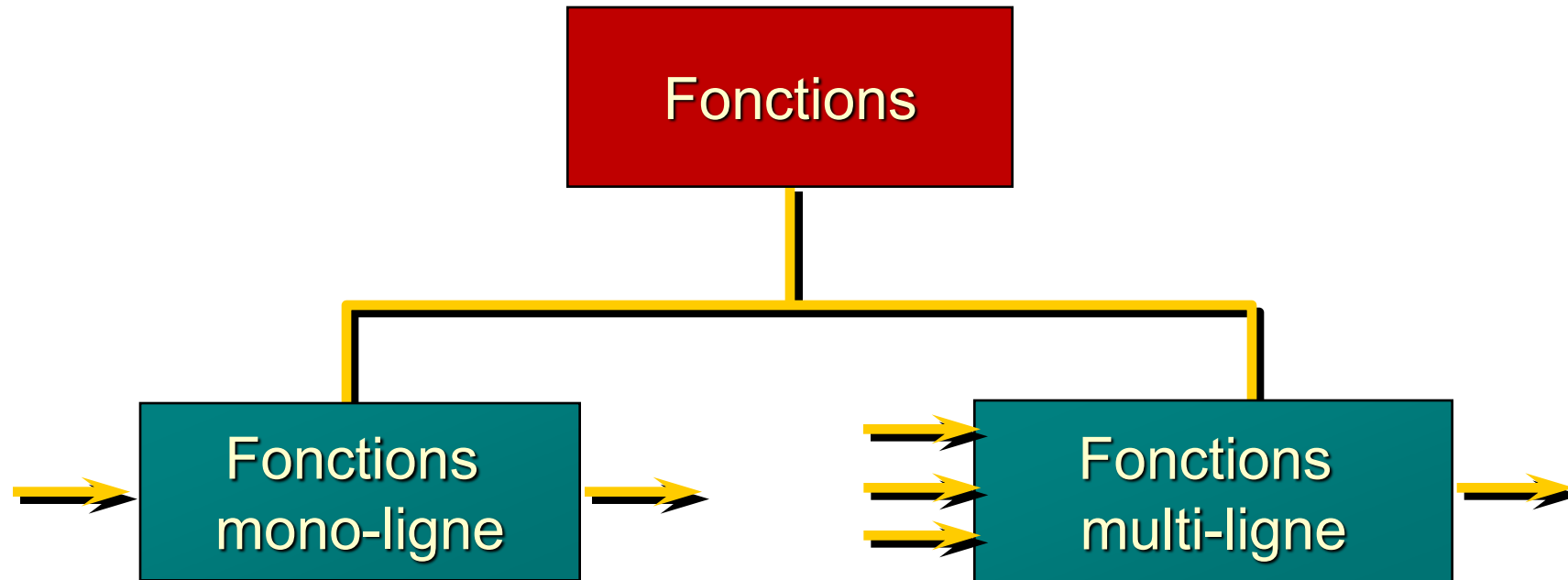
Qu'est ce qu'une fonction ?

- Une fonction est une expression d'un type de données spécifique qui fait partie d'une instruction utilisée pour calculer une valeur .

Fonctions SQL



Types de Fonctions SQL

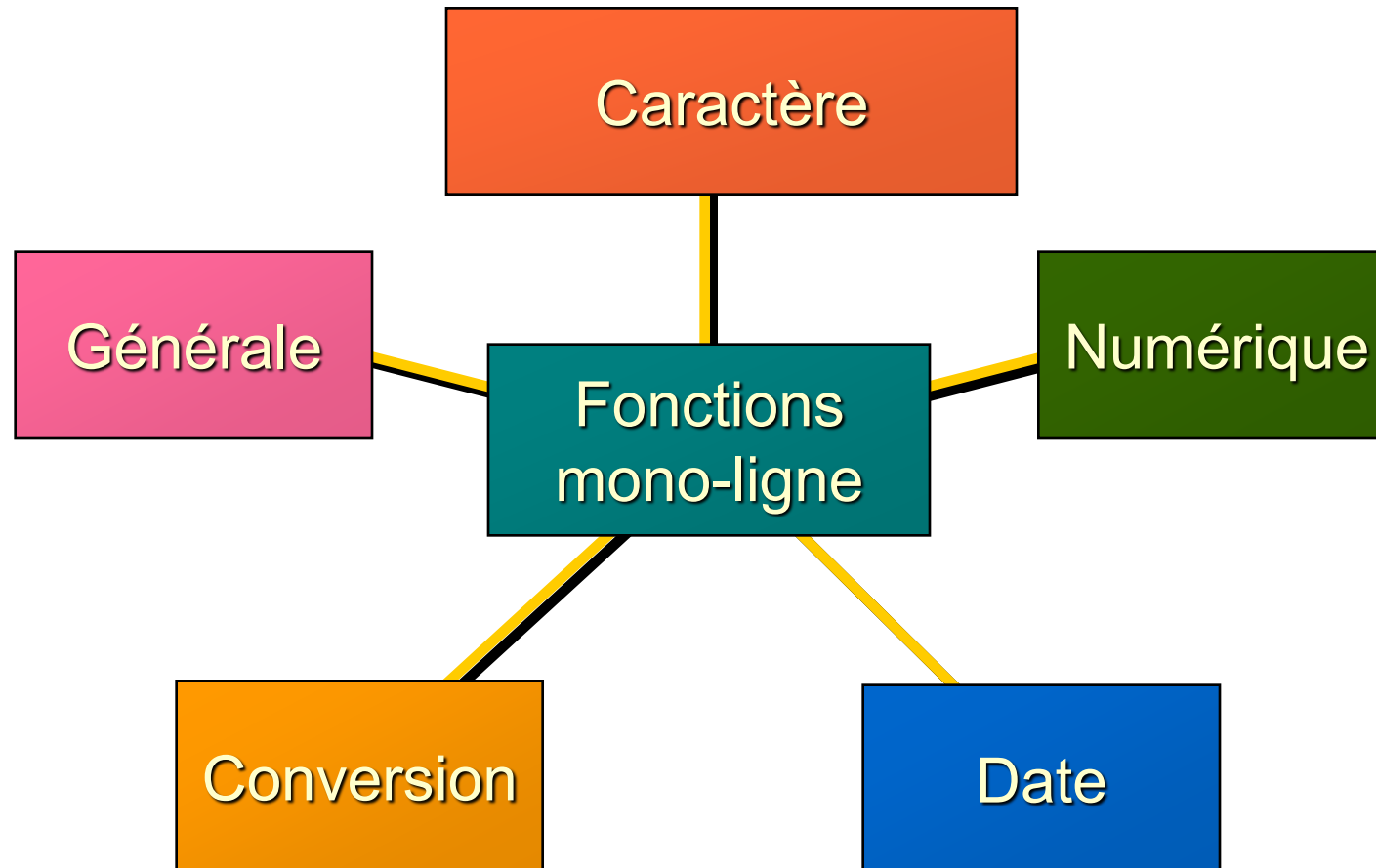


Fonctions Mono-Ligne

- Manipulent des éléments de données
- Acceptent des arguments et ramènent une valeur
- Agissent sur chacune des lignes rapportées
- Ramènent un seul résultat par ligne
- Peuvent modifier les types de données
- Peuvent être imbriquées

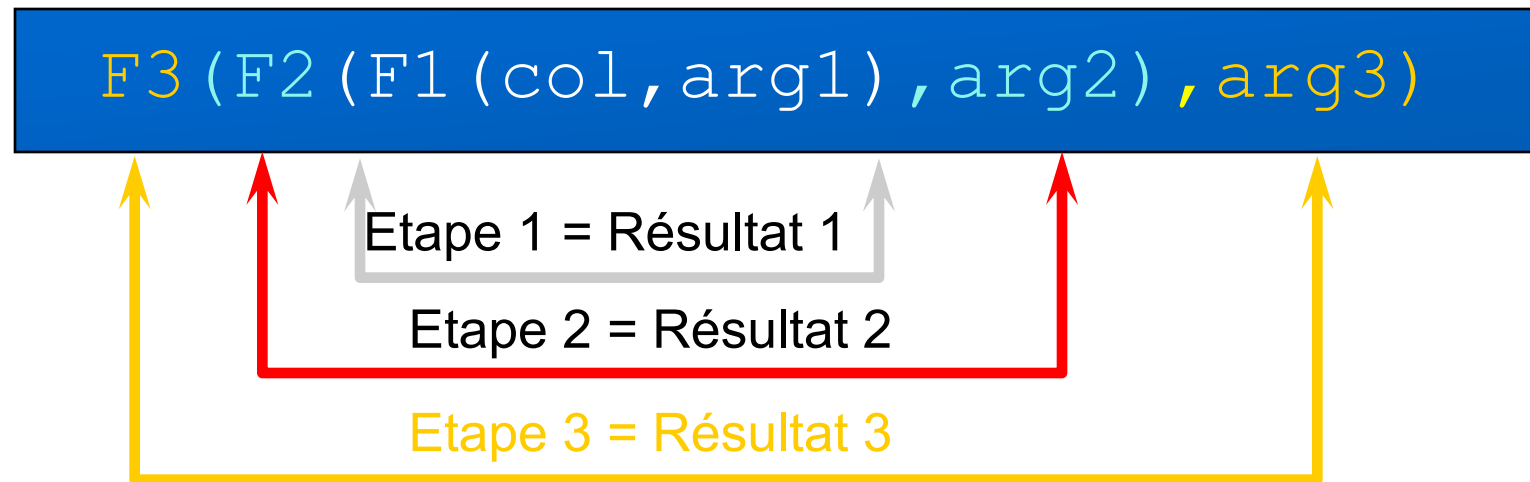
```
function_name (column|expression, [arg1, arg2,...])
```

Fonctions Mono-Ligne

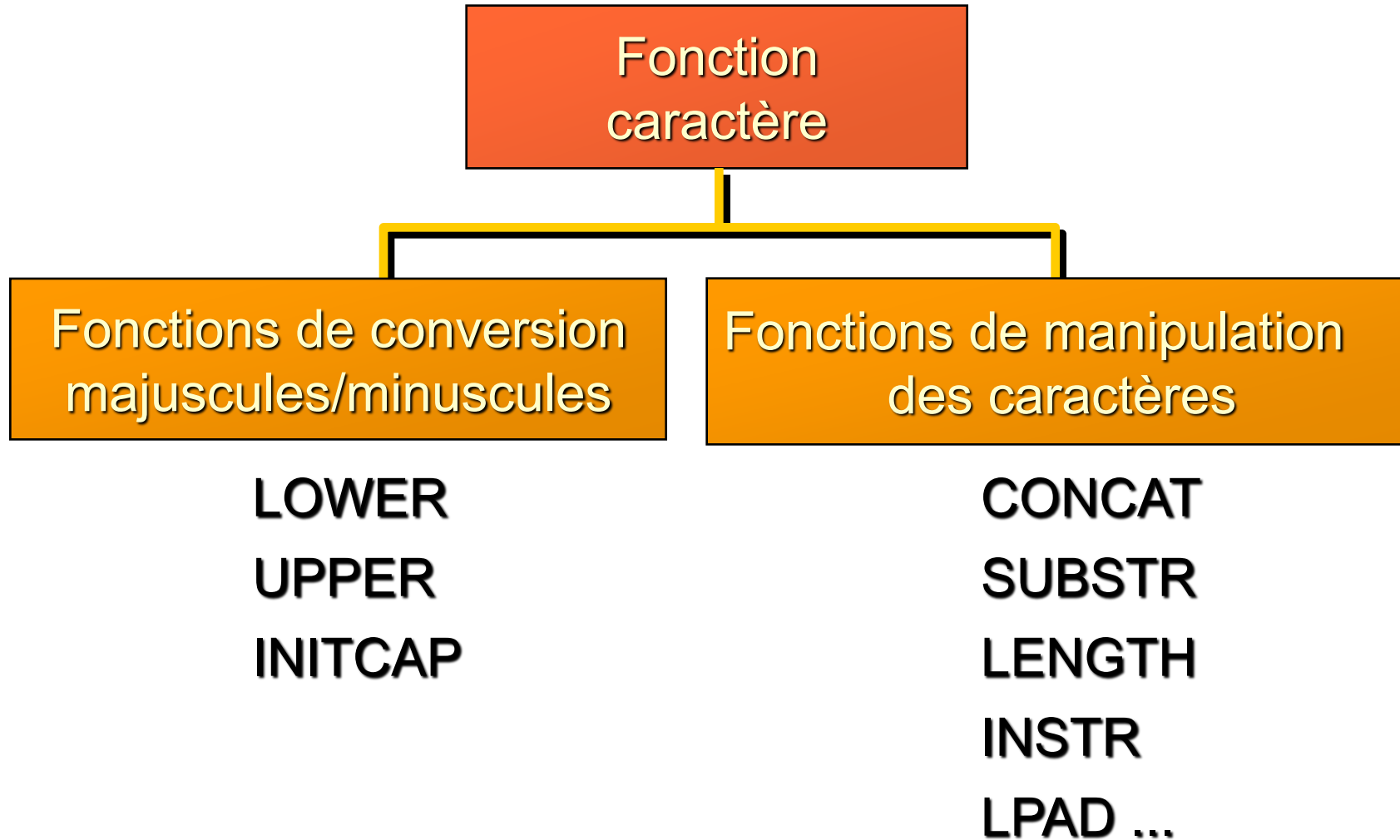


Imbrication des Fonctions

- Le niveau d'imbrication des fonctions mono-ligne est illimité
- Les fonctions imbriquées sont évaluées de l'intérieur vers l'extérieur



Fonctions Caractère



Fonctions de Conversion Majuscules/Minuscules

Fonction	Résultat
LOWER('Cours SQL')	cours sql
UPPER('Cours SQL')	COURS SQL
INITCAP('Cours SQL')	Cours Sql

Utilisation des Fonctions de Conversion Majuscules/Minuscules

- Afficher le matricule, le nom et le numéro de département de l'employé Blake.

```
SQL> SELECT empno, ename, deptno  
2 FROM emp  
3 WHERE ename = 'blake';
```

no rows selected

```
SQL> SELECT empno, ename, deptno  
2 FROM emp  
3 WHERE LOWER(ename) = 'blake';
```

EMPNO	ENAME	DEPTNO
7698	BLAKE	30

Fonctions de Manipulation des Caractères

- Manipulation de chaînes de caractères

Fonction	Résultat
CONCAT('Une', 'Chaîne')	UneChaîne
SUBSTR('Chaîne',1,3)	Cha
LENGTH('Chaîne')	6
INSTR('Chaîne', 'a')	3
LPAD(sal,10,'*')	*****5000

DUAL

- La table DUAL appartient à l'utilisateur SYS,
- Tous les utilisateurs peuvent y accéder.
- Elle contient une seule colonne, DUMMY, et une seule ligne contenant la valeur X.
- La table DUAL est utile lorsque vous souhaitez ramener une valeur une seule fois, par exemple, la valeur d'une constante, d'une pseudo-colonne ou d'une expression qui ne dépend pas d'une table de données utilisateur.

Utilisation des Fonctions de Manipulation des Caractères

```
SQL> SELECT ename, CONCAT (CONCAT(ename, ' '), job) JOB,  
2 LENGTH(ename), INSTR(ename, 'A')  
3 FROM emp  
4 WHERE SUBSTR(job, 1, 5) = 'SALES';
```

ENAME	JOB	LENGTH(ENAME)	INSTR(ENAME, 'A')
MARTIN	MARTIN SALESMAN	6	2
ALLEN	ALLEN SALESMAN	5	1
TURNER	TURNER SALESMAN	6	0
WARD	WARD SALESMAN	4	2

Fonctions Numériques

ROUND : Arrondit la valeur à la précision spécifiée

ROUND(45.926, 2)  **45.93**

TRUNC : Tronque la valeur à la précision spécifiée

TRUNC(45.926, 2)  **45.92**

MOD : Ramène le reste d'une division

MODE(1600,300)  **100**

Utilisation de la Fonction ROUND

- Affichage de la valeur 45.923 arrondie au centième, à 0 décimale et à la dizaine supérieure.

```
SQL> SELECT ROUND(45.923, 2), ROUND(45.923, 0),  
2          ROUND(45.923, -1)  
3 FROM DUAL;
```

ROUND(45.923, 2)	ROUND(45.923, 0)	ROUND(45.923, -1)
----- 45.92	----- 46	----- 50

Utilisation de la Fonction TRUNC

- **Affichage de la valeur 45.923 tronquée au centième, à 0 décimale et à la dizaine.**

```
SQL> SELECT TRUNC (45.923, 2), TRUNC (45.923),  
2          TRUNC (45.923, -1)  
3 FROM DUAL;
```

TRUNC (45.923, 2)	TRUNC (45.923)	TRUNC (45.923, -1)	
-----	-----	-----	
45.92	45	40	

Utilisation de la Fonction MOD

- Calculer le reste de la division salaire par commission pour l'ensemble des employés ayant un poste de vendeur.

```
SQL> SELECT  ename, sal, comm, MOD(sal, comm)
2  FROM      emp
3  WHERE     job = 'SALESMAN';
```

ENAME	SAL	COMM	MOD (SAL, COMM)
MARTIN	1250	1400	1250
ALLEN	1600	300	100
TURNER	1500	0	1500
WARD	1250	500	250

Autres Fonctions Numériques

ABS(x) : Valeur absolue de x.

CEIL(n) : Plus petit entier supérieur ou égal à n.

SIGN(n) : Si $n < 0$, -1; si $n = 0$, 0; si $n > 0$, 1.

FLOOR(n) : Plus grand entier inférieur ou égal à n.

Utilisation des Dates

- Oracle stocke les dates dans un format numérique interne : siècle, année, mois, jour, heures, minutes, secondes.
- Le format de date par défaut est **DD-MON-YY**.
- La fonction **SYSDATE** ramène la date et l'heure courante.
- **DUAL** est une table factice qu'on peut utiliser pour visualiser **SYSDATE**.

Opérations Arithmétiques sur les Dates

- Ajout ou soustraction d'un nombre à une date pour obtenir un résultat de type *date*.
- Soustraction de deux dates afin de déterminer le **nombre** de jours entre ces deux dates.
- Ajout **d'un nombre d'heures** à une date en divisant le nombre d'heures par 24.

Utilisation d'Opérateurs Arithmétiques avec les Dates

Afficher le nom et le nombre de semaines d'ancienneté de tous les employés du département 10.

```
SQL> SELECT ename, (SYSDATE-hiredate)/7 WEEKS  
2 FROM emp  
3 WHERE deptno = 10;
```

ENAME	WEEKS
KING	830.93709
CLARK	853.93709
MILLER	821.36566

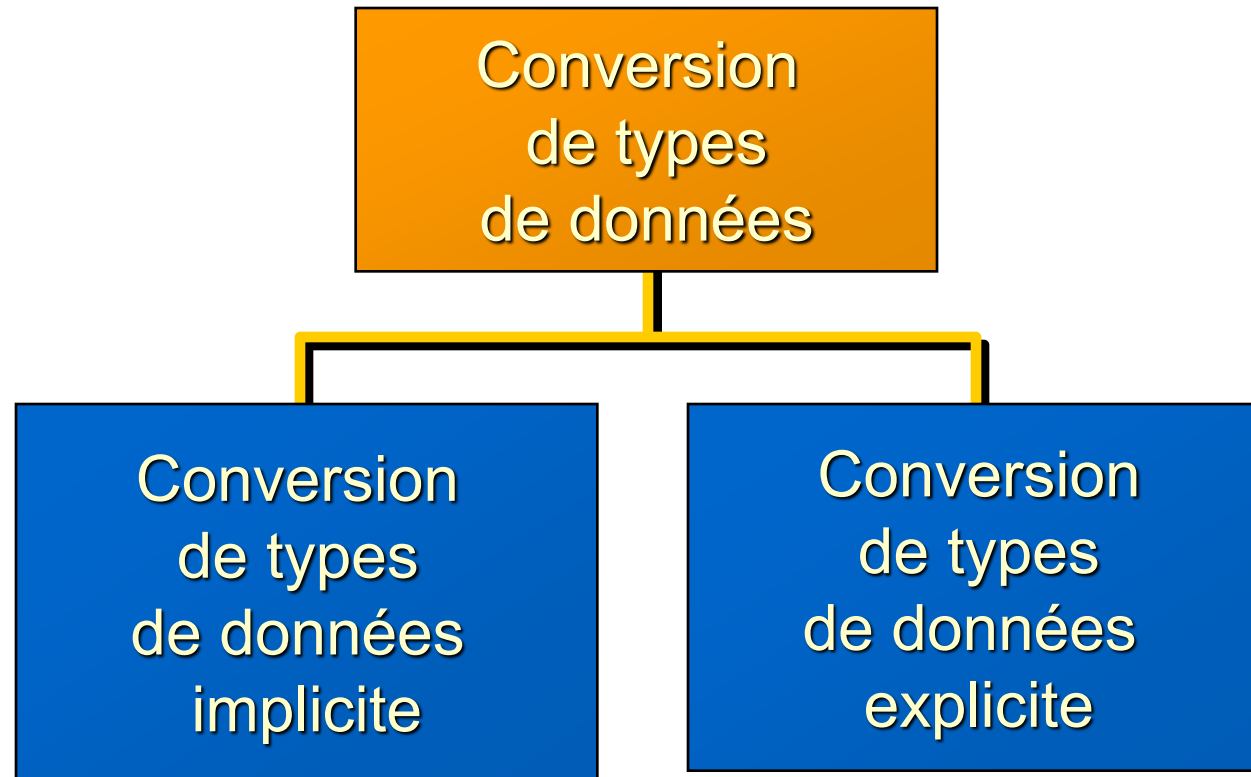
Fonctions Date

FONCTION	DESCRIPTION
MONTHS_BETWEEN(d1,d2)	Nombre de mois situés entre deux dates
ADD_MONTHS(date, n)	Ajoute des mois calendaires à une date
NEXT_DAY(date,'char')	Jour qui suit la date spécifiée
LAST_DAY(date)	Dernier jour du mois
ROUND(date [, 'fmt'])	Arrondit une date
TRUNC (date [, 'fmt'])	Tronque une date

Utilisation des Fonctions Date

- MONTHS_BETWEEN (DATE '2025-12-31', DATE '2025-01-01')
→ '11,9677419'
- ADD_MONTHS (DATE '2025-01-15', 3) → '15/04/25 '
- NEXT_DAY(SYSDATE, 'MONDAY') → '19/01/26 '
- LAST_DAY(SYSDATE) → '31/01/26 '

Fonctions de Conversion

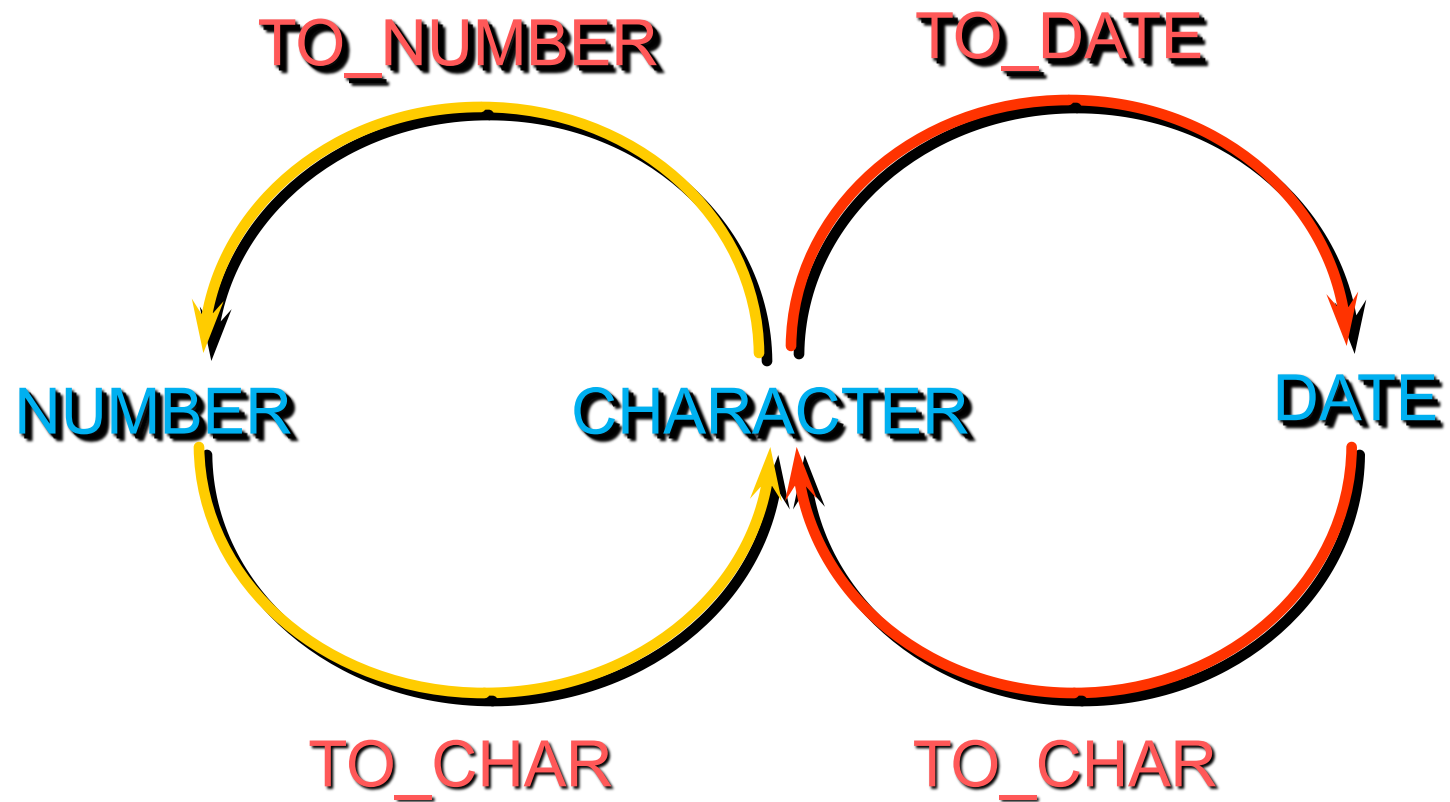


Conversion de Types de Données Implicite

- Pour les affectations, Oracle effectue automatiquement les conversions suivantes

De	Vers
VARCHAR2 ou CHAR	NUMBER
VARCHAR2 ou CHAR	DATE
NUMBER	VARCHAR2
DATE	VARCHAR2

Conversion de Types de Données Explicite



Utilisation de la Fonction TO_CHAR avec les Dates

```
TO_CHAR(date, 'fmt')  

```

- **Le modèle de format :**
 - Doit être placé entre simples quotes et différencie les majuscules et minuscules.
 - Peut inclure tout élément valide de format date
 - Comporte un élément *fm* qui supprime les espaces de remplissage ou les zéros de tête
 - Est séparé de la valeur date par une virgule

Modèles de Format Date

YYYY	Année exprimée avec 4 chiffres
YEAR	Année exprimée en toutes lettres
MM	Mois exprimé avec 2 chiffres
MONTH	Mois exprimé en toutes lettres
DY	3 premières lettres du nom du jour
DAY	Jour exprimé en toutes lettres

Modèles de Format pour les Dates

- Les éléments horaires formatent la partie horaire de la date.

HH24:MI:SS AM	15:45:32 PM
---------------	-------------

- Pour ajouter des chaînes de caractères, les placer entre guillemets.

DD "of" MONTH	12 of OCTOBER
---------------	---------------

- Différents suffixes existent pour les nombres.

ddspth	fourteenth
--------	------------

Utilisation de la Fonction TO_CHAR avec les Dates

```
SQL> SELECT ename,  
2         TO_CHAR(hiredate, 'fmDD Month YYYY') HIREDATE  
3 FROM emp;
```

ENAME	HIREDATE
-----	-----
KING	17 November 1981
BLAKE	1 May 1981
CLARK	9 June 1981
JONES	2 April 1981
MARTIN	28 September 1981
ALLEN	20 February 1981
...	

14 rows selected.

Utilisation de la Fonction TO_CHAR avec les Nombres

```
TO_CHAR(number, 'fmt')  

```

- Utilisez les formats suivants avec TO_CHAR pour afficher un nombre sous la forme d'une chaîne de caractère.

9	Représente un chiffre
0	Force l'affichage du zéro
\$	Place un signe dollar flottant
L	Utilise le symbole monétaire local flottant
.	Imprime un point décimal
,	Imprime un séparateur de milliers

Utilisation de la Fonction TO_CHAR avec les Nombres

```
SQL> SELECT TO_CHAR(sal, '$99,999') SALARY  
2 FROM emp  
3 WHERE ename = 'SCOTT';
```

```
SALARY  
-----  
$3,000
```

Fonctions TO_NUMBER et TO_DATE

- Conversion d'une chaîne de caractères en format numérique avec la fonction **TO_NUMBER**

```
TO_NUMBER(char)
```

- Conversion d'une chaîne de caractères en format date avec la fonction **TO_DATE**

```
TO_DATE(char [, 'fmt'])
```

Fonction NVL

Convertit une valeur NULL en une valeur réelle

- **Fonctionne avec les données de type date, caractère et numérique.**
- **Les types de données doivent correspondre**
 - **NVL(comm,0)**
 - **NVL(hiredate,'01-JAN-97')**
 - **NVL(job,'No Job Yet')**

Utilisation de la Fonction NVL

```
SQL> SELECT ename, sal, comm, (sal*12)+NVL(comm,0)
       2 FROM emp;
```

ENAME	SAL	COMM	(SAL*12)+NVL(COMM,0)
KING	5000		60000
BLAKE	2850		34200
CLARK	2450		29400
JONES	2975		35700
MARTIN	1250	1400	16400
ALLEN	1600	300	19500
...			

14 rows selected.

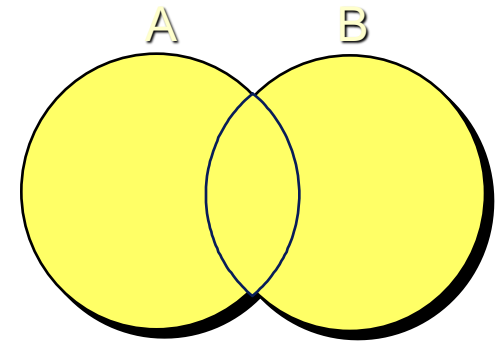
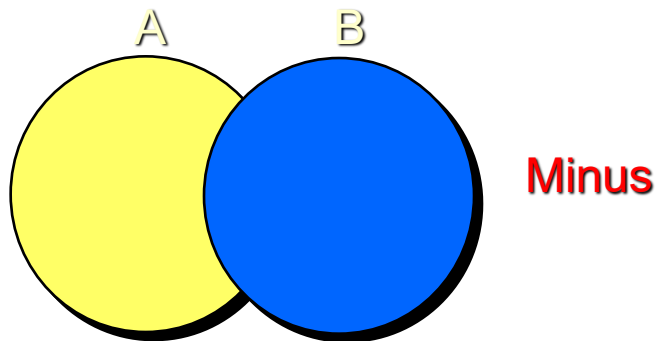
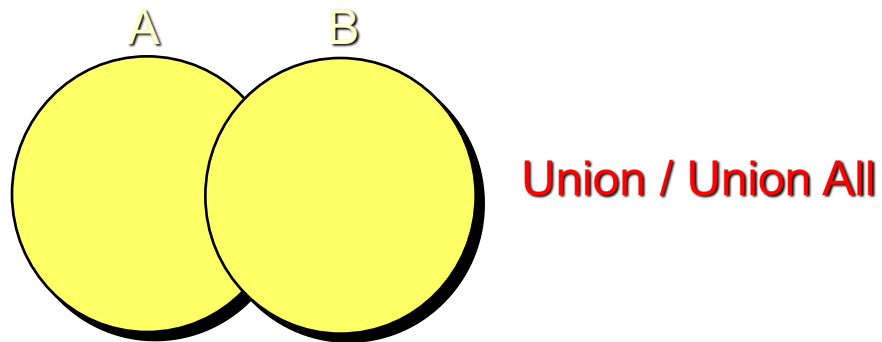
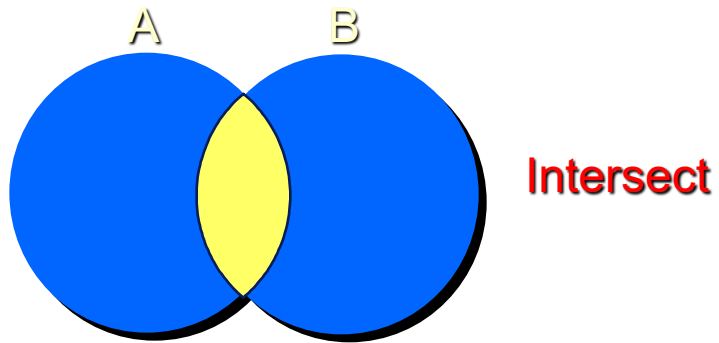
Imbrication des Fonctions

```
SQL> SELECT  ename,  
2           NVL (TO_CHAR(mgr), 'No Manager')  
3 FROM      emp  
4 WHERE     mgr IS NULL;
```

ENAME	NVL (TO_CHAR (MGR) , 'NOMANAGER')
-----	-----
KING	No Manager

Opérateurs Ensemblistes

Opérateurs Ensemblistes



Tables Utilisées dans ce Chapitre

EMP

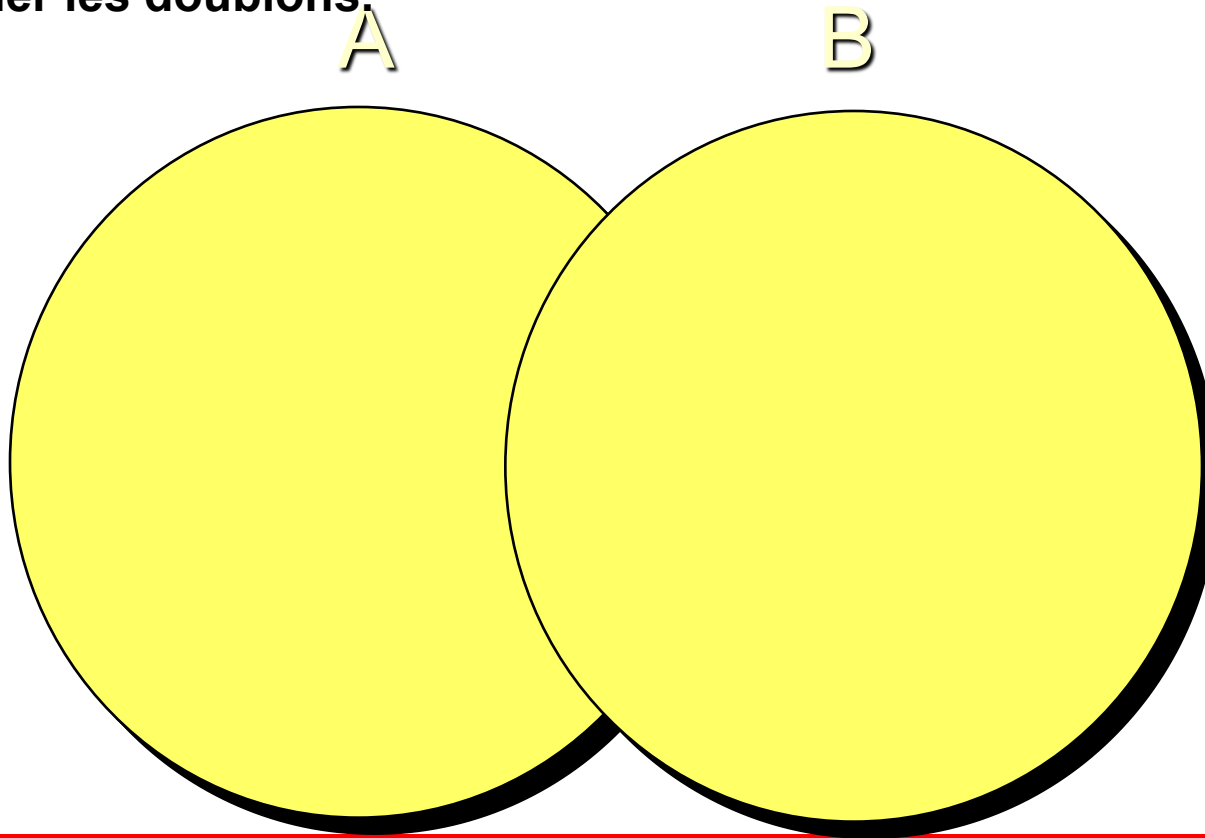
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
DEPTNO						
10	7839	KING		PRESIDENT	17-NOV-81	5000
30	7698	BLAKE	7839	MANAGER	01-MAY-81	2850
10	7782	CLARK	7839	MANAGER	09-JUN-81	1500
20	7566	JONES	7839	MANAGER	02-APR-81	2975
30	7654	MARTIN		SALESMAN		
...	7499	ALLEN		SALESMAN		

EMP_HISTORY

EMPID	NAME	TITLE	DATE_OUT
DEPTID			
20	6087	SPENCER	OPERATOR
10	6185	VANDYKE	MANAGER
20	6235	BALFORD	CLERK
...	7788	SCOTT	ANALYST

UNION

- Cet opérateur combine le résultat de deux requêtes.
- Il permet de ramener toutes les lignes issues de plusieurs requêtes et d'éliminer les doublons.



Utilisation de l'Opérateur UNION

- Affichez le nom, le poste et le département de tous les employés.

```
SQL> SELECT ename, job, deptno  
2 FROM emp  
3 UNION  
4 SELECT name, title, deptid  
5 FROM emp_history;
```

ENAME	JOB	DEPTNO
ADAMS	CLERK	30
ALLEN	SALESMAN	30
ALLEN	SALESMAN	20
BALFORD	CLERK	20
BLAKE	MANAGER	30
...		

20 rows selected.

Utilisation de l'Opérateur UNION

- Affichez le nom, le poste et le département et le salaire de tous les employés.

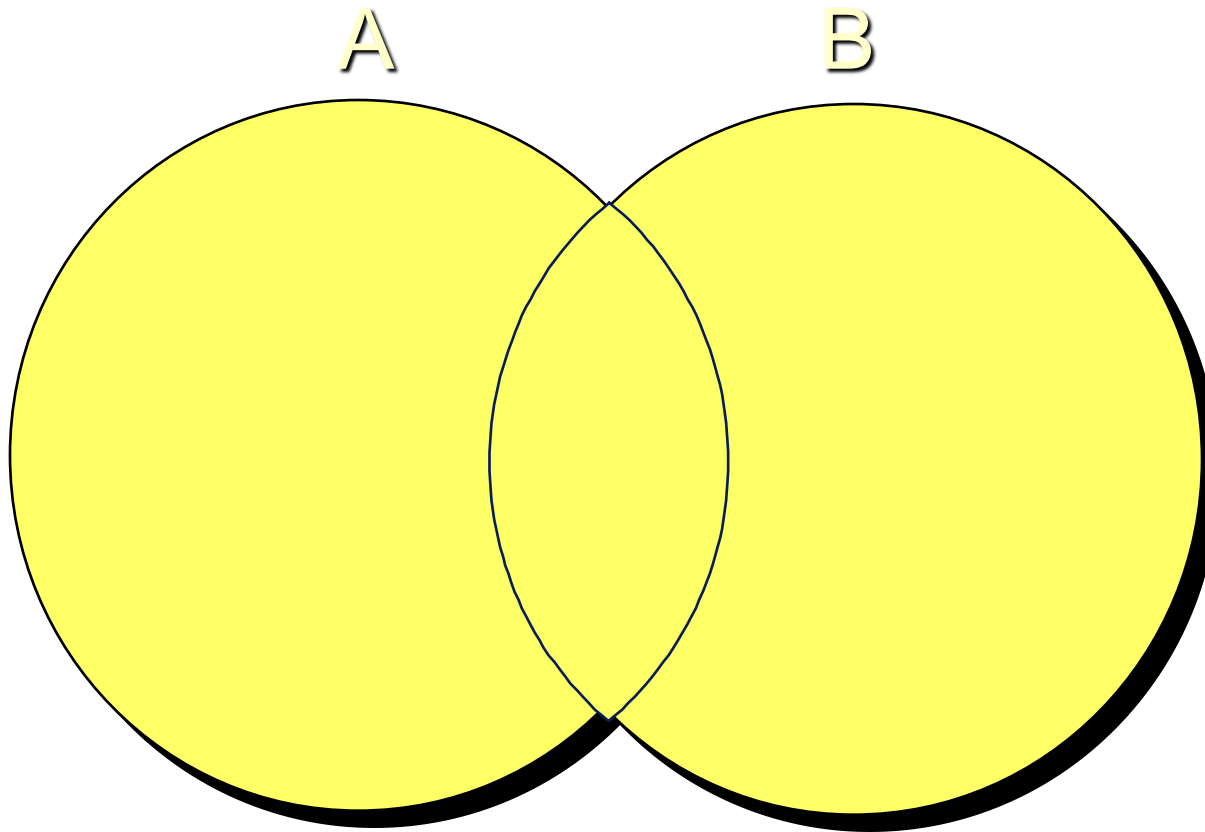
```
SQL> SELECT ename, job, deptno, sal
2  FROM emp
3  UNION
4  SELECT name, title, deptid, 0
5  FROM emp_history;
```

ENAME	JOB	DEPTNO	SAL
ADAMS	CLERK	30	1100
ALLEN	SALESMAN	30	0
ALLEN	SALESMAN	20	1600
BALFORD	CLERK	20	0
BLAKE	MANAGER	30	
...			

20 rows selected.

UNION ALL

- Cet opérateur permet de ramener toutes les lignes issues de plusieurs requêtes.
- Contrairement à l'opérateur UNION, les doublons ne sont pas éliminés et le résultat n'est pas trié par défaut.



Utilisation de l'Opérateur UNION ALL

- Affichez le nom, le numéro et le poste de tous les employés.

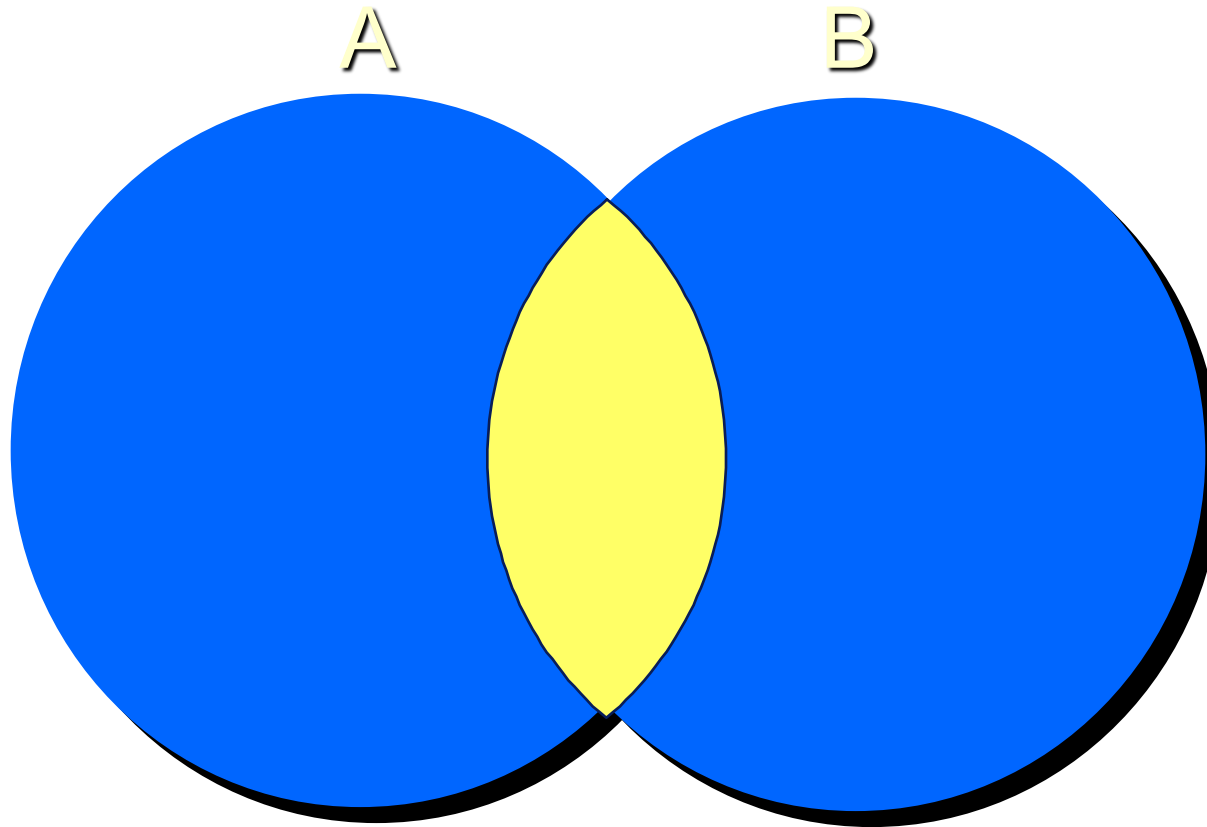
```
SQL> SELECT ename, empno, job
      2 FROM emp
      3 UNION ALL
      4 SELECT name, empid, title
      5 FROM emp_history;
```

ENAME	EMPNO	JOB
KING	7839	PRESIDENT
BLAKE	7698	MANAGER
CLARK	7782	MANAGER
CLARK	7782	MANAGER
MARTIN	7654	SALESMAN
...		

23 rows selected.

INTERSECT

- Cet opérateur permet de ramener toutes les lignes communes aux deux requêtes.
- Le nombre de colonnes et les types de données des deux colonnes doivent être identiques dans les deux ordres SELECT.



Utilisation de l'Opérateur INTERSECT

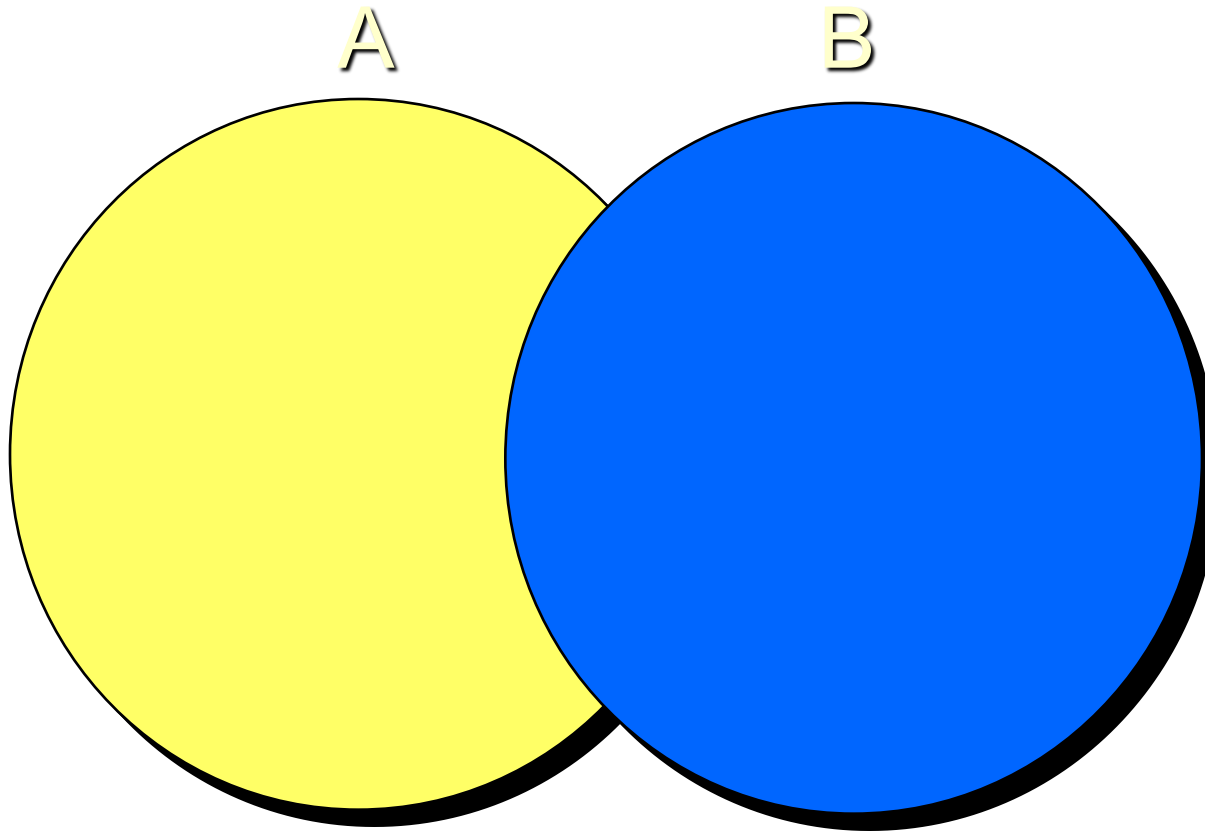
- Affichez les différents noms, numéros et postes des employés présents dans les tables EMP et EMP_HISTORY.

```
SQL> SELECT ename, empno, job
      2 FROM emp
      3 INTERSECT
      4 SELECT name, empid, title
      5 FROM emp_history;
```

ENAME	EMPNO	JOB
-----	-----	-----
ALLEN	7499	SALESMAN
CLARK	7782	MANAGER
SCOTT	7788	ANALYST

MINUS

- Cet opérateur ramène les lignes retournées par la première requête, qui ne le sont pas par la seconde (premier ordre **SELECT** moins le second).
- Le nombre de colonnes et les types de données des colonnes doivent être identiques dans les deux ordres **SELECT**.



MINUS

Affichez le nom, le numéro et le poste de tous les employés ayant quitté la société.

```
SQL> SELECT name, empid, title
2   FROM    emp_history
3   MINUS
4   SELECT  ename, empno, job
5   FROM    emp;
```

NAME	EMPID	TITLE
BALFORD	6235	CLERK
BRIGGS	7225	PAY CLERK
JEWELL	7001	ANALYST
SPENCER	6087	OPERATOR
...		

6 rows selected.

Règles des Opérateurs Ensemblistes

- Les expressions citées dans la clause **SELECT** doivent être égales en nombre et avoir des données du même type.
- Les doublons sont automatiquement éliminés, sauf avec l'opérateur **UNION ALL**.
- Les noms de colonnes apparaissant dans le résultat sont ceux de la première requête.
- Par défaut, le résultat est trié par ordre croissant, sauf avec l'opérateur **UNION ALL**.
- Utilisez des parenthèses pour modifier la séquence d'exécution.

Les Fonctions de Groupe

Fonctions de Groupe

- Les fonctions de groupe agissent sur des groupes de lignes et donnent un résultat par groupe.

EMP

DEPTNO	SAL
10	2450
10	5000
10	1300
20	800
20	1100
20	3000
20	3000
20	2975
30	1600
30	2850
30	1250
30	950
30	1500
30	1250

"salaire maximum
de la table EMP"

MAX (SAL)

5000

Types de Fonctions de Groupe

- **AVG ([DISTINCT|ALL]n)**
- **COUNT ({ *|[DISTINCT|ALL]expr})**
- **MAX ([DISTINCT|ALL]expr)**
- **MIN ([DISTINCT|ALL]expr)**
- **STDDEV ([DISTINCT|ALL]n)**
- **SUM ([DISTINCT|ALL]n)**
- **VARIANCE ([DISTINCT|ALL]n)**

Fonctions AVG et SUM

- **AVG et SUM s'utilisent avec des données numériques.**

```
SQL> SELECT  AVG(sal), MAX(sal),  
2           MIN(sal), SUM(sal)  
3 FROM      emp  
4 WHERE     job LIKE 'SALES%';
```

AVG (SAL)	MAX (SAL)	MIN (SAL)	SUM (SAL)
-----	-----	-----	-----
1400	1600	1250	5600

Fonctions MIN et MAX

- **MIN et MAX s'utilisent avec tous types de données.**

```
SQL> SELECT MIN(hiredate), MAX(hiredate)
2 FROM emp;
```

MIN(HIRED	MAX(HIRED
-----	-----
17-DEC-80	12-JAN-83

Utilisation de la Fonction COUNT

- **COUNT(*)** ramène le nombre de lignes d'une table.

```
SQL> SELECT COUNT (*)  
2 FROM emp  
3 WHERE deptno = 30;
```

```
COUNT (*)
```

```
-----
```

```
6
```

Utilisation de la Fonction COUNT

- **COUNT(*expr*)** ramène le nombre de lignes non NULL.

```
SQL> SELECT COUNT(comm)
      2 FROM emp
      3 WHERE deptno = 30;
```

```
COUNT (COMM)
```

```
-----
```

```
4
```

Fonctions de Groupe et Valeurs NULL

- Les fonctions de groupe ignorent les valeurs NULL des colonnes.

```
SQL> SELECT AVG(comm)
      2 FROM emp;
```

```
AVG (COMM)
-----
      550
```

Utilisation de la Fonction NVL avec les Fonctions de Groupe

- La fonction NVL force la prise en compte des valeurs NULL dans les fonctions de groupe.

```
SQL> SELECT AVG (NVL (comm, 0) )  
2 FROM emp;
```

```
AVG (NVL (COMM, 0) )  
-----  
157.14286
```

Création de Groupes de Données

EMP

DEPTNO	SAL
10	2450
10	5000
10	1300
20	800
20	1100
20	3000
20	3000
20	2975
30	1600
30	2850
30	1250
30	950
30	1500
30	1250

2916.6667

2175

1566.6667

"salaire moyen pour
chaque
département de la table
EMP"

DEPTNO	AVG (SAL)
10	2916.6667
20	2175
30	1566.6667

Création de Groupes de Données : la Clause GROUP BY

```
SELECT      column, group_function  
FROM        table  
[WHERE      condition]  
[GROUP BY   group_by_expression]  
[ORDER BY   column];
```

- **Divisez une table en groupes de lignes avec la clause GROUP BY.**

Utilisation de la Clause GROUP BY

- La clause **GROUP BY** doit inclure toutes les colonnes de la liste **SELECT** qui ne figurent pas dans des fonctions de groupe.

```
SQL> SELECT deptno, AVG(sal)
2 FROM emp
3 GROUP BY deptno;
```

DEPTNO	AVG (SAL)
10	2916.6667
20	2175
30	1566.6667

Utilisation de la Clause GROUP BY

- **La colonne citée en GROUP BY ne doit pas nécessairement figurer dans la liste SELECT.**

```
SQL> SELECT      AVG(sal)
      2 FROM      emp
      3 GROUP BY deptno;
```

```
AVG(SAL)
-----
2916.6667
2175
1566.6667
```


Regroupement sur Plusieurs Colonnes

EMP

DEPTNO	JOB	SAL
10	MANAGER	2450
10	PRESIDENT	5000
10	CLERK	1300
20	CLERK	800
20	CLERK	1100
20	ANALYST	3000
20	ANALYST	3000
20	MANAGER	2975
30	SALESMAN	1600
30	MANAGER	2850
30	SALESMAN	1250
30	CLERK	950
30	SALESMAN	1500
30	SALESMAN	1250

"somme des salaires de la table EMP pour chaque poste, regroupés par département"

DEPTNO	JOB	SUM (SAL)
10	CLERK	1300
10	MANAGER	2450
10	PRESIDENT	5000
20	ANALYST	6000
20	CLERK	1900
20	MANAGER	2975
30	CLERK	950
30	MANAGER	2850
30	SALESMAN	5600

Utilisation de la Clause GROUP BY sur Plusieurs Colonnes

```
SQL> SELECT    deptno, job, sum(sal)
  2  FROM      emp
  3  GROUP BY  deptno, job;
```

DEPTNO	JOB	SUM(SAL)
10	CLERK	1300
10	MANAGER	2450
10	PRESIDENT	5000
20	ANALYST	6000
20	CLERK	1900
...		

9 rows selected.

Erreurs d'Utilisation des Fonctions de Groupe dans une Requête

- Toute colonne ou expression de la liste SELECT autre qu'une fonction de groupe, doit être incluse dans la clause GROUP BY.

```
SQL> SELECT deptno, COUNT(ename)
2 FROM emp;
```

```
SELECT deptno, COUNT(ename)
      *
ERROR at line 1:
ORA-00937: not a single-group group function
```

Colonne manquante dans la clause GROUP BY

Erreurs d'utilisation des Fonctions de Groupe dans une Requête

- Vous ne pouvez utiliser la clause WHERE pour limiter les groupes.
- Utilisez la clause HAVING.

```
SQL> SELECT      deptno, AVG(sal)
  2  FROM          emp
  3  WHERE         AVG(sal) > 2000
  4  GROUP BY     deptno;
```

```
WHERE AVG(sal) > 2000
```

```
*
```

```
ERROR at line 3:
```

```
ORA-00934: group function is not allowed here
```

N'utilisez pas la clause WHERE pour limiter les groupes

Exclusion de Groupes

EMP

DEPTNO	SAL

10	2450
10	5000
10	1300
20	800
20	1100
20	3000
20	3000
20	2975
30	1600
30	2850
30	1250
30	950
30	1500
30	1250

5000

3000

2850

"salaire maximum supérieur à
\$2900 dans chaque département"

DEPTNO	MAX (SAL)

10	5000
20	3000

Exclusion de Groupes : la Clause HAVING

- Utilisez la clause HAVING pour restreindre les groupes
 - Les lignes sont regroupées.
 - La fonction de groupe est appliquée.
 - Les groupes qui correspondent à la clause HAVING sont affichés.

```
SELECT      column, group_function
FROM        table
[WHERE      condition]
[GROUP BY   group_by_expression]
[HAVING     group_condition]
[ORDER BY   column];
```

Utilisation de la clause HAVING

```
SQL> SELECT    deptno, max(sal)
  2  FROM      emp
  3  GROUP BY   deptno
  4  HAVING     max(sal)>2900;
```

DEPTNO	MAX (SAL)
10	5000
20	3000

Utilisation de la Clause HAVING

```
SQL> SELECT      job, SUM(sal) PAYROLL
  2  FROM          emp
  3  WHERE         job NOT LIKE 'SALES%'
  3  GROUP BY     job
  4  HAVING        SUM(sal)>5000
  5  ORDER BY     SUM(sal);
```

JOB	PAYROLL
ANALYST	6000
MANAGER	8275

Imbrication des Fonctions de Groupe

- **Afficher le salaire moyen maximum.**

```
SQL> SELECT      max (avg (sal) )  
      2  FROM      emp  
      3  GROUP BY deptno;
```

```
MAX (AVG (SAL) )  
-----  
      2916.6667
```

Sous-Interrogations

Définition

- Une Sous-interrogation est un ordre **SELECT** imbriqué dans une clause d'un autre ordre **SELECT** .
- Elles permettent de sélectionner des lignes d'une table lorsque la condition dépend des données de la table elle-même .
- Peuvent être placées dans les clauses SQL suivantes :
 - **WHERE**
 - **HAVING**
 - **FROM**

Utilisation d'une Sous-Interrogation pour Résoudre un Problème

- "Qui a un salaire supérieur à celui de Jones ?"

Requête principale



"Quel employé a un salaire supérieur à celui de Jones ?"

sous-interrogation



"Quel est le salaire de Jones ?"



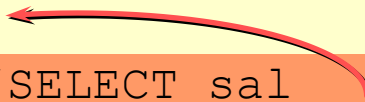
Sous-Interrogations

```
SELECT  select_list
FROM    table
WHERE   expr operator
        (SELECT      select_list
         FROM         table);
```

- La sous-interrogation (requête interne) est exécutée une fois avant la requête principale.
- Le résultat de la sous-interrogation est utilisé par la requête principale (externe).

Utilisation d'une Sous-Interrogation

```
SQL> SELECT  ename  
      2  FROM    emp  
      3  WHERE   sal > 2975  
      4          (SELECT sal  
      5              FROM    emp  
      6              WHERE   empno=7566) ;
```



```
ENAME  
-----  
KING  
FORD  
SCOTT
```

Conventions d'Utilisation des Sous-Interrogations

- Placez les sous-interrogations entre parenthèses.
- Placez les sous-interrogations à droite de l'opérateur de comparaison.
- N'ajoutez jamais de clause ORDER BY à une sous-interrogation.
- Utilisez les opérateurs mono-ligne avec les sous-interrogations mono-ligne.
- Utilisez les opérateurs multi-ligne avec les sous-interrogations multi-ligne.

Types de Sous-Interrogations

- **Sous-interrogation mono-ligne**



- **Sous-interrogation multi-ligne**



- **Sous-interrogation multi-colonne**



Sous-Interrogations Mono-ligne

- Ne ramènent qu'une seule ligne
- Utilisent des opérateurs de comparaison mono-ligne

Opérateur	Signification
=	Egal à
>	Supérieur à
>=	Supérieur ou égal à
<	Inférieur à
<=	Inférieur ou égal à
<>	Différent de

Exécution de Sous-Interrogations Mono-ligne

```
SQL> SELECT      ename, job
  2  FROM          emp
  3  WHERE         job =
  4                (SELECT      job
  5                  FROM        emp
  6                  WHERE       empno = 7369)
  7  AND          sal >
  8                (SELECT      sal
  9                  FROM        emp
 10                  WHERE       empno = 7876) ;
```

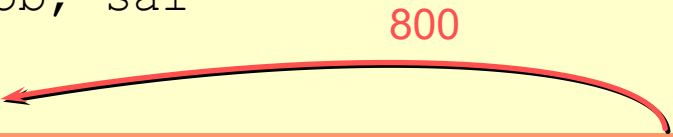
CLERK

1100

ENAME	JOB
-----	-----
MILLER	CLERK

Utilisation de Fonctions de Groupe dans une Sous-Interrogation

```
SQL> SELECT  ename, job, sal
2  FROM      emp
3  WHERE      sal =
4              (SELECT  MIN(sal)
5              FROM      emp) ;
```



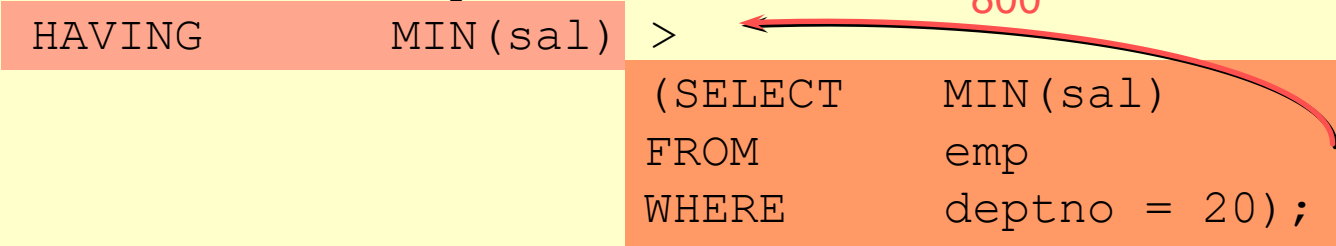
800

ENAME	JOB	SAL
-----	-----	-----
SMITH	CLERK	800

Clause HAVING avec Sous-Interrogations

- Oracle Server exécute les sous-interrogations en premier.
- Oracle Server ramène les résultats dans la clause HAVING de la requête principale.

```
SQL> SELECT      deptno, MIN(sal)
2  FROM          emp
3  GROUP BY      deptno
4  HAVING        MIN(sal) >
5                (SELECT      MIN(sal)
6                FROM          emp
7                WHERE         deptno = 20) ;
```



Qu'est-ce Qui ne Va pas dans cet Ordre ?

```
SQL> SELECT empno, ename  
2 FROM emp  
3 WHERE sal =  
4 (SELECT MIN(sal)  
5 FROM emp  
6 GROUP BY deptno);
```

```
ERROR:  
ORA-01427: single-row sub-query returns more than  
one row  
  
no rows selected
```

Opérateur mono-ligne avec sous-interrogation multi-ligne

Cet Ordre Va-t-il Fonctionner ?

```
SQL> SELECT  ename, job
2  FROM      emp
3  WHERE     job =
4             (SELECT job
5              FROM    emp
6              WHERE   ename='SMYTHE') ;
```

```
no rows selected
```

La sous-interrogation ne ramène aucune valeur

Sous-Interrogation Multi-ligne

- **Ramène plusieurs lignes**
- **Utilise des opérateurs de comparaison multi-ligne**

Opérateur	Signification
IN	Egal à un élément quelconque de la liste
ANY	Compare la valeur à chaque valeur ramenée par la sous-interrogation
ALL	Compare la valeur à toutes les valeurs ramenées par la sous-interrogation

Utilisation de l'Opérateur ANY dans les Sous-Interrogations Multi-ligne

Afficher les employés dont le salaire est inférieur à celui de n'importe quel employé 'CLERK' et qui ne sont pas eux-mêmes 'CLERK'.

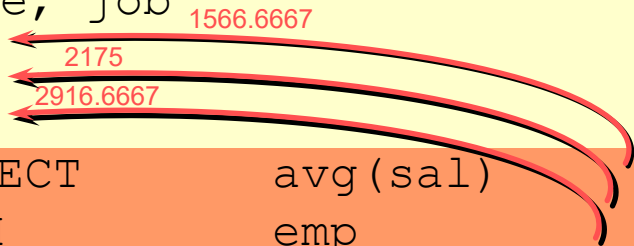
```
SQL> SELECT  empno, ename, job 1300
      2 FROM    emp 1100
      3 WHERE   sal < ANY 800
      4         (SELECT sal 950
      5         FROM    emp
      6         WHERE   job = 'CLERK')
      7 AND     job <> 'CLERK';
```

EMPNO	ENAME	JOB
7654	MARTIN	SALESMAN
7521	WARD	SALESMAN

Utilisation de l'Opérateur ALL dans les Sous-Interrogations Multi-ligne

Afficher les employés dont le salaire est supérieur au salaire moyen de tous les départements.

```
SQL> SELECT empno, ename, job
2 FROM emp
3 WHERE sal > ALL
4 (SELECT avg(sal)
5 FROM emp
6 GROUP BY deptno)
```



EMPNO	ENAME	JOB
7839	KING	PRESIDENT
7566	JONES	MANAGER
7902	FORD	ANALYST
7788	SCOTT	ANALYST

Résumé

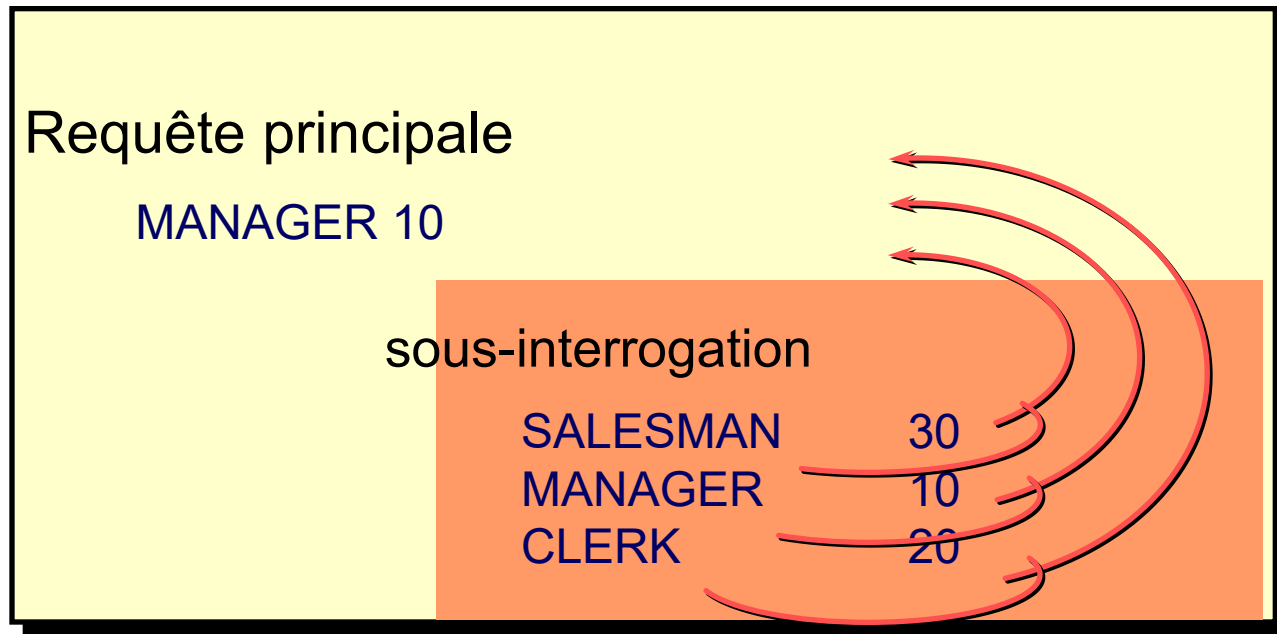
- **Les sous-interrogations sont utiles lorsqu'une requête fait appel à des valeurs inconnues.**

```
SELECT  select_list
FROM    table
WHERE   expr operator
        (SELECT select_list
         FROM   table);
```

Sous-Interrogations Multi-colonne

Sanae MAZOUZ

Sous-Interrogations Multi-colonne



La requête
principale
compare

aux

Valeurs d'une sous-interrogation
multi-ligne et multi-colonne

MANAGER 10

SALESMAN 30
MANAGER 10
CLERK 20

syntaxe

```
SELECT column, column, ...  
FROM table  
WHERE (column, column, ...) IN  
      (SELECT column, column, ...  
       FROM table  
       WHERE condition);
```

Utilisation des Sous-Interrogations Multi-colonne

Afficher le nom, le numéro de département, le salaire et la commission de tout employé dont le salaire et la commission correspondent à la fois aux salaire et commission d'un des employés du département 30.

```
SQL> SELECT  ename, deptno, sal, comm
        FROM emp
        WHERE  (sal, NVL(comm,-1)) IN

                (SELECT sal, NVL(comm,-1)
                 FROM    emp
                 WHERE   deptno = 30);
```

Comparaison de Colonnes

Par groupe de colonnes

SAL		COMM
1600	↔	300
1250	↔	500
1250	↔	1400
2850	↔	
1500		0
950	↔	

Colonne par colonne

SAL		COMM
1600	↔	300
1250	↔	500
1250	↔	1400
2850	↔	
1500	↔	0
950	↔	

Sous-Interrogation avec Comparaison Colonne par Colonne

Afficher le nom, le n° de département, le salaire et la commission de tout employé dont le salaire et la commission correspondent au salaire et à la commission d'un des employés du département 30.

```
SQL> SELECT  ename, deptno, sal, comm
2  FROM      emp
3  WHERE      sal IN          (SELECT sal
4                               FROM    emp
5                               WHERE    deptno = 30)
6  AND
7            NVL(comm, -1) IN (SELECT NVL(comm, -1)
8                               FROM    emp
9                               WHERE    deptno = 30);
```


Comparaison Colonne par Colonne

```
SQL> SELECT  ename,deptno, sal, comm
2  FROM      emp
3  WHERE      sal IN          (SELECT sal
4                               FROM    emp
5                               WHERE    deptno = 30)
6  AND
7            NVL(comm,-1) IN  (SELECT NVL(comm,-1)
8                               FROM    emp
9                               WHERE    deptno = 30);
```

ENAME	DEPTNO	SAL	COMM
JAMES	30	950	
BLAKE	30	2850	
TURNER	30	1500	0
CLARK	10	1500	300
...			

7 rows selected.

Comparaison par Groupe de Colonne

```
SQL> SELECT  ename, deptno, sal, comm
  2  FROM      emp
  3  WHERE      (sal, NVL(comm,-1)) IN
  4              (SELECT sal, NVL(comm,-1)
  5                  FROM      emp
  6                  WHERE      deptno = 30);
```

ENAME	DEPTNO	SAL	COMM
JAMES	30	950	
WARD	30	1250	500
MARTIN	30	1250	1400
TURNER	30	1500	0
ALLEN	30	1600	300
BLAKE	30	2850	

6 rows selected.

Mise en Forme des Résultats avec SQL*Plus

Sanae MAZOUZ

Etats Interactifs



Variables de Substitution

Les variables de substitution SQL*Plus permettent de stocker temporairement des valeurs

- Simple "et commercial" (&)**
- Double "et commercial" (&&)**
- Commandes DEFINE et ACCEPT**

Utilisation de la Variable de Substitution &

- Pour demander une valeur à un utilisateur, utilisez une variable préfixée par un simple "et commercial" (&).

```
SQL> SELECT empno, ename, sal, deptno
2 FROM emp
3 WHERE empno = &employee_num;
```

Enter value for employee_num: 7369

EMPNO	ENAME	SAL	DEPTNO
7369	SMITH	800	20

Utilisation de la Commande SET VERIFY

- **Affichage du texte de la commande avant et après que SQL*Plus remplace les variables de substitution par les valeurs.**

```
SQL> SET VERIFY ON
SQL> SELECT      empno,  ename,  sal,  deptno
   2  FROM        emp
   3  WHERE       empno = &employee_num;
```

```
Enter value for employee_num: 7369
old   3: WHERE empno = &employee_num
new   3: WHERE empno = 7369
```

```
...
```

Valeurs Caractères et Dates dans les Variables de Substitution

- Placez les valeurs de type caractère et date entre simples quotes.

```
SQL> SELECT ename, deptno, sal*12
2 FROM emp
3 WHERE job='&job_title';
```

Enter value for job_title: ANALYST

ENAME	DEPTNO	SAL*12
SCOTT	20	36000
FORD	20	36000

Spécification de Noms de Colonnes, d'Expressions et de Texte lors de l'Exécution

- Les variables de substitution peuvent remplacer :
 - Une condition WHERE
 - Une clause ORDER BY
 - Un nom de colonne
 - Un nom de table
 - Une expression
 - Un ordre SELECT

Spécification de Noms de Colonnes, d'Expressions et de Texte lors de l'Exécution

```
SQL> SELECT      empno, ename, job, &column_name
  2  FROM          emp
  3  WHERE         &condition
  4  ORDER BY     &order_column;
```

```
Enter value for column_name: sal
Enter value for condition: sal>=3000
Enter value for order_column: ename
```

EMPNO	ENAME	JOB	SAL
7902	FORD	ANALYST	3000
7839	KING	PRESIDENT	5000
7788	SCOTT	ANALYST	3000

Utilisation de la Variable de Substitution &&

- **Spécifiez un double "et commercial" (&&) si vous voulez réutiliser la valeur de la variable sans interroger l'utilisateur à chaque fois.**

```
SQL> SELECT      empno, ename, job, &&column_name
  2  FROM          emp
  3  ORDER BY      &column_name;
```

Enter value for column_name: deptno

EMPNO	ENAME	JOB	DEPTNO
7839	KING	PRESIDENT	10
7782	CLARK	MANAGER	10
7934	MILLER	CLERK	10
...			

14 rows selected.

Définition des Variables Utilisateur

- **Deux commandes SQL*Plus permettent de prédéfinir des variables :**
 - **DEFINE** : crée une variable utilisateur de type CHAR
 - **ACCEPT** : lit la valeur saisie par l'utilisateur et la stocke dans une variable
- **Si, avec la commande DEFINE, vous voulez spécifier comme valeur une chaîne de caractères contenant un ou plusieurs espaces, vous devez l'inclure entre simples quotes.**

La Commande ACCEPT

- Affiche un message d'invite personnalisé et récupère la saisie de l'utilisateur
- Permet un contrôle du type de donnée (CHAR, NUMBER ou DATE)
- Peut masquer la valeur saisie par l'utilisateur à des fins de confidentialité

```
ACCEPT variable [datatype] [FORMAT format]  
[PROMPT text] {HIDE}
```

Utilisation de la Commande ACCEPT

```
ACCEPT    dept PROMPT 'Provide the department name: '  
SELECT    *  
FROM      dept  
WHERE     dname = UPPER('&dept')  
/
```

Provide the department name: **Sales**

DEPTNO	DNAME	LOC
30	SALES	CHICAGO

Commandes DEFINE et UNDEFINE

- Une variable reste définie jusqu'à ce que vous exécutiez l'une des actions suivantes :
 - Utiliser la commande UNDEFINE
 - Quitter SQL*Plus
- La commande DEFINE permet de lister les variables définies.

Utilisation de la Commande DEFINE

- **Créer une variable pour stocker le nom de département.**

```
SQL> DEFINE deptname = sales
SQL> DEFINE deptname
```

```
DEFINE DEPTNAME          = "sales" (CHAR)
```

- **Utiliser la variable normalement.**

```
SQL> SELECT *
      2 FROM dept
      3 WHERE dname = UPPER('&deptname');
```


LES TRANSACTIONS

Sanae MAZOUZ

Transactions de Base de Données

- Un ordre du LMD est exécuté lorsque :
 - Vous ajoutez des lignes à une table
 - Vous modifiez des lignes existantes dans une table
 - Vous supprimez des lignes d'une table
- Une *transaction* est un ensemble d'ordres du LMD formant une unité de travail logique.

Transactions de Base de Données

- Une transaction se compose des éléments suivants :
 - Ensemble d'ordres du LMD effectuant une modification cohérente des données
 - Un ordre du LDD : *CREATE, ALTER, DROP, TRUNCATE*
 - Un ordre du LCD :
 - ✓ **GRANT** : Donner des privilèges à un utilisateur
 - ✓ **REVOKE** : Retirer des privilèges

LCT – Langage de Contrôle des Transactions

- **En SQL, les commandes COMMIT et ROLLBACK font partie du LCT – Langage de Contrôle des Transactions (en anglais TCL – Transaction Control Language).**
- **Elles servent à gérer les transactions, c'est-à-dire les modifications faites sur les données.**

COMMIT

- **COMMIT** est une commande SQL qui permet de valider définitivement les modifications effectuées dans la base de données.
- Après un **COMMIT**, les changements :
 - sont enregistrés définitivement
 - deviennent visibles pour les autres utilisateurs
 - ne peuvent plus être annulés

COMMIT

- **Effectuez les modifications.**

```
SQL> UPDATE    emp
      2  SET      deptno = 10
      3  WHERE    empno = 7782;
1 row updated.
```

- **Validez les modifications.**

```
SQL> COMMIT;
Commit complete.
```

ROLLBACK

- L'ordre ROLLBACK rejette toutes les modifications de données en instance.
 - Les modifications sont annulées.
 - L'état précédent des données est restauré.
 - Les lignes verrouillées sont libérées.

```
SQL> DELETE FROM employee;  
14 rows deleted.  
SQL> ROLLBACK;  
Rollback complete.
```

Transactions de Base de Données

Une transaction :

- Commence à l'exécution du premier ordre SQL
- Se termine par l'un des événements suivants :
 - COMMIT ou ROLLBACK
 - *Exécution d'un ordre LDD ou LCD (validation automatique)*
 - Fin de session utilisateur
 - Panne du système

Avantages des Ordres COMMIT et ROLLBACK

- **Garantit la cohérence des données**
- **Possibilité d'afficher le résultat des modifications avant qu'elles ne soient définitives**
- **Regroupement logique d'opérations**

SAVEPOINT

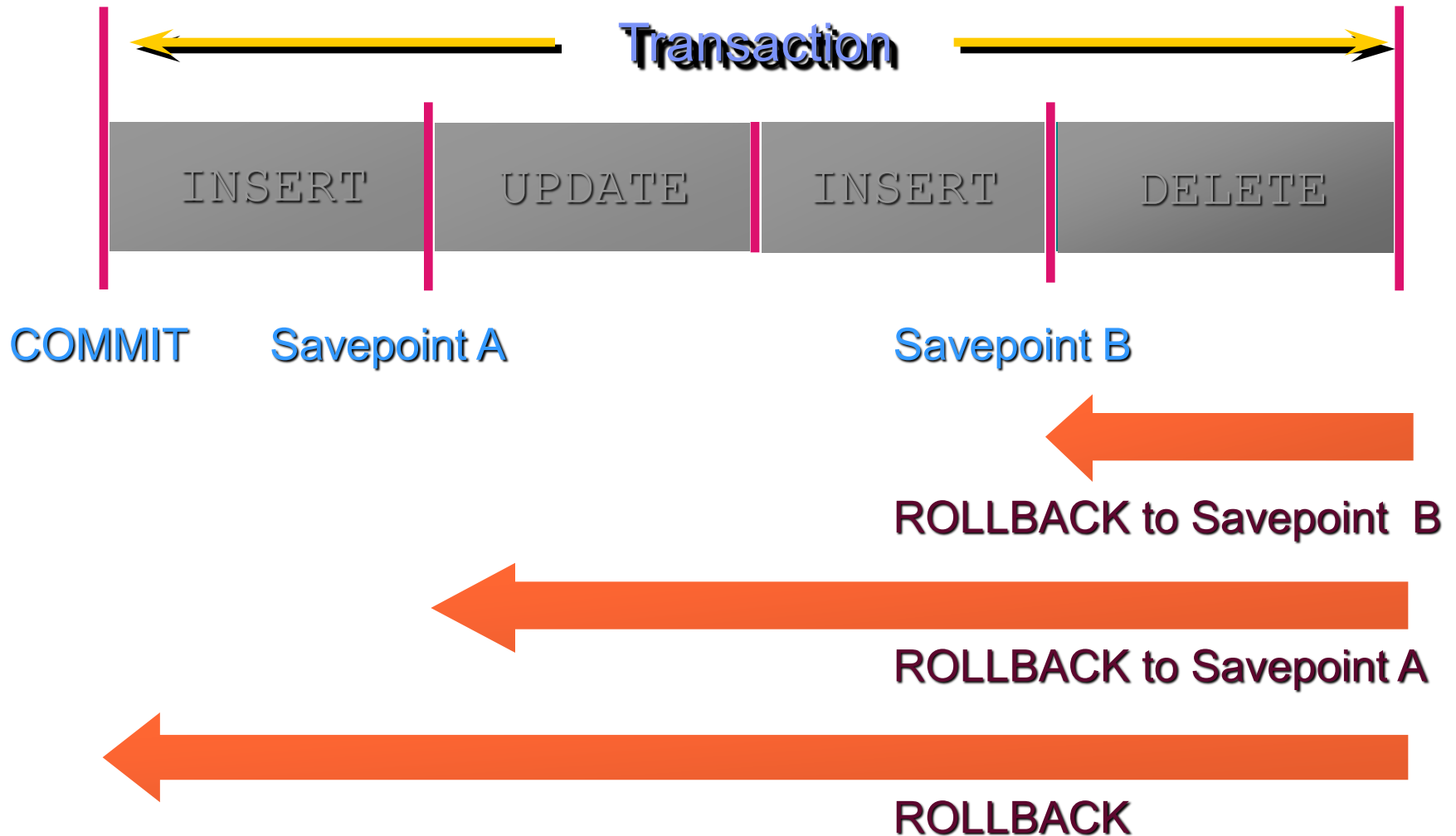
- **SAVEPOINT** est une commande SQL qui permet de définir un point intermédiaire dans une transaction.
- Elle permet de revenir à ce point précis avec la commande **ROLLBACK**, sans annuler toute la transaction.
- Ainsi, **SAVEPOINT** offre la possibilité d'annuler seulement une partie des modifications effectuées dans une transaction.

SAVEPOINT

- Posez une étiquette dans la transaction courante au moyen de l'ordre **SAVEPOINT**.

```
SQL> UPDATE...  
SQL> SAVEPOINT update_done;  
Savepoint created.
```

Contrôle des Transactions



Traitement Implicite des Transactions

- Une validation automatique a lieu dans les situations suivantes :
 - Exécution d'un ordre du LDD
 - Exécution d'un ordre du LCD
 - Sortie normale de SQL*Plus, sans ordre COMMIT ou ROLLBACK explicite
- Il se produit un rollback automatique en cas de sortie anormale de SQL*Plus ou d'une panne du système

Etat des Données Avant COMMIT ou ROLLBACK

- Il est possible de restaurer l'état précédent des données.
- L'utilisateur courant peut afficher le résultat des opérations du LMD au moyen de l'ordre **SELECT**.
- Les résultats des ordres du LMD exécutés par l'utilisateur courant *ne peuvent pas* être affichés par d'autres utilisateurs.
- Les lignes concernées sont *verrouillées*. Aucun autre utilisateur ne peut les modifier.

Etat des Données Après COMMIT

- **Les modifications des données dans la base sont définitives.**
- **L'état précédent des données est irrémédiablement perdu.**
- **Tous les utilisateurs peuvent voir le résultat des modifications.**
- **Les lignes verrouillées sont libérées et peuvent de nouveau être manipulées par d'autres utilisateurs.**
- **Tous les savepoints sont effacés.**

Validation de Données

- **Effectuez les modifications.**

```
SQL> UPDATE    emp
      2  SET      deptno = 10
      3  WHERE    empno = 7782;
1 row updated.
```

- **Validez les modifications.**

```
SQL> COMMIT;
Commit complete.
```


Etat des Données Après ROLLBACK

- L'ordre ROLLBACK rejette toutes les modifications de données en instance.
 - Les modifications sont annulées.
 - L'état précédent des données est restauré.
 - Les lignes verrouillées sont libérées.

```
SQL> DELETE FROM employee;  
14 rows deleted.  
SQL> ROLLBACK;  
Rollback complete.
```

Annulation des Modifications Jusqu'à une Étiquette

- Posez une étiquette dans la transaction courante au moyen de l'ordre **SAVEPOINT**.
- Annulez la transaction jusqu'à cette étiquette en utilisant l'ordre **ROLLBACK TO SAVEPOINT**.

```
SQL> UPDATE...  
SQL> SAVEPOINT update_done;  
Savepoint created.  
SQL> INSERT...  
SQL> ROLLBACK TO update_done;  
Rollback complete.
```

Rollback au Niveau Ordre

- **Si un seul ordre du LMD dans la transaction échoue, seul cet ordre est annulé.**
- **Oracle met en œuvre un savepoint implicite.**
- **Toutes les autres modifications sont conservées.**
- **L'utilisateur doit terminer explicitement les transactions en exécutant un ordre COMMIT ou ROLLBACK.**

Lecture Cohérente

- **La lecture cohérente garantit à tout moment une vue homogène des données.**
- **Les modifications effectuées par un utilisateur n'entrent pas en conflit avec celles d'un autre utilisateur.**
- **Sur les mêmes données, garantit que :**
 - **la lecture ignore les écritures en cours**
 - **l'écriture ne perturbe pas la lecture**

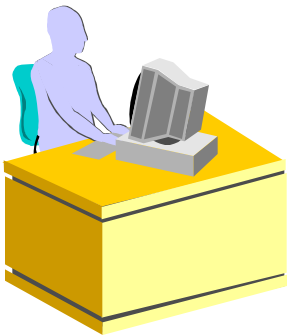
Lecture Cohérente

- On peut accéder à une base de données de deux manières différentes :
 - En lecture (ordre SELECT)
 - En écriture (ordres INSERT, UPDATE, DELETE)
- Le principe de lecture cohérente a les effets suivants :
 - L'utilisateur qui lit la base de données et celui qui y écrit ont une vue cohérente des données
 - Les utilisateurs qui accèdent en lecture ne peuvent pas voir les données en cours de modification

Lecture Cohérente

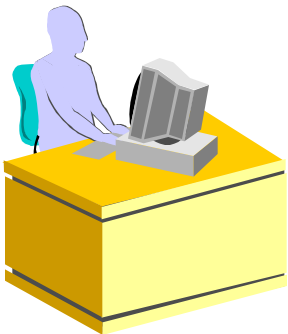
- Les utilisateurs qui accèdent en écriture sont sûrs que leurs modifications seront cohérentes
- Les modifications effectuées par un utilisateur ne peuvent pas interrompre ou gêner les modifications en cours d'un autre utilisateur
- L'objectif de la lecture cohérente est de garantir que chaque utilisateur voit les données telles qu'elles se présentaient lors de la dernière validation, c'est-à-dire avant le démarrage d'une opération du LMD.

Implémentation de la Lecture Cohérente



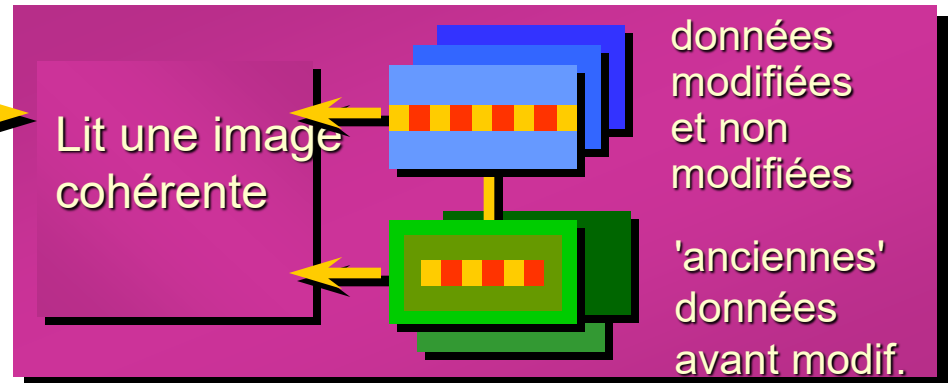
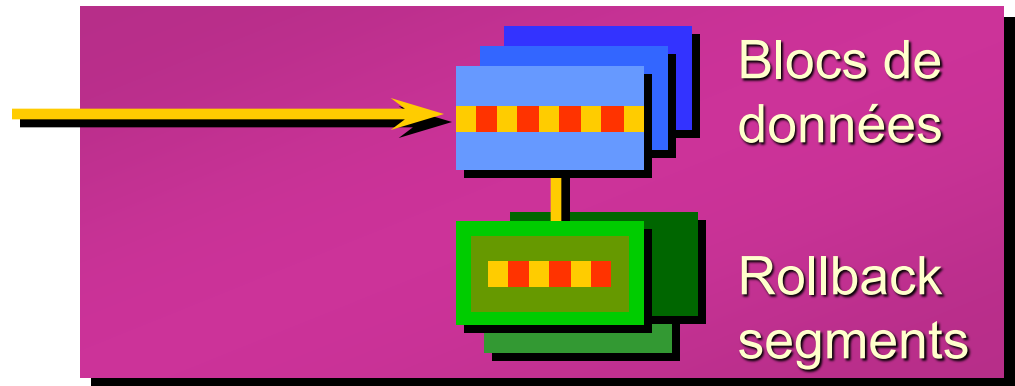
```
update emp  
set sal = 2000  
where ename =  
'SCOTT'
```

Utilisateur A



```
select *  
from emp
```

Utilisateur B



Implémentation de la Lecture Cohérente

- Le principe de lecture cohérente s'applique automatiquement. Il consiste à conserver une copie partielle de la base de données dans des "*rollback segments*".
- Lorsqu'une opération d'insertion, de mise à jour ou de suppression a lieu dans la base de données, Oracle copie les données avant leur modification dans un *rollback segment*.

Implémentation de la Lecture Cohérente

- **Tous les utilisateurs, excepté celui à l'origine de la modification, continuent à voir les données telles qu'elles se présentaient avant les modifications. En réalité, ils voient un "cliché" des données copiées dans les rollback segments.**
- **Tant que les modifications ne sont pas validées dans la base de données, seul l'auteur des mises à jour voit comment se présentera la base de données une fois modifiée.**

Implémentation de la Lecture Cohérente

- **Tous les autres continuent à voir des copies des "anciennes" données. Ainsi, les utilisateurs ont toujours une vue cohérente des données qui ne sont pas en cours de modification.**
- **Après la validation d'un ordre LMD, les modifications effectuées deviennent visibles par tout utilisateur exécutant un ordre SELECT.**

Implémentation de la Lecture Cohérente

- **Si un ordre ROLLBACK est exécuté, les modifications sont "annulées" et :**
 - **Les données d'origine présentes dans le rollback segment sont réécrites dans la table.**
 - **Tous les utilisateurs voient la base de données telle qu'elle était avant le début de la transaction.**

Les Vues

Sanae MAZOUZ

Pourquoi les vues?

- La confidentialité est assurée par l'utilisation de vues (views), qui agissent comme des fenêtres sur la base de données.
- Les vues correspondent au niveau externe qui reflète la partie visible de la base de données pour chaque utilisateur.
- Pour l'utilisateur, une vue apparaît comme une table.
- En théorie, les utilisateurs ne devraient accéder aux informations qu'en questionnant des vues.
- Les vues masquent la structure des tables interrogées.

Qu'est-ce qu'une Vue ?

- Une vue est considérée comme une table virtuelle car elle ne nécessite aucune allocation en mémoire pour contenir les données.
- Une vue n'a pas d'existence propre car seule sa structure est stockée dans le dictionnaire de données.
- Une vue est créée à l'aide d'une instruction **SELECT** appelée « requête de définition ».
- Cette requête interroge une ou plusieurs table(s), vue(s).
- Une vue se recharge chaque fois qu'elle est interrogée.

Qu'est-ce qu'une Vue ?

Table *EMP*

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT					10
7782	CLARK	MANAGER					10
7934	MILLER	CLERK					20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	20
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	20
7900	JAMES	CLERK	7698	03-DEC-81	950		20
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30

Vue *EMP_VU*

EMPNO	ENAME	JOB
7839	KING	PRESIDENT
7782	CLARK	MANAGER
7934	MILLER	CLERK

Qu'est-ce qu'une Vue ?

Une vue est une table virtuelle définie par une requête *SELECT* stockée dans la base de données. Elle ne contient pas physiquement de données, mais présente les données des tables sous-jacentes.

A Quoi Sert une Vue ?

- **Limite l'accès à la base de données.**
- **Facilite la création de requêtes complexes.**
- **Garantit l'indépendance des données.**
- **Présente les mêmes données sous différentes formes.**

Création d'une Vue

```
CREATE [OR REPLACE] [FORCE|NOFORCE] VIEW view  
    [(alias[, alias]...)]  
AS subquery  
[WITH CHECK OPTION [CONSTRAINT constraint]]  
[WITH READ ONLY]
```

- La sous-requête peut contenir une syntaxe **SELECT** complexe.
- Elle ne peut pas contenir une clause **ORDER BY**.

Création d'une Vue

Où :

- ***OR REPLACE*** : recrée la vue, si celle-ci existe déjà.
- ***FORCE*** : crée la vue, que les tables de base existent ou non.
- ***NOFORCE*** : ne crée la vue que s'il existe des tables de base (valeur par défaut).
- ***view*** : nom de la vue.
- ***alias*** : indique les noms des expressions sélectionnées par la requête de la vue. Le nombre d'alias doit être égal au nombre d'expressions sélectionnées par la vue.

Création d'une Vue

- ***subquery*** : ordre **SELECT** complet. Vous pouvez utiliser des alias de colonnes dans la liste **SELECT**.
- **WITH CHECK OPTION** : n'autorise l'insertion et la mise à jour que pour les lignes auxquelles la vue peut accéder.
- ***constraint*** : nom attribué à la contrainte **CHECK OPTION**.
- **WITH READ ONLY** : garantit qu'aucune opération **LMD** ne peut être exécutée dans la vue.

Création d'une Vue

- **Créez la vue EMPVU10 pour afficher en détail les employés du département 10.**

```
SQL> CREATE VIEW      empvu10
  2  AS SELECT        empno, ename, job
  3  FROM              emp
  4  WHERE             deptno = 10;
View created.
```

- **Décrivez la structure de la vue à l'aide de la commande SQL*Plus DESCRIBE.**

```
SQL> DESCRIBE empvu10
```

Création d'une Vue

- **Créez une vue à l'aide des alias de colonnes de la sous-interrogation.**

```
SQL> CREATE VIEW      salvu30
  2  AS SELECT        empno EMPLOYEE_NUMBER,  ename NAME,
  3                  sal SALARY
  4  FROM              emp
  5  WHERE             deptno = 30;
View created.
```

- **Sélectionnez les colonnes de cette vue par leur nom d'alias.**

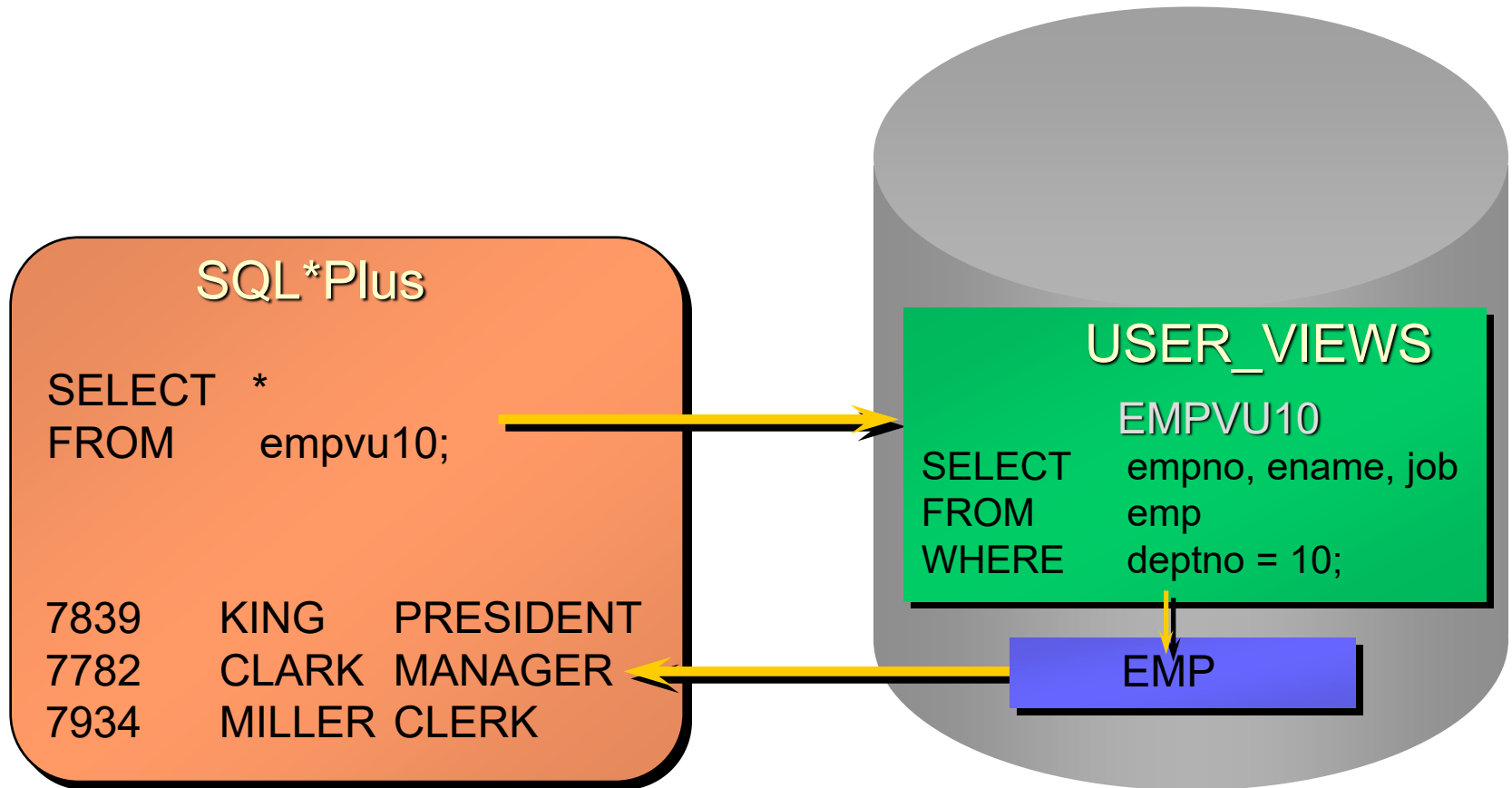
Extraction des Données d'une Vue

```
SQL> SELECT *  
2 FROM salvu30;
```

EMPLOYEE_NUMBER	NAME	SALARY
7698	BLAKE	2850
7654	MARTIN	1250
7499	ALLEN	1600
7844	TURNER	1500
7900	JAMES	950
7521	WARD	1250

6 rows selected.

Interrogation d'une Vue



Modification d'une Vue

- Modifiez la vue EMPVU10 à l'aide de la clause **CREATE OR REPLACE VIEW**. Ajoutez un alias pour chaque nom de colonne.

```
SQL> CREATE OR REPLACE VIEW empvu10
2      (employee_number, employee_name, job_title)
3  AS SELECT      empno, ename, job
4  FROM          emp
5  WHERE         deptno = 10;
View created.
```

- Les alias de colonnes de la clause CREATE VIEW s'affichent dans le même ordre que les colonnes de la sous-interrogation.

Modification d'une vue (ALTER VIEW)

- Pour pouvoir modifier une vue, il faut posséder le privilège ***ALTER ANY VIEW***.

```
ALTER VIEW [schéma.]nomVue  
{ ADD | DROP { CONSTRAINT nomContrainte | PRIMARY KEY  
  | UNIQUE(col1 [, col2]... ) }  
COMPILE ;
```

NB : Les modifications concernent l'ajout ou la suppression de contraintes.

Vues Simples et Vues Complexes

Caractéristiques	Vue simple	Vue complexe
Nombre de tables	Une	Une ou plusieurs
Fonctions	Non	Oui
Groupes de données	Non	Oui
Opérations LMD	Oui	Pas toujours

Vues complexes

- Une vue complexe est caractérisée par le fait de contenir, dans sa définition, plusieurs tables (jointures), et une fonction appliquée à des regroupements, ou des expressions.
- *La mise à jour de telles vues n'est pas possible.*
- Les restrictions de création sont les suivantes :
 - Si la requête de définition contient une sous-interrogation (jointure procédurale), elle ne doit pas faire intervenir la table source.
 - Il n'est pas possible d'utiliser les opérateurs ensemblistes (UNION [ALL], INTERSECT ou MINUS).

Création d'une Vue Complexe

- Créez une vue complexe qui inclut des fonctions de groupe pour afficher des valeurs venant de deux tables.

```
SQL> CREATE VIEW      dept_sum_vu
2      (name, minsal, maxsal, avgsal)
3  AS SELECT          d.dname, MIN(e.sal), MAX(e.sal),
4                      AVG(e.sal)
5  FROM               emp e, dept d
6  WHERE              e.deptno = d.deptno
7  GROUP BY           d.dname;
View created.
```

Règles d'Exécution des Ordres du LMD dans une Vue

- Vous pouvez exécuter des opérations du LMD dans des vues simples.
- Vous ne pouvez pas **supprimer** une ligne si la vue contient :
 - Des fonctions de groupe
 - Une clause GROUP BY
 - Le mot-clé DISTINCT

Règles d'Exécution des Ordres du LMD dans une Vue

- Vous ne pouvez pas **modifier** les données d'une vue s'elle contient :
 - Une des conditions citées dans la page précédente
 - Des colonnes définies par des expressions
 - La pseudo colonne ROWNUM
- Vous ne pouvez pas **ajouter** de données si :
 - La vue contient une des conditions citées ci-dessus ou dans la page précédente
 - Les tables de base contiennent des colonnes NOT NULL non sélectionnées par la vue

Utilisation de la Clause WITH CHECK OPTION

- Vous pouvez garantir que le LMD reste dans le domaine de la vue à l'aide de la clause WITH CHECK OPTION.

```
SQL> CREATE OR REPLACE VIEW empvu20
  2  AS SELECT *
  3  FROM    emp
  4  WHERE   deptno = 20
  5  WITH CHECK OPTION CONSTRAINT empvu20_ck;
View created.
```

- Toute tentative de modification du numéro de département dans une ligne de la vue échouera, car elle transgresse la contrainte WITH CHECK OPTION.

Utilisation de la Clause WITH CHECK OPTION

```
SQL> UPDATE empvu20  
      2 SET      deptno = 10  
      3 WHERE    empno = 7788;
```

```
update empvu20
```

```
*
```

```
ERROR at line 1:
```

```
ORA-01402: view WITH CHECK OPTION where-clause violation
```

Utilisation de la Clause WITH CHECK OPTION

Remarque :

Aucune ligne n'est mise à jour si le numéro du département est modifié (10), car la vue ne pourra plus afficher cet employé. Avec la clause **WITH CHECK OPTION**, la vue ne peut afficher que les employés du département 20 et n'autorise pas la modification du numéro de département de ces employés au travers de la vue.

Vues en lecture seule

```
SQL> CREATE VIEW vue_emps AS  
  2 SELECT employee_id, first_name, last_name,  
  3 hire_date  
  4 FROM employees  
  5 WITH READ ONLY;
```

Vue créée.

```
SQL> INSERT INTO vue_emps (employee_id, first_name,  
  2 last_name, hire_date)  
  3 VALUES (210, 'Ali', 'Azami', SYSDATE);
```

```
INSERT INTO vue_emps (employee_id, first_name,  
last_name, hire_date)
```

*

ERREUR à la ligne 1 :

ORA-42399: impossible d'effectuer une opération DML
sur une vue en mode lecture seule

Refus des Ordres du LMD

- **Aucune opération du LMD ne pourra être exécutée si vous ajoutez l'option WITH READ ONLY dans la définition de votre vue.**

```
SQL> CREATE OR REPLACE VIEW empvu10
  2      (employee_number, employee_name, job_title)
  3  AS SELECT      empno, ename, job
  4  FROM            emp
  5  WHERE           deptno = 10
  6  WITH READ ONLY;
View created.
```

- Toute tentative d'exécution d'un ordre du LMD sur une ligne de la vue génère l'erreur Oracle8 Server ORA-01752.

Suppression d'une Vue

- La suppression d'une vue n'entraîne pas la perte des données, car toute vue est basée sur des tables sous-jacentes de la base de données.

```
DROP VIEW view;
```

```
SQL> DROP VIEW empvu10;  
View dropped.
```

Résumé

- **Une vue est dérivée des données d'autres tables ou d'autres vues.**
- **Une vue offre les avantages suivants :**
 - **Restreint L'accès à la base de données**
 - **Les requêtes sont simplifiées**
 - **Indépendance de données**
 - **Visualisation des mêmes données sous différentes formes**
 - **Peut être supprimée sans perdre les données sous-jacentes**

PLSQL

Sanae MAZOUZ

Introduction

- **Le langage Oracle PL SQL est un langage procédural et est une extension procédurale du langage SQL. Il permet de grouper des traitements et de les soumettre au noyau en un bloc unique de traitement.**
- **Le langage SQL est non procédural alors que le PL SQL est un langage procédural.**
- **Le PL SQL sert à programmer des procédures, des fonctions, des triggers, des packages.**

Introduction

PL/SQL peut être utilisé sous trois formes :

- **Un bloc de code, exécuté comme une commande SQL, via un interpréteur standard (SQL*plus ou SQL Developer)**
- **Un fichier de commande PL/SQL.**
- **Un programme stocké (procédure, fonction, package ou trigger).**

Avantages

Les principaux avantages de PL/SQL sont :

☐ La modularité :

- **Un bloc d'instruction peut être composé d'un autre.**
- **Un bloc peut être nommé pour devenir une procédure ou une fonction cataloguée → réutilisable.**
- **Une procédure, ou fonction, cataloguée peut être incluse dans un paquetage (*package*) pour mieux contrôler et réutiliser ces composants logiciels.**

Avantages

❑ La portabilité :

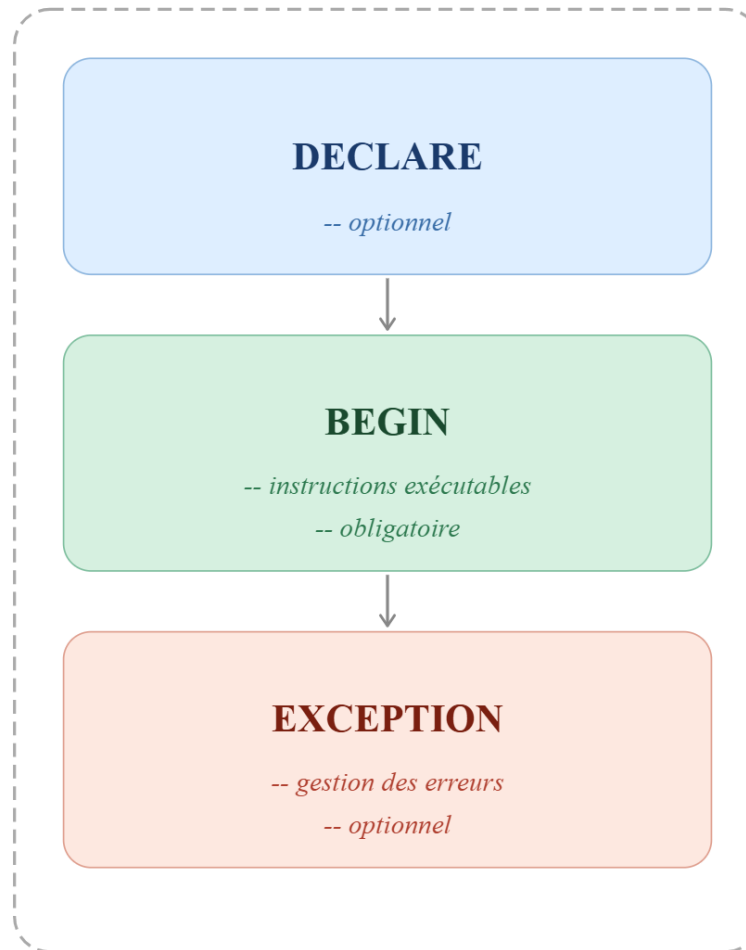
- **Un programme PL/SQL est indépendant du système d'exploitation qui héberge le serveur Oracle.**
- **En changeant de système, les applicatifs n'ont pas à être modifiés.**

Structure d'un programme

Un programme PL/SQL est composé de trois sections

- **DECLARE** (section optionnelle)
 - *déclare les variables, types, curseurs, exceptions, etc.*
- **BEGIN** (section obligatoire)
 - contient le code PL/SQL incluant ou non des directives SQL (jusqu'à l'instruction END;).
 - Le caractère « / » termine un bloc pour son exécution dans l'interface SQL*Plus.
- **EXCEPTION** (section optionnelle)
 - *permet de traiter les erreurs retournées par le SGBD à la suite d'exécutions d'instructions SQL.*

Structure d'un programme



Sous-bloc

- Un bloc peut être imbriqué dans le code d'un autre bloc (on parle de sous-bloc).
- Un sous-bloc peut aussi se trouver dans la partie des exceptions.
- Un sous-bloc commence par **BEGIN** et se termine par **END**.
- Un objet déclaré dans un sous-bloc n'est pas visible du bloc supérieur (principe des accolades de C et Java).

Sous-bloc

```
DECLARE    --section optionnelle
           déclaration variables, constantes, types, curseurs,...

BEGIN      --section obligatoire
           contient le code PL/SQL
           DECLARE    --section optionnelle
                   déclaration variables, constantes, types, curseurs,...

           BEGIN      --section obligatoire
                   contient le code PL/SQL

           EXCEPTION  --section optionnelle
                   traitement des erreurs

           END;        --obligatoire
EXCEPTION  --section optionnelle
           traitement des erreurs

END;        --obligatoire
```

Jeu de caractères

- ❑ Les programmes PL/SQL sont capables d'interpréter les caractères suivants :
 - Lettres A à Z et a à z ;
 - Chiffres de 0 à 9 ;
 - Symboles () + - * / < > = ! ~ ^ ; : . ' @ % , " # \$ & _ | { } ? [] ;
 - Tabulations, espaces et retours-chariot.
- ❑ PL/SQL n'est pas sensible à la casse (*not case sensitive*).

Identificateurs

- Un identificateur commence par une lettre suivie (optionnel) de symboles (lettres, chiffres, \$, _, #).
- Un identificateur peut contenir jusqu'à trente caractères.
- Les autres signes (pourtant connus du langage) sont interdits.

Identificateurs

Autorisés

X

t2

téléphone#

code_brevet

codeBrevet

oracle\$nombre

Interdits

moi&toi (symbole &)

debit-credit (symbole -)

on/off (symbole /)

code brevet (espace)

Commentaires

PL/SQL supporte deux types de commentaires :

- ***monolignes*** : commençant par le symbole **--** et finissant à la fin de la ligne.

Sur une ligne

```
-- Lecture de la table
SELECT salaire INTO v_salaire
FROM Pilote      -- Extraction du salaire
WHERE nom = 'Thierry Albaric';
v_bonus := v_salaire * 0.15;      -- Calcul
```

Commentaires

- **Multilignes** : commençant par `/*` et finissant par `*/`.

Sur plusieurs lignes

```
/* Lecture de la table Pilote */  
SELECT salaire INTO v_salaire  
FROM Pilote  
    /* Extraction du salaire  
       pour calculer le bonus */  
WHERE nom = 'Thierry Albaric';  
v_bonus := v_salaire * 0.15;      /*Calcul*/
```

Variables

- **Un programme PL/SQL est capable de manipuler des variables et des constantes (dont la valeur est invariable).**

Les variables et les constantes sont déclarées (et éventuellement initialisées) dans la section DECLARE.

Variables

- Plusieurs types de variables sont manipulés par un programme PL/SQL:
 - Scalaires recevant une seule valeur d'un type SQL (exemple : colonne d'une table) ;
 - Composites (%ROWTYPE, RECORD et TYPE) ;
 - Références (REF) .

Variables scalaires

La déclaration d'une variable scalaire est de la forme suivante :

- **CONSTANT** précise qu'il s'agit d'une constante (empêche toute modification de la variable ;
- **NOT NULL** pose une contrainte en ligne sur la variable ;
- **DEFAULT** permet d'initialiser la variable (équivalent à l'affectation :=).

identificateur [CONSTANT] typeDeDonnée [NOT NULL] [:= | DEFAULT expression];

Exemples

Variables	Constantes et expressions
<pre>DECLARE v_dateNaissance DATE; /* équivaut à v_dateNaissance DATE:= NULL; */ v_capacité NUMBER(3) := 999; v_téléphone CHAR(14) NOT NULL := '06-76-85-14-89'; v_trouvé BOOLEAN NOT NULL := TRUE; BEGIN ... </pre>	<pre>DECLARE c_pi CONSTANT NUMBER := 3.14159; v_rayon NUMBER := 1.5; v_aire NUMBER := c_pi * v_rayon**2; -- v_groupeSanguin CHAR(3) := 'O+'; /* équivaut à v_groupeSanguin CHAR(3) DEFAULT 'O+'; */ v_dateValeur DATE := SYSDATE + 2; BEGIN ... </pre>

Variables scalaires

Remarque :

Il n'est pas possible d'affecter **une valeur nulle** à une variable définie **NOT NULL** (*l'erreur renvoyée est l'exception prédéfinie VALUE_ERROR*).

La contrainte **NOT NULL** doit être suivie d'une clause d'initialisation.

Restrictions

- Il est impossible d'utiliser un identificateur dans une expression s'il n'est pas déclaré au préalable.
- Ici, la déclaration de la variable maxi est incorrecte

```
DECLARE
```

```
maxi NUMBER := 2 * mini;
```

```
mini NUMBER := 15;
```

Erreur de déclaration de la variable mini

Restrictions

- À l'inverse de la plupart des langages récents, les déclarations multiples ne sont pas permises.
- Celle qui suit est incorrecte :

```
DECLARE  
i, j, k NUMBER;
```

Erreur de déclaration des variables

Conventions recommandées

Objet	Convention	Exemple
Variable	<i>v_nomVariable</i>	v_compteur
Constante	<i>c_nomConstante</i>	c_pi
Exception	<i>e_nomException</i>	e_pasTrouvé
Type RECORD	<i>nomRecord_rec</i>	employe_rec
Variable RECORD	<i>r_nomVariable</i>	r_employe
Variable ROWTYPE	<i>rty_nomVariable</i>	rty_employe
Type-tableau	<i>nomTypeTableau_tytabs</i>	employes_tytabs
Variable tableau	<i>tab_nomTableau</i>	tab_employes
Curseur	<i>nomCurseur_cur</i>	employes_cur
Variable de substitution (SQL*Plus)	<i>s_nomVariable</i>	s_salaire
Variable de session (globale)	<i>g_nomVariable</i>	g_salaire

Types scalaires				
BINARY_INTEGER	DEC	DECIMAL	DOUBLE PRECISION	FLOAT
INT	INTEGER	NATURAL	NATURALN	NUMBER
NUMERIC	PLS_INTEGER	POSITIVE	POSITIVEN	REAL
SIGNTYPE	SMALLINT	CHAR	CHARACTER	LONG
LONG RAW	NCHAR	NVARCHAR2	RAW	ROWID
STRING	UROWID	VARCHAR	VARCHAR2	
BOOLEAN	DATE			
INTERVAL DAY TO SECOND (9i)	INTERVAL YEAR TO MONTH (9i)	TIMESTAMP (9i)	TIMESTAMP WITH LOCAL TIME ZONE (9i)	TIMESTAMP WITH TIME ZONE (9i)

Types composés		
RECORD	TABLE	VARRAY

Types références	
REF CURSOR	REF type_objet

Types grands objets			
BFILE	BLOB	CLOB	NCLOB

Types supplémentaires				
SYS.ANYDATA	SYS.ANYTYPE	SYS.ANYDATASET		
XMLTYPE	URITYPE			
MDSYS.SDO_GEOMETRY				
ORDSYS.ORDAUDIO	ORDSYS.ORDIMAGE	ORDSYS.ORDVIDEO	ORDSYS.ORDDOC	ORDSYS.ORDIMAGESIGNATURE

Les variables (types caractères)

- **CHAR(n)**: Chaîne de caractères de longueur fixe avec n compris entre 1 et 32767 (par défaut 1).
- **VARCHAR2(n)**: Chaîne de caractères de longueur variable avec n compris entre 1 et 32767.
- **LONG**: Chaîne de caractères de longueur variable avec au maximum 32760 octets.
- **RAW(n)** : Chaîne de caractères ou données binaires de longueur variable avec n compris entre 1 et 32767. Le contenu d'une variable de ce type n'est pas interprété par PL/SQL (pas de gestion des caractères nationaux).

Les variables (types caractères)

- **LONG RAW**: Identique au type LONG qui peut contenir des données binaires Jeux de caractères multi-octets.
- **NCHAR(n)** : Chaîne de caractères de longueur fixe avec n compris entre 1 et 32767 (par défaut 1).
- **NVARCHAR2(n)** : Chaîne de caractères de longueur variable avec n compris entre 1 et 32767. Le nombre de caractères réellement stockés dépend du nombre d'octets utilisés pour coder chaque caractère.
- **UROWID, ROWID** : Permet de stocker l'adresse absolue d'une ligne dans une table sous la forme d'une chaîne de caractères.

Les variables (types numériques)

- ***NUMBER[(e,d)]*** : Nombre réel avec e chiffres significatifs stockés et d décimales.
- ***BINARY_INTEGER*** : Nombre entier compris entre -2 147 483 647 et +2 147 483 647 (Utilise les fonctions de la librairie arithmétique).
- ***PLS_INTEGER*** : Nombre entier compris entre -2 147 483 647 et +2 147 483 647 (Plus rapide que BINARY_INTEGER car il utilise les registres du processeur).

Les variables (Types pour les grands objets)

- **BFILE** : Stocke la référence vers un fichier du système d'exploitation
- **BLOB** : Permet de stocker un objet binaire jusqu'à 4 Go
- **CLOB** : Permet de stocker un ensemble de caractères, jusqu'à 4 Go
- **NCLOB** : Permet de stocker un ensemble de caractères, codés sur un ou plusieurs octets, jusqu'à 4 Go

Variables %TYPE

- La directive **%TYPE** déclare une variable selon la définition d'une colonne d'une table ou d'une vue existante.
- Elle permet aussi de déclarer une variable conformément à une autre variable précédemment déclarée.
- Il faut faire préfixer la directive **%TYPE** avec le nom de la table et celui de la colonne.

(identificateur1 nomTable.nomColonne%TYPE)

Variables %TYPE

- ***ou avec le nom d'une variable existante***

(identificateur2 identificateur1%TYPE)

Variables %TYPE

Exemple :

Code PL/SQL

Commentaires

DECLARE

v_empno emp.empno%TYPE;

v_empno prend le type de la colonne empno de la table emp.

v_sal NUMBER(7,2) := 3000.00;

v_sal_min prend le type de la variable

v_sal_min v_sal%TYPE := v_sal*0.9;

v_sal et est initialisée à 2700,00.

BEGIN

...

Variables %ROWTYPE

- La directive **%ROWTYPE** permet de travailler au niveau d'un enregistrement (*record*).
- Il est composé d'un ensemble de colonnes.
- L'enregistrement peut contenir toutes les colonnes d'une table ou seulement certaines.
- Il est aussi possible d'insérer dans une table ou de modifier une table en utilisant une variable du type %ROWTYPE.

Code PL/SQL	Commentaires
DECLARE rty_emp emp%ROWTYPE; v_job emp.job%TYPE; BEGIN	La structure rty_emp est composée de toutes les colonnes de la table emp.
SELECT * INTO rty_emp FROM emp WHERE empno = 7369; v_job := rty_emp.job; -- ...	Chargement de l'enregistrement rty_emp à partir d'une ligne (employé 7369). Accès à une valeur par la notation pointée.
rty_emp.empno := 8000; rty_emp.ename := 'CLARK'; -- ... INSERT INTO emp VALUES rty_emp; END;	Insertion dans la table emp directement à partir de l'enregistrement.

Variables %ROWTYPE

Remarque :

Les colonnes récupérées par la directive **%ROWTYPE** n'héritent pas des contraintes NOT NULL qui seraient éventuellement déclarées au niveau de la table.

Variables RECORD

- Alors que la directive *%ROWTYPE* permet de déclarer une structure composée de colonnes de tables, elle ne convient pas à des structures de données personnalisées.
- Pour cela, nous avons le type de données RECORD :
 - Disponible depuis la version 7.
 - Définit vos propres structures de données.

Variables RECORD

- La syntaxe générale pour déclarer un **RECORD** est la suivante :

```
TYPE nomRecord IS RECORD  
( nomChamp typeDonnées [[NOT NULL] {:= | DEFAULT}  
expression]  
[,nomChamp typeDonnées... ]... );
```

Exemple

Code PL/SQL	Commentaires
<pre>DECLARE TYPE employe_rec IS RECORD (empno NUMBER(4), ename CHAR(10), job CHAR(9) := 'CLERK', sal NUMBER(7,2)); r_emp1 employe_rec; r_emp2 employe_rec; BEGIN</pre>	Déclaration du RECORD contenant quatre champs ; initialisation du champ job par défaut. Déclaration de deux variables de type RECORD.
<pre> r_emp1.empno := 7369; r_emp1.ename := 'SMITH'; r_emp1.sal := 800.00; r_emp2 := r_emp1;</pre>	Initialisation des champs d'un RECORD. Affectation d'un RECORD.
<pre>...</pre>	

Variables RECORD

- Les types RECORD ne peuvent pas être stockés dans une table.
- Il est possible qu'un champ d'un RECORD soit lui-même un RECORD, ou soit déclaré avec les directives %TYPE ou %ROWTYPE.
- Les RECORD ne peuvent pas être comparés (nullité, égalité et inégalité).

Exemple

```
DECLARE
TYPE employe_rec IS RECORD
    (empno    NUMBER(4) ,
     ename     CHAR(10) ,
     job       CHAR(9)  := 'CLERK' ,
     sal       NUMBER(7,2) ) ;

TYPE contrat_rec IS RECORD
    (r_employe    employe_rec,
     dateEmbauche  DATE,
     rty_coEmp     Emp%ROWTYPE,
     deptnom       Dept.dname%TYPE) ;
```

Variables tableaux (type TABLE)

- Les variables de type TABLE (*associative arrays*) *permettent de définir et de manipuler des tableaux dynamiques* (car définis sans dimension initiale).
- Un tableau est composé :
 - d'une clé primaire (de type BINARY_INTEGER)
 - et d'une colonne (de type scalaire, TYPE, ROWTYPE ou RECORD) pour stocker chaque élément.

Variables tableaux (type TABLE)

Syntaxe :

La syntaxe générale pour déclarer un type de tableau et une variable tableau est la suivante :

```
TYPE nomTypeTableau IS TABLE OF  
{typeScalaire | variable%TYPE | table.colonne%TYPE |  
table.%ROWTYPE};  
nomTableau nomTypeTableau;
```

Code PL/SQL	Commentaires
<pre> DECLARE TYPE empnos_tytabs IS TABLE OF NUMBER(4) INDEX BY BINARY_INTEGER; TYPE enames_tytabs IS TABLE OF Emp.ename%TYPE INDEX BY BINARY_INTEGER; TYPE emps_tytabs IS TABLE OF Emp%ROWTYPE INDEX BY BINARY_INTEGER; tab_empnos empnos_tytabs; tab_enames enames_tytabs; tab_emps emps_tytabs; </pre>	NUMBER(4) numéros d'employés %TYPE type ancré sur Emp.ename %ROWTYPE ligne complète de Emp Déclaration des tableaux.
<pre> BEGIN tab_empnos(1) := 7369; tab_empnos(2) := 7499; tab_enames(7369) := 'SMITH'; tab_enames(7499) := 'ALLEN'; tab_emps(0).empno := 7369; tab_emps(0).ename := 'SMITH'; END; </pre>	Initialisations. 1, 2... Convention classique PL/SQL. empno Accès direct par clé métier. 0, 1... Convention informatique.

Fonctions pour les tableaux

- **PL/SQL propose un ensemble de fonctions qui permettent de manipuler des tableaux.**
- **Il n'est pas possible actuellement d'appeler une de ces fonctions dans une instruction SQL(SELECT, INSERT, UPDATE ou DELETE).**

Exemple

Code PL/SQL	Commentaires
<code>IF tab_emps.EXISTS(0) THEN...</code>	Renvoie « vrai » car il existe un élément à l'indice 0.
<code>v_nombre := tab_empnos.COUNT;</code>	La variable v_nombre contient 2.
<code>v_premier := tab_empnos.FIRST;</code> <code>v_dernier := tab_empnos.LAST;</code>	La variable v_premier contient 1, v_dernier contient 2.
<code>v_avant := tab_empnos.PRIOR(2);</code>	La variable v_avant contient 1.
<code>tab_empnos.DELETE;</code>	Suppression de tous les éléments de tab_empnos.

Résolution de noms

- Lors des conflits potentiels de noms (variables ou colonnes) dans des instructions SQL (principalement INSERT, UPDATE, DELETE et SELECT), le nom de la colonne de la table est prioritairement interprété au détriment de la variable (de même nom).

Résolution de noms

- Dans l'exemple suivant, l'instruction **DELETE** supprime tous les pilotes (et non pas seulement le pilote 'Pierre Lamothe'), car Oracle considère les deux identificateurs comme la colonne de la table et non pas comme deux variables différentes !

```
DECLARE
nom CHAR(20) := 'Pierre Lamothe';
BEGIN
DELETE FROM Pilote WHERE nom = nom;
...
```

Résolution de noms

- Pour résoudre ce problème, deux solutions existent.
- La première consiste à nommer toutes les variables explicitement et différemment des colonnes.
- La deuxième consiste à utiliser une étiquette de bloc (*block label*) pour lever les ambiguïtés.

Préfixer les variables	Étiquette de bloc
<pre>DECLARE v_nom CHAR(20) := 'Pierre Lamothe'; BEGIN DELETE FROM Pilote WHERE nom = v_nom; END;</pre>	<pre><<principal>> DECLARE nom CHAR(20) := 'Pierre Lamothe'; BEGIN DELETE FROM Pilote WHERE nom = principal.nom; END;</pre>

Opérateurs

- Les opérateurs SQL (logiques, arithmétiques, concaténation...) sont disponibles aussi avec PL/SQL.

Code PL/SQL	Commentaires
<pre>DECLARE v_compteur NUMBER(3) DEFAULT 0; v_booléen BOOLEAN; v_nombre NUMBER(3); BEGIN</pre>	
<pre> v_compteur := v_compteur+1;</pre>	Incrémentation, opérateur +
<pre> v_booléen := (v_compteur = v_nombre);</pre>	v_booléen reçoit NULL du fait de la condition.
<pre> v_booléen := (v_nombre IS NULL);</pre>	v_booléen reçoit TRUE car la condition est vraie.

Variables de substitution

Variables de substitution

- Variables passées en paramètres d'entrée d'un bloc PL/SQL des variables définies sous SQL*Plus.
- On accède aux valeurs d'une telle variable dans le code PL/SQL en faisant préfixer le nom de la variable du symbole « & ».
- La directive ACCEPT (lecture de la variable au clavier)
- **Il ne faut pas déclarer des variables de substitution.**

Exemple

Code PL/SQL

```
ACCEPT s_empno PROMPT 'Entrer numéro employé : '  
ACCEPT s_augment PROMPT 'Entrer augmentation sal : '  
DECLARE  
    v_ename Emp.ename%TYPE;  
    v_sal Emp.sal%TYPE;  
BEGIN  
    SELECT ename, sal INTO v_ename, v_sal  
        FROM Emp WHERE empno = '&s_empno';  
    v_sal := v_sal + &s_augment;  
    DBMS_OUTPUT.PUT_LINE('Nouveau salaire : '  
        || v_sal || ' de ' || v_ename);  
END;
```

Sous SQL*Plus

Entrer numéro
employé : 7369
Entrer augmentation
sal : 500

Nouveau salaire :
1300 de SMITH

Procédure PL/SQL
terminée avec succès.

Variables de substitution

Remarque importante :

Il faut exécuter le bloc à l'aide de la commande start ou @ et non pas par copier-coller d'un éditeur de texte vers la fenêtre SQL*Plus (à cause des instructions d'entrée ACCEPT).

Variables de session

- Il est possible de définir des variables de session (globales) définies sous SQL*Plus au niveau d'un bloc PL/SQL.
- La directive SQL*Plus à utiliser en début de bloc est VARIABLE.
- Dans le code PL/SQL, il faut faire préfixer le nom de la variable de session du symbole « : ».
- L'affichage de la variable sous SQL*Plus est réalisé par la directive PRINT.

Variables de session

Code PL/SQL

```
VARIABLE g_compteur NUMBER;  
DECLARE  
    v_compteur NUMBER(3) := 99;  
BEGIN  
    :g_compteur := v_compteur+1;  
END;
```

Sous SQL*Plus

```
SQL> PRINT g_compteur ;  
  
G_COMPTEUR  
-----  
100
```

Paquetage DBMS_OUTPUT

- Il est possible d'afficher sous SQL*Plus des résultats calculés par un bloc PL/SQL :
 - avec des variables de session.
 - En utilisant des procédures du paquetage DBMS_OUTPUT.
- Ce paquetage assure la gestion des entrées/sorties de blocs ou sous-programmes PL/SQL (fonctions et procédures cataloguées, paquetages ou déclencheurs).

Procédure	Explication
<code>ENABLE (taille_tampon IN INTEGER DEFAULT 2000)</code>	Activation du paquetage dans la session.
<code>DISABLE</code>	Désactivation du paquetage dans la session.
<code>PUT(ligne IN VARCHAR2 DATE NUMBER);</code>	Mise dans le tampon d'un paramètre.
<code>NEW_LINE(ligne OUT VARCHAR2, statut OUT INTEGER)</code>	Écriture du caractère fin de ligne dans le tampon.
<code>PUT_LINE(ligne IN VARCHAR2 DATE NUMBER);</code>	<code>PUT</code> puis <code>NEW_LINE</code> .
<code>GET_LINE(ligne OUT VARCHAR2, statut OUT INTEGER)</code>	Affectation d'une chaîne du tampon dans une variable.
<code>GET_LINES(tab OUT DBMS_OUTPUT.CHARARR, nombreLignes IN OUT INTEGER);</code> <code>CHARARR</code> table de <code>VARCHAR2(255)</code>	Affectation de chaînes du tampon dans un tableau.

Code PL/SQL	Commentaires
<pre> SET SERVEROUTPUT ON DECLARE v_nbrEmp NUMBER; BEGIN </pre>	Activation du paquetage sous SQL*Plus.
<pre> DBMS_OUTPUT.ENABLE; DBMS_OUTPUT.PUT_LINE('Nous sommes le : ' SYSDATE); DBMS_OUTPUT.PUT_LINE('La racine de 2 = ' SQRT(2)); SELECT COUNT(*) INTO v_nbrEmp FROM emp; DBMS_OUTPUT.PUT_LINE ('Il y a ' v_nbrEmp ' employes dans la table'); END; / </pre>	Activation du paquetage sous PL/SQL. Affichage.
<pre> Nous sommes le : 14/07/03 La racine de 2 = 1,41421356237309504880168872420969807857 Il y a 14 employes dans la table Procédure PL/SQL terminée avec succès. </pre>	Résultats.

La section exécution

- ***Initialisation des variables***

- ✓ Opérateur :=

- ou

- ✓ SELECT ... INTO ...

- ***Remarque***

le **SELECT** doit ramener **une et une seule ligne**, sinon erreur.

- ***Visibilité des variables***

Une variable est visible dans le bloc où elle a été déclarée et dans les blocs imbriqués si elle n'a pas été redéfinie.

Structures de contrôles

PL/SQL offre la possibilité de programmer :

- les structures conditionnelles *si et cas (IF... et CASE) ;*
- les structures répétitives *tant que, répéter et pour (WHILE, LOOP, FOR).*

La structure IF

Trois formes de IF

- On peut distinguer trois formes de structure IF :
 - IF-THEN
 - *IF-THEN-ELSE*
 - *IF-THEN-ELSIF*
- Les opérateurs dans les conditions sont du type = , < , <= , != , LIKE , BETWEEN , AND , OR ,...
- Il faut Noter « END IF » en fin de structure et non pas « ENDIF ».

La structure IF

IF-THEN	IF-THEN-ELSE	IF-THEN-ELSIF
<pre>IF condition THEN instructions; END IF;</pre>	<pre>IF condition THEN instructions; ELSE instructions; END IF;</pre>	<pre>IF condition1 THEN instructions; ELSIF condition2 THEN instructions; ELSE instructions; END IF;</pre>
<pre>DECLARE v_téléphone CHAR(14) NOT NULL := '06-76-85-14-89'; BEGIN IF SUBSTR(v_téléphone,1,2)='06' THEN DBMS_OUTPUT.PUT_LINE ('C'est un portable!'); ELSE DBMS_OUTPUT.PUT_LINE ('C'est un fixe...'); END IF; END;</pre>		

Conditions booléennes

- Les variables booléennes peuvent prendre trois valeurs (TRUE, FALSE, NULL).
- Il est à noter que la négation de NULL (NOT NULL) renvoie une valeur nulle.

AND	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

OR	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

La structure CASE

- Comme l'instruction IF, la structure CASE permet d'exécuter une séquence d'instructions en fonction de différentes conditions.
- La structure CASE est utile lorsqu'il faut évaluer une même expression et proposer plusieurs traitements pour diverses conditions.
- La clause ELSE est optionnelle. Si elle n'est pas présente, PL/SQL ajoute par défaut l'instruction :
« ELSE RAISE CASE_NOT_FOUND » .

La structure CASE

CASE

[<<étiquette>>]

CASE *variable*

WHEN *expr1* THEN *instructions1*;

WHEN *expr2* THEN *instructions2*;

...

WHEN *exprN* THEN *instructionsN*;

[ELSE *instructionsN+1*];

END CASE [*étiquette*];

searched CASE

[<<étiquette>>]

CASE

WHEN *condition1* THEN *instructions1*;

WHEN *condition2* THEN *instructions2*;

...

WHEN *conditionN* THEN *instructionsN*;

[ELSE *instructionsN+1*];

END CASE [*étiquette*];

```
DECLARE
```

```
  v_mention CHAR(2);
```

```
  v_note    NUMBER(4,2) := 9.8;
```

```
BEGIN
```

```
  ...
```

```
IF v_note >= 16 THEN
```

```
  v_mention := 'TB';
```

```
ELSIF v_note >= 14 THEN
```

```
  v_mention := 'B';
```

```
ELSIF v_note >= 12 THEN
```

```
  v_mention := 'AB';
```

```
ELSIF v_note >= 10 THEN
```

```
  v_mention := 'P';
```

```
ELSE
```

```
  v_mention := 'R';
```

```
END IF;
```

```
END;
```

```
/
```

```
CASE
```

```
  WHEN v_note >= 16 THEN
```

```
    v_mention := 'TB';
```

```
  WHEN v_note >= 14 THEN
```

```
    v_mention := 'B';
```

```
  WHEN v_note >= 12 THEN
```

```
    v_mention := 'AB';
```

```
  WHEN v_note >= 10 THEN
```

```
    v_mention := 'P';
```

```
ELSE
```

```
  v_mention := 'R';
```

```
END CASE;
```

```
END;
```

```
/
```

Les structures répétitives

- Les structures répétitives sont :
 - *WHILE*
 - *WHEN*
 - *FOR*
- Utilisent l'instruction :

```
LOOP...  
.....  
END LOOP
```

La structure répétitive: *WHILE*

- *Avant chaque itération*, la condition est évaluée. Si elle est vraie, la séquence d'instructions est exécutée, puis la condition est réévaluée pour un éventuel nouveau passage dans la boucle.
- Ce processus continue jusqu'à ce que la condition soit fausse pour passer en séquence après le ***END LOOP***.

Syntaxe :

```
WHILE condition  
LOOP  
  instructions;  
END LOOP;
```

Exemple

Calculer le reste de la division de 5432 par 5.

```
SET SERVEROUTPUT ON
DECLARE
reste NUMBER := 5432;
BEGIN
    WHILE reste >= 5
    LOOP
        reste := reste -5;
    END LOOP;
    dbms_output.put_line('La valeur de a est:' || reste);
END;

/
```


La structure répétitive : WHEN

- EXIT force la sortie de la boucle sans conditions.
- EXIT WHEN permet une sortie de boucle si la condition est vraie.
- EXIT <<label>> WHEN permet une sortie d'une boucle nommée label si la condition est vraie.
- EXIT <<label>> force une sortie de boucle nommée label.

Syntaxe :

```
<<label>>  
LOOP  
    instruction1;  
    instruction2;  
EXIT [label][WHEN condition1]  
END LOOP label;
```

Exemple

Insérer les 10 premiers chiffres dans la table result.

```
DECLARE
    nb      NUMBER := 1;
BEGIN
    LOOP
        INSERT INTO result VALUES (nb);
        nb := nb + 1;
        EXIT WHEN nb > 10;
    END LOOP;
END ;
```

La structure répétitive : FOR

Dans un traitement itératif la structure FOR IN permet de boucler grâce à un nombre d'itérations connu à l'avance.

```
FOR indice IN [REVERSE] exp1 .. exp2  
  LOOP  
    instructions;  
  END LOOP;
```

Remarque :

- ✓ Inutile de déclarer l'indice
- ✓ Indice varie de exp1 à exp2 de 1 en 1
- ✓ Si REVERSE est précisé, indice varie de exp2 à exp1 avec un pas de -1.

Exemple

Calcul de la factorielle de 5

```
DECLARE
    fact NUMBER := 1;
BEGIN
    FOR i IN 1 .. 5
    LOOP
        fact := fact * i ;
    END LOOP;
END;
```

Interactions simples avec la base

Extraire des données : Erreurs

la requête **SELECT INTO** doit renvoyer impérativement 1 seul enregistrement, sinon PL / SQL déclenche une exception:

- Si le **select** renvoie plus d'une ligne, une exception « **TOO_MANY_ROWS** » (**ORA-01422**) est levée.
- Si le select ne renvoie aucune ligne, une exception « **NO_DATA_FOUND** » (**ORA-01403**) est levée.

Interactions simples avec la base

Modification de données

- Les requêtes SQL (**insert**, **update**, **delete**,...) peuvent utiliser les variables PL/SQL.
- Les **commit** et **rollback** doivent être explicites ; aucun changement n'est effectué automatiquement à la sortie d'un bloc.

Interactions simples avec la base

Exemple :

```
DECLARE
```

```
    v_emp emp%ROWTYPE;
```

```
BEGIN
```

```
    select * into v_emp from emp where ename = 'KING';
```

```
    v_emp.empno := v_emp.empno + 5;
```

```
    v_emp.ename := 'SMITH';
```

```
    INSERT INTO emp values v_emp;
```

```
END;
```

Les curseurs

- **Curseur** : zone de mémoire de taille fixe, utilisée par le noyau d'Oracle pour analyser et interpréter tout ordre SQL.
- Deux types de curseurs :
 - ✓ **Implicite** : créés et gérés par Oracle à chaque ordre SQL (lorsque la close **INTO** accompagne le **SELECT**).
 - ✓ **Explicite** : créés et gérés par l'utilisateur afin de pouvoir traiter un **SELECT** qui retourne **plusieurs tuples**.

Les curseurs

- Les curseurs sont définis dans la section DECLARE d'un bloc PL/SQL.
- Pour utiliser un curseur explicite, on doit passer par les étapes suivantes :
 1. Déclaration du curseur,
 2. Ouverture du curseur,
 3. Traitement des lignes du résultat,
 4. Fermeture du curseur.

Les curseurs

Déclaration du curseur

- Association d'un nom de curseur à une requête SELECT.
- Se fait dans la section DECLARE d'un bloc PL/SQL.

```
DECLARE  
  
CURSOR nom_curseur IS  
  
Requête_Select ;  
  
.....
```

Exemple

Soit la table employe (id, nomemp, sal, ville, ...)

```
DECLARE
```

```
CURSOR emp_rabat IS
```

```
SELECT nomemp, sal FROM employe WHERE ville =  
'Rabat';
```

```
.....
```

Les curseurs

Ouverture du curseur

- Alloue un espace mémoire au curseur et positionne les éventuels verrous.
- Après avoir déclaré le curseur, il faut l'ouvrir dans la section exécutable **BEGIN**.
- L'ouverture du curseur **amorce** l'analyse de SELECT et son exécution.
- La réponse est calculée et rangée dans un espace temporaire.

```
OPEN nom_curseur ;
```

Exemple

```
DECLARE
  CURSOR emp_rabat IS
    SELECT nomemp, sal FROM employe
    WHERE ville = 'Rabat';
BEGIN
  OPEN emp_rabat ;

  .....
END ;
```

Les curseurs

Traitement des lignes

- L'accès aux données se fait par la clause : **FETCH INTO**

```
FETCH nom_curseur INTO var1, var2, ...
```

- **FETCH** permet de récupérer une ligne de l'ensemble des lignes associés au curseur et stocke les valeurs dans des variables réceptrices.
- Pour traiter plusieurs tuples, il faut utiliser une boucle.

Les curseurs

Fermeture du curseur

- Après le traitement des lignes pour libérer la place mémoire, il faut fermer le curseur.

```
CLOSE nom_curseur ;
```

DECLARE

```
CURSOR emp_rabat IS SELECT nomemp, sal FROM  
employe WHERE ville = 'Rabat';  
nom employe.nomemp%TYPE ;  
salaire employe.sal%TYPE ;  
-- emp emp_rabat%ROWTYPE;
```

BEGIN

```
OPEN emp_rabat ;  
  LOOP  
    FETCH emp_rabat INTO nom, salaire ;  
    -- FETCH emp_rabat INTO emp;  
    ...  
    EXIT WHEN salaire > 10000;  
  END LOOP;  
CLOSE emp_rabat ;  
END;
```


Les curseurs

Statut d'un curseur (Attribut)

- Les attributs d'un curseur sont des indicateurs sur son état.
- Quatre attributs permettent d'évaluer l'état du curseur:
 - **%Found** : vrai si le dernier **FETCH** a ramené un tuple.
 - **%NotFound** : vrai si le dernier **FETCH** n'a ramené aucun tuple.
 - **%Rowcount** : compte le nombre de **FETCH** exécutés sur un curseur.
 - **%Isopen** : vrai si le curseur est ouvert.

DECLARE

```
CURSOR cur_emp IS
  SELECT empno, ename, sal
  FROM   emp
  WHERE  deptno = 10;
```

```
v_empno emp.empno%TYPE;
v_ename emp.ename%TYPE;
v_sal   emp.sal%TYPE;
totalSal NUMBER := 0;
```

BEGIN

```
OPEN cur_emp;
FETCH cur_emp
  INTO v_empno, v_ename, v_sal;
WHILE (cur_emp%FOUND) LOOP
  totalSal := totalSal + v_sal;
  FETCH cur_emp
    INTO v_empno, v_ename, v_sal;
END LOOP;
```

```
CLOSE cur_emp;
DBMS_OUTPUT.PUT_LINE(
  'Total sal dept 10 = '
  || totalSal);
```

END;

/

BEGIN

```
OPEN cur_emp;
LOOP
  FETCH cur_emp
    INTO v_empno, v_ename, v_sal;
  EXIT WHEN cur_emp%NOTFOUND;
  totalSal := totalSal + v_sal;
END LOOP;
```

```
CLOSE cur_emp;
DBMS_OUTPUT.PUT_LINE(
  'Total sal dept 10 = '
  || totalSal);
```

END;

/

Les curseurs

Avec un curseur implicite créé par Oracle, les mêmes attributs aux colonnes sont disponibles à condition de les préfixer par SQL. Ces attributs référents au dernier curseur implicite utilisé par l'application.

Curseur implicite	Curseur explicite
SQL%FOUND	nom-curseur%FOUND
SQL%NOTFOUND	nom-curseur%NOTFOUND
SQL%ISOPEN	nom-curseur%ISOPEN
SQL%ROWCOUNT	nom-curseur%ROWCOUNT

Les curseurs

Exemple

```
SET SERVEROUTPUT ON
    nb number;
BEGIN
    DELETE FROM emp WHERE ename='KING';
    nb := SQL%ROWCOUNT; -- curseur sql
    dbms_output.put_line('nb = ' || nb);
END;
```

Retourne le nombre d'enregistrement supprimés dont le champ nom est 'KING'.

Les curseurs

Utilisation simplifiée des curseurs

- L'utilisation **FOR LOOP** remplace **OPEN**, **FETCH** et **CLOSE**.
- Lorsque le curseur est invoqué, un enregistrement est automatiquement créé avec les mêmes éléments de données que ceux définies dans **SELECT**.
- La boucle **FOR** s'arrête d'elle-même à la fin de l'extraction de la dernière ligne du curseur.

Les curseurs

Simplification d'écriture

- L'attribut **%ROWTYPE** permet de déclarer une variable de même type que l'enregistrement (la ligne) de la table.
- Les éléments de la structure (nom_variable) sont identifiés par: **nom_variable.nom_colonne**.

Syntaxe :

```
DECLARE  
    CURSOR nom_curseur IS Requete_SELECT ;  
    nom_variable  nom_curseur%ROWTYPE ;
```

Les curseurs

```
DECLARE
CURSOR nom_curseur IS SELECT...;
nom_variable
nom_curseur%ROWTYPE;
BEGIN
OPEN nom_curseur;
LOOP
FETCH nom_curseur INTO
nom_variable;
EXIT WHEN nom_curseur%NOTFOUND;
...
END LOOP;
CLOSE nom_curseur;
END;
```

Ecriture complète

```
DECLARE
CURSOR nom_curseur IS SELECT ...;
BEGIN
FOR nom_variable IN nom_curseur
LOOP
...
END LOOP;
END;
```

Ecriture simplifiée

Exemple

Code PL/SQL	Commentaires
<pre>DECLARE CURSOR cur_emp IS SELECT empno, ename FROM emp WHERE deptno != 10;</pre>	Déclaration du curseur.
<pre>BEGIN FOR enreg IN cur_emp LOOP DBMS_OUTPUT.PUT_LINE('nom : ' enreg.ename ' (' enreg.empno ').'); END LOOP; END; /</pre>	Itération dans le curseur.

PLSQL- Suite-

Sanae MAZOUZ

LES EXCEPTIONS

Les Exceptions

Afin d'éviter qu'un programme s'arrête à la première erreur :

- **Requête ne retournant aucune ligne,**
- **Valeur incorrecte à écrire dans la base,**
- **Conflit de clés primaires,**
- **Division par zéro, etc...**

Il est indispensable de prévoir tous les cas potentiels d'erreurs et d'associer à chacun de ces cas la programmation d'une exception PL/SQL.

Les Exceptions

Les exceptions peuvent se programmer dans :

- **Un bloc PL/SQL**
- **Un sous-programme (fonction ou procédure cataloguée)**
- **Un paquetage**
- **Un déclencheur**

Les Exceptions

Une exception PL/SQL correspond à une condition d'erreur et est associée à un identificateur.

Une exception est détectée (aussi dite « levée ») au cours de l'exécution d'une partie de programme (entre un BEGIN et un END).

Une fois levée :

- **l'exception termine le corps principal des instructions**
- **et renvoie au bloc EXCEPTION du programme en question**

Les Exceptions

La syntaxe générale d'un bloc d'exceptions est la suivante:

```
EXCEPTION
```

```
    WHEN exception1 [OR exception2 ...] THEN  
        instructions;
```

```
    [WHEN exception3 [OR exception4 ...] THEN  
        instructions; ]
```

```
    [WHEN OTHERS THEN  
        instructions; ]
```

Les Exceptions

PL/SQL offre au développeur un mécanisme de gestion des exceptions. Il permet de préciser la logique du traitement des erreurs survenues dans l'un de ses blocs.

Il existe deux types d'exceptions :

- **Erreur ORACLE**
- **Erreur utilisateur**

Les Exceptions

Exceptions définies par ORACLE

- Nommées par Oracle,
- Se déclenchent automatiquement,
- Nécessitent de prévoir la prise en compte de l'erreur dans la section EXCEPTION.

Exception prédéfinie	Erreur Oracle	Valeur de SQLCODE
ACCESS_INTO_NULL	ORA-06530	-6530
CASE_NOT_FOUND	ORA-06592	-6592
COLLECTION_IS_NULL	ORA-06531	-6531
CURSOR_ALREADY_OPEN	ORA-06511	-6511
DUP_VAL_ON_INDEX	ORA-00001	-1
INVALID_CURSOR	ORA-01001	-1001
INVALID_NUMBER	ORA-01722	-1722
LOGIN_DENIED	ORA-01017	-1017
NO_DATA_FOUND	ORA-01403	+100
NOT_LOGGED_ON	ORA-01012	-1012
PROGRAM_ERROR	ORA-06501	-6501
ROWTYPE_MISMATCH	ORA-06504	-6504
SELF_IS_NULL	ORA-30625	-30625
STORAGE_ERROR	ORA-06500	-6500
SUBSCRIPT_BEYOND_COUNT	ORA-06533	-6533
SUBSCRIPT_OUTSIDE_LIMIT	ORA-06532	-6532
SYS_INVALID_ROWID	ORA-01410	-1410
TIMEOUT_ON_RESOURCE	ORA-00051	-51
TOO_MANY_ROWS	ORA-01422	-1422
VALUE_ERROR	ORA-06502	-6502
ZERO_DIVIDE	ORA-01476	-1476

Les Exceptions

Exceptions prédéfinies

- **ZERO_DIVIDE** : tentative de division par zéro.
- **TOO_MANY_ROWS** : la commande ***SELECT INTO*** retourne plus d'une ligne.
- **NO_DATA_FOUND** : déclenchée si la commande ***SELECT INTO*** ne retourne aucune ligne ou si l'on fait référence à un enregistrement ,non initialisé, d'un tableau PL/SQL.
- **LOGIN_DENIED** : connexion à la base est échouée, car le nom utilisateur ou le mot de passe est invalide.
- **CURSOR_ALREADY_OPEN** : tentative d'ouverture d'un curseur déjà ouvert.
-

Les Exceptions

Exceptions définies par l'utilisateur

- Nommés par le programmeur
- Arrêt de l'exécution du bloc
- Sont déclenchées par une instruction du programme soit automatiquement ou manuellement
- Nécessitent de prévoir la prise en compte de l'erreur dans la section EXCEPTION

Oracle fournit les fonctions **SQLCODE** et **SQLERRM**, qui renvoient respectivement le code et le message d'erreur correspondant à l'exception.

Les Exceptions

- Si une anomalie se produit, le bloc **EXCEPTION** s'exécute.
- Si le programme prend en compte l'erreur dans une entrée **WHEN...**, les instructions de cette entrée sont exécutées et le programme se termine.
- Si l'exception n'est pas prise en compte dans le bloc **EXCEPTION** :
 - il existe une section **OTHERS** où des instructions s'exécutent ;
 - S'il n'existe pas une section **OTHERS**, l'exception sera propagée au programme appelant

Les Exceptions

Syntaxe erreur ORACLE prédéfinie :

Certaines erreurs ORACLE ont déjà un nom. Il est donc inutile de les déclarer comme précédemment. On utilise leur nom dans la section EXCEPTION.

Exemple :

```
DECLARE
    ...
BEGIN
    ...
EXCEPTION
    WHEN    nom_erreur    THEN...traitement de l'erreur
END;
```

Les Exceptions

Exemple :

```
DECLARE
    salaire emp.sal%TYPE ;
BEGIN
    SELECT sal into salaire FROM emp WHERE empno =10;
    .....
EXCEPTION
    WHEN TOO_MANY_ROWS THEN traitements ;
    WHEN NO_DATA_FOUND THEN traitements ;
END ;
```

Les Exceptions

Fonctionnement :

Lorsqu'une exception est détectée, il y a :

- Arrêt de l'exécution du bloc
- Branchement sur la section exception
- Parcours des clauses WHEN jusqu'au bon choix
- Exécution des instructions associées
- Une fois le traitement de l'erreur est terminé, c'est le bloc suivant qui est effectué

Les Exceptions utilisateur

- Il est possible de définir ses propres exceptions
- *Deux étapes essentielles :*

Déclaration:

- La déclaration du nom de l'exception doit se trouver dans la section déclarative du sous-programme.
- *Nom_Exception* **EXCEPTION;**

Déclenchement

- Dans ce cas, le programme doit explicitement rediriger le traitement vers le bloc des exceptions par la directive **RAISE.**

Les Exceptions utilisateur

Syntaxe :

```
DECLARE  
    .....  
    Nom_exception EXCEPTION  
BEGIN  
    .....--Instructions  
    If (condition_erreur) THEN RAISE Nom_exception;  
    .....  
EXCEPTION  
    WHEN Nom_exception THEN traitements;  
END;
```

*On nomme
l'erreur*

*On déclenche
l'erreur*

*On traite
l'erreur*

Remarque :

On sort du bloc après l'exécution du traitement d'erreur.

DECLARE

CURSOR emp_cur IS SELECT ename, sal FROM emp WHERE deptno = 10;

v_ename emp.ename%TYPE;

v_sal emp.sal%TYPE;

ERR_salaire EXCEPTION;

BEGIN

OPEN emp_cur;

FETCH emp_cur INTO v_ename, v_sal; --premier enregistrement

WHILE emp_cur%FOUND LOOP -- S'il y a un enregistrement récupéré

IF v_sal IS NULL THEN

RAISE ERR_salaire;

..... --Traitement de l'enregistrement récupéré

FETCH emp_cur INTO v_ename, v_sal; --enregistrement suivant

END LOOP;

CLOSE emp_cur;

EXCEPTION

WHEN ERR_salaire THEN INSERT INTO temp VALUES (v_ename, 'Salaire non défini');

WHEN NO_DATA_FOUND THEN INSERT INTO temp VALUES (v_ename, 'Employé non trouvé');

END;

Les Exceptions utilisateur

Exception 'OTHERS'

Le module **EXCEPTION** permet également de traiter un code erreur qui n'est traité par aucune des exceptions. Le nom générique de cette exception (prédéfinie) est **OTHERS**.

Syntaxe :

```
EXCEPTION
    WHEN exception1 THEN
        traitement1;
    WHEN exception2 THEN
        traitement2;
    WHEN OTHERS THEN
        traitement3;
```

Deux fonctions permettent de récupérer des informations sur l'erreur Oracle :

- **Sqlcode** : retourne une valeur numérique: numéro de l'erreur.
- **Sqlerrm** : renvoie le libellé de l'erreur.

Les Exceptions utilisateur

Remarque :

Sqlcode : varie de **-20999** à **-20000**

c'est la plage réservée au développeur pour définir ses propres erreurs avec ***RAISE_APPLICATION_ERROR***

Plage	Signification
0	Pas d'erreur
+100	NO_DATA_FOUND
-1 à -19999	Erreurs Oracle prédéfinies
-20000 à -20999	Erreurs personnalisées développeur
-21000 et au-delà	Erreurs Oracle internes

```

SET SERVEROUTPUT ON;

DECLARE
    v_nom emp.ename%TYPE;
    v_sal emp.sal%TYPE;
BEGIN
    SELECT ename, sal INTO v_nom, v_sal      FROM emp      WHERE empno
= 7839;
    DBMS_OUTPUT.PUT_LINE('Employé : ' || v_nom);
    DBMS_OUTPUT.PUT_LINE('Salaire : ' || v_sal);

    IF v_sal > 4000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salaire supérieur à la
limite autorisée');
    END IF;

    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Erreur détectée !');
            DBMS_OUTPUT.PUT_LINE('SQLCODE : ' || SQLCODE);
            DBMS_OUTPUT.PUT_LINE('SQLERRM : ' || SQLERRM);

END;
/

```

Les Exceptions utilisateur

Résultat :

Employé : KING

Salaire : 5000

Erreur détectée !

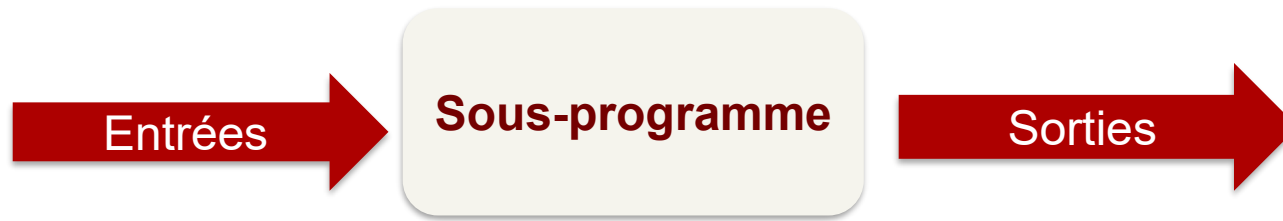
SQLCODE : -20001

SQLERRM : ORA-20001: Salaire supérieur à la
limite autorisée

LES SOUS-PROGRAMMES

Les Sous-programmes

- Les sous-programmes sont des blocs PL/SQL nommés et capables d'inclure des paramètres en entrée et en sortie.



- Dans PL/SQL:

Il existe deux types de sous-programmes :

- les procédures
 - et les fonctions
- les procédures réalisent des actions
 - les fonctions retournent un unique résultat

Les Sous-programmes

Les avantages :

- **Sécurité** : les droits d'accès ne portent plus sur des objets (table, vue, variable...) mais sur des programmes stockés.
- **Intégrité** : les traitements dépendants sont exécutés dans le même bloc (transactions).
- **Performance** : réduction du nombre d'appels à la base (utilisation d'un programme partagé).
- **Productivité** : simplicité de la maintenance des programmes (modularité, extensibilité, réutilisabilité) notamment par l'utilisation de paquetages.

Les sous-programmes

- ✓ Une procédure ou une fonction, peut être appelée à l'aide de :
 - L'interface de commande SQL*Plus (commande EXECUTE).
 - Dans un programme externe (Java, C...).
 - Par d'autres procédures ou fonctions.
 - Ou dans le corps d'un déclencheur.
- ✓ Les fonctions peuvent être appelées dans une instruction SQL (SELECT, INSERT, et UPDATE).
- ✓ Le cycle de vie d'un sous-programme est le suivant :
 - Création de la procédure ou fonction (compilation et stockage)
 - Appels
 - Et éventuellement suppression du sous-programme de la base

Les Procédures

- Pour créer une procédure dans son propre schéma, le privilège ***CREATE PROCEDURE*** est requis.
- Pour créer une procédure dans un autre schéma, il faut posséder le privilège ***CREATE ANY PROCEDURE***.

Les sous-programmes ont une partie :

- de déclaration des variables,
- des instructions
- et une dernière pour gérer les exceptions (erreurs produites durant l'exécution).

Les Procédures

- Une procédure est paramétrable afin d'en faciliter la réutilisation.
- Une procédure est appelée :
 - Dans un bloc PL/SQL :
`nom_Procédure[(liste paramètres)]`
 - Dans SQL*Plus :
`SQL> execute nom_Procédure[(liste paramètres)];`

Les Procédures

Syntaxe

```
CREATE [OR REPLACE] PROCEDURE
nom_proc
(param1 [IN |OUT|INOUT],
param2 [IN |OUT|INOUT] ...)
IS|AS
-- Déclarations locales
BEGIN
-- Instructions
[EXCEPTION
-- Traitement des exceptions]
END;
```

Les Procédures

Au niveau de la liste d'arguments :

Les arguments sont précédés des mots réservés **IN**, **OUT**, ou **IN OUT** et ils sont séparés par des virgules,

- **IN** : paramètre "entrant". il s'agit d'un paramètre dont la valeur est fournie à la procédure stockée. Cette valeur sera utilisée pendant la procédure.
- **OUT** : paramètre "sortant", dont la valeur va être établie au cours de la procédure et qui pourra ensuite être utilisé en dehors de cette procédure.
- **IN OUT** : un tel paramètre sera utilisé pendant la procédure, verra éventuellement sa valeur modifiée par celle-ci, et sera ensuite utilisable en dehors.

Les Procédures

Remarques :

- Contrairement au PL/SQL, la zone de déclaration n'a pas besoin d'être précédé du mot réservé **DECLARE**. Elle se trouve entre le **IS** et le **BEGIN**.
- La dernière ligne de chaque procédure doit être composée du seul caractère **/** pour spécifier au moteur le déclenchement de son exécution.

```

SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE modifie_salaire (id IN NUMBER,
taux IN NUMBER)
IS
BEGIN
    UPDATE emp SET sal = sal * (1 + taux) WHERE empno = id;
    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Employé inexistant');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Salaire modifie avec succès');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
END;
/

```


Les Procédures

Compilation de la procédure « **modifie_salaire** »

Il faut compiler le fichier SQL qui s'appelle ici *modifie_salaire.sql* (le nom du script, dans cet exemple, correspond à celui de la procédure; il faut donc passer, en argument, le nom du script sql à la commande *start* ou bien la commande *@*).

Exemple :

```
SQL> start modifie_salaire  
Procedure created.
```

Les Procédures

Exécution de la procédure

```
SQL> BEGIN  
    modifie_salaire (15,1);  
    END;  
    /  
    PL/SQL procedure successfully completed.
```

Les Procédures

- ***Modification d'une procédure***

Si la base de données évolue, il faut recompiler les procédures existantes pour qu'elles tiennent compte de ces modifications.

```
ALTER PROCEDURE nom_procedure COMPILE;
```

- ***Exemple :***

```
ALTER PROCEDURE modifie_salaire COMPILE;
```

Les Procédures

- ***Suppression d'une procédure***

```
DROP PROCEDURE nom_procedure;
```

- ***Exemple :***

```
DROP PROCEDURE modifie_salaire;
```

Les Fonctions

- Une fonction est une procédure qui retourne une valeur. La seule différence syntaxique par rapport à une procédure se traduit par la présence du mot clé ***RETURN.***
- Une fonction précise le type de donnée qu'elle retourne dans son prototype (signature de la fonction).

Les Fonctions

Syntaxe

```
CREATE [OR REPLACE] FUNCTION nom_func
    (param1 [IN |OUT|INOUT],
     param2 [IN |OUT|INOUT] ...)
RETURN type_valeur_de_retour
IS|AS
-- Déclarations locales
BEGIN
-- Instructions
RETURN valeur_de_retour;
[EXCEPTION
-- Traitement des exceptions]
END;
```

Les Fonctions

Exemple :

```
CREATE OR REPLACE FUNCTION get_salaire (p_empno IN NUMBER)
RETURN NUMBER
IS
    v_sal emp.sal%TYPE;
BEGIN
    SELECT sal INTO v_sal
    FROM emp
    WHERE empno = p_empno;

    RETURN v_sal;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20010, 'Employé inexistant');
END;
/
```

Compilation

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
  2      DBMS_OUTPUT.PUT_LINE('Salaire = ' || get_salaire(7369));
  3  END;
  4  /
Salaire = 2400

Procédure PL/SQL terminée avec succès.

SQL> SELECT get_salaire(7369) FROM dual;

GET_SALAIRE(7369)
-----
                2400

SQL> |
```


Compilation

- Si le message suivant apparaît :

Avertissement : « *Fonction/Procédure créée avec erreurs de compilation* ».

Deux techniques peuvent être utilisées pour visualiser les erreurs de compilation :

- Faire SHOW ERRORS sous SQL*Plus
- Interroger la vue USER_ERRORS :

**SELECT LINE,POSITION,TEXT FROM USER_ERRORS WHERE
NAME='nomFonction/nomProcédure';**

*Sinon, si le message « **Fonction/Procédure créée.** » apparaît,*

→ le sous-programme est correctement compilé et stocké en base.

Compilation

```
SQL> @F:\moyenne_salaire.sql
```

```
Avertissement : Fonction créée avec erreurs de compilation.
```

```
SQL> show errors
```

```
Erreurs pour FUNCTION MOYENNE_SALAIRE :
```

```
LINE/COL ERROR
```

```
-----  
6/4      PL/SQL: SQL Statement ignored
```

```
6/54     PL/SQL: ORA-00904: "V_ID_EMP" : identificateur non valide
```

Récurtivité

- La récursivité est permise dans PL/SQL.
- Comme dans tout programme récursif, il ne faut pas oublier la condition de terminaison !

Récurtivité

Code PL/SQL	Commentaires
<pre>CREATE FUNCTION factorielle(n POSITIVE) RETURN INTEGER IS BEGIN IF n = 1 THEN RETURN 1; ELSE RETURN n * factorielle(n - 1); END IF; END factorielle; /</pre>	Condition de terminaison. Appel récursif.
<pre>SQL> SELECT factorielle(30) "Factorielle 30" FROM DUAL; Factorielle 30 ----- 2,6525E+32</pre>	Appel de la fonction.

Recompilation d'une fonction

- Oracle recompile automatiquement un sous-programme quand un objet qui en dépend directement (table, vue, synonyme, séquence, etc.) a été modifié dans sa structure.
- La syntaxe suivante permet de recompiler manuellement une fonction :

ALTER FUNCTION nomFonction COMPILE;

Destruction d'un sous-programme

- Pour supprimer une fonction dans un autre schéma, le privilège **DROP ANY PROCEDURE** est requis.
- **DROP FUNCTION** [*schéma.*]*nomFonction*;

Il ne faut pas utiliser cette commande pour enlever une procédure ou une fonction d'un paquetage. Pour cela, nous verrons qu'il faudra redéfinir la spécification et le corps du nouveau paquetage en utilisant la directive : **OR REPLACE**

LES PAQUETAGES

Les Paquetages (packages)

- Un paquetage (*package*) est un composant qui regroupe *plusieurs objets (variables, exceptions, curseurs, fonctions, procédures, etc.)* formant un ensemble de services homogènes.
- Un paquetage est un ensemble de procédures et fonctions regroupées dans un objet nommé.
- Par exemple le paquetage Oracle DBMS_LOB regroupe toutes les fonctions et procédures manipulant les grands objets (LOBs).
- Le paquetage UTL_FILE regroupe les procédures et fonctions permettant de lire et écrire des fichiers du système d'exploitation

Le Package

Un paquetage est organisé en deux parties distinctes :

- **Une partie spécification** : Elle permet de spécifier à la fois les fonctions et procédures publiques ainsi que les déclarations des types, variables, constantes, exceptions et curseurs utilisés dans le paquetage et visibles par le programme appelant.
- **Une partie corps** : Elle contient les blocs et les spécifications de tous les objets publics listés dans la partie spécification. Cette partie peut inclure des objets qui ne sont pas listés dans la partie spécification, et sont donc privés. Cette partie peut également contenir du code qui sera exécuté à chaque invocation du paquetage par l'utilisateur.

Spécification

- La syntaxe simplifiée de la déclaration de la spécification d'un paquetage (**CREATE PACKAGE**) est la suivante :

```
CREATE [OR REPLACE] PACKAGE nom_package
AS
    [-- Définition de types]
    [-- Spécification de curseurs]
    [-- Déclaration de variables]
    [-- Déclaration de sous-programmes]
END;
```

Implémentation

- La syntaxe simplifiée de l'implémentation d'un paquetage (**CREATE PACKAGE BODY**) est la suivante :

```
CREATE [OR REPLACE] PACKAGE BODY nom_package
AS
    [-- Définition de types]
    [-- Spécification de curseurs]
    [-- Déclaration de variables]
    [-- Définition de sous-programmes]
END;
```

Exemple

Spécification

```
CREATE PACKAGE GestionEmployes AS

    FUNCTION NbEmployes(pdeptno IN NUMBER)
    RETURN NUMBER;

    PROCEDURE AugmenterSalaire(pempno IN NUMBER,
                                ppct    IN NUMBER);

END GestionEmployes;
/
```

```

CREATE PACKAGE BODY GestionEmployes AS

    FUNCTION NbEmployes(pdeptno IN NUMBER)
    RETURN NUMBER IS
        resultat NUMBER;
    BEGIN
        SELECT COUNT(*) INTO resultat
        FROM EMP
        WHERE DEPTNO = pdeptno;
        RETURN resultat;
    END NbEmployes;

    PROCEDURE AugmenterSalaire(pempno IN NUMBER,
                                ppct    IN NUMBER) IS
    BEGIN
        UPDATE EMP
        SET SAL = SAL + (SAL * ppct / 100)
        WHERE EMPNO = pempno;
    END AugmenterSalaire;

END GestionEmployes;
/

```

Appel

- L'accès à un sous-programme « *sp* » d'un paquetage « *paq* » s'écrit *paq.sp*.
- L'appel de ce sous-programme suit les mêmes règles que celles étudiées dans les sections précédentes.

LES TRIGGERS

Un déclencheur

- Un trigger permet de spécifier les réactions du système d'information lorsque l'on « touche » à ses données.
- Concrètement il s'agit de définir un traitement (un bloc PL/SQL) à réaliser lorsqu'un événement survient
- Déclencheur :
 - **Action** ou ensemble d'actions déclenchée(s) **automatiquement** lorsqu'une **condition** se trouve satisfaite après l'apparition d'un **événement**.

Un déclencheur

Un déclencheur est une règle **ECA** :

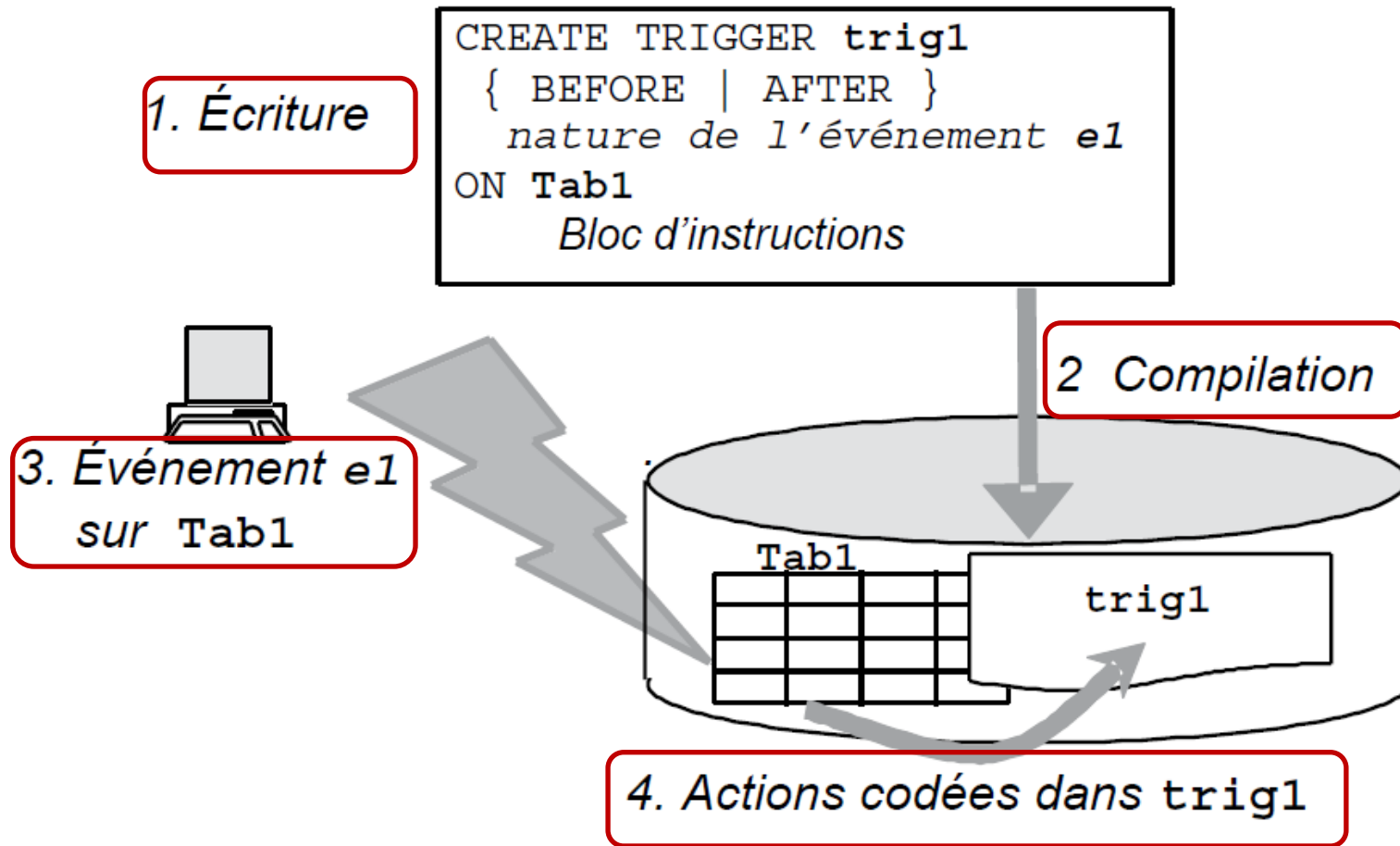
- **Événement** = mise à jour, suppression, insertion
- **Condition** = expression logique vraie ou fausse, optionnelle
- **Action** = procédure exécutée lorsque la condition est vérifiée.

À quoi sert un déclencheur ?

Un déclencheur permet de :

- **Programmer toutes les règles de gestion qui n'ont pas pu être mises en place par des contraintes au niveau des tables.**
- **Déporter des contraintes au niveau du serveur et alléger ainsi la programmation client.**

Mécanisme général



Triggers

Utilité des triggers

- Renforcer la cohérence des données d'une façon transparente pour le développeur,
- Propagation des MAJ dans une base de données distribuée.
- Sécurité...

Privilège

- Pour pouvoir créer un déclencheur, il faut disposer du privilège CREATE TRIGGER.
- Pour créer un déclencheur dans un autre schéma, le privilège CREATE ANY TRIGGER est requis.

Triggers

Remarques :

- Avec ORACLE, les événements sont prédéfinis et les conditions et les actions sont spécifiées par le langage procédural PL/SQL.
- Les événements sont de six types et ils peuvent porter sur des tables ou des colonnes :
 - BEFORE INSERT
 - AFTER INSERT
 - BEFORE UPDATE
 - AFTER UPDATE
 - BEFORE DELETE
 - AFTER DELETE

Triggers

Syntaxe

```
CREATE [ OR REPLACE ] TRIGGER  nom_trigger
  { BEFORE | AFTER }           // événement
  { INSERT | DELETE | UPDATE [ OF liste de
colonnes ] }
  ON table
  [ FOR EACH ROW ]
  [ WHEN ( condition de déclenchement ) ] //
condition
  corps (bloc PL/SQL)
```

Triggers

Entête du trigger

On définit :

- la table à laquelle le trigger est lié.
- les instructions du LMD qui déclenchent le trigger.
- le moment où le trigger va se déclencher par rapport à l'instruction LMD (avant ou après).
- si le trigger se déclenche une seule fois pour toute l'instruction (trigger d'état), ou une fois pour chaque ligne modifiée/insérée/supprimée (trigger ligne, avec l'option FOR EACH ROW).
- et éventuellement une condition supplémentaire de déclenchement (clause WHEN).

Triggers

Type de trigger

Nature de l'événement	État (<i>statement trigger</i>) sans FOR EACH ROW	Ligne (<i>row trigger</i>) avec FOR EACH ROW
BEFORE	Exécution une fois avant la mise à jour.	Exécution avant chaque ligne mise à jour.
AFTER	Exécution une fois après la mise à jour.	Exécution après chaque ligne mise à jour.

Statement trigger

- Exemple : si un UPDATE touche 1000 lignes → le trigger tourne 1 fois.

Row trigger

- Exemple : si un UPDATE touche 1000 lignes → le trigger tourne 1000 fois.

Les Déclencheurs de Lignes

- Un déclencheur de lignes est déclaré avec la directive **FOR EACH ROW**.
- Ce n'est que dans ce type de déclencheur qu'on a accès aux *anciennes valeurs (OLD)* et aux *nouvelles valeurs (NEW)* des colonnes de la ligne affectée par la mise à jour prévue par l'événement.

```
CREATE OR REPLACE TRIGGER TrigSalaireDept
```

```
BEFORE INSERT ON employees ← Déclaration de l'événement  
FOR EACH ROW déclencheur
```

```
DECLARE v_min_sal jobs.min_salary%TYPE;
```

```
v_max_sal jobs.max_salary%TYPE;
```

```
v_job_name jobs.job_title%TYPE;
```

```
BEGIN ← Corps du déclencheur
```

```
SELECT min_salary, max_salary, job_title INTO v_min_sal,  
v_max_sal, v_job_name FROM jobs WHERE job_id = :NEW.job_id;
```

```
IF :NEW.salary BETWEEN v_min_sal AND v_max_sal THEN NULL;
```

```
ELSE RAISE_APPLICATION_ERROR(-20200, 'Salaire ' || :NEW.salary  
|| ' hors grille pour ' || v_job_name || ' [' || v_min_sal ||  
'-' || v_max_sal || ']');
```

← Vérification du salaire

```
END IF;
```

```
EXCEPTION WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-  
20201, 'Job inconnu : ' || :NEW.job_id);
```

```
WHEN OTHERS THEN RAISE;
```

```
END;
```

← Retour de l'erreur courante

Tests du déclencheur

Événement déclencheur	Sortie Oracle
SQL> INSERT INTO employees (employee_id, first_name, last_name, email, hire_date, job_id, salary, department_id) VALUES (300, 'Ali', 'Bensalem', 'ABENSALEM', SYSDATE, 'IT_PROG', 6000, 60);	1 ligne créée. SQL> SELECT employee_id, first_name, salary, job_id FROM employees WHERE employee_id = 300; EMPLOYEE_ID FIRST_NAME SALARY JOB_ID ----- 300 Ali 6000 IT_PROG
SQL> INSERT INTO employees (employee_id, first_name, last_name, email, hire_date, job_id, salary, department_id) VALUES (301, 'Sara', 'Idrissi', 'SIDRISSI', SYSDATE, 'IT_PROG', 500, 60);	ERREUR à la ligne 1 : ORA-20200: Salaire 500 hors grille pour Programmer [4000-10000] ORA-06512: à "HR.TRIGSALAIREDEPT", ligne 14 ORA-04088: erreur lors d'exécution du déclencheur 'HR.TRIGSALAIREDEPT'
SQL> INSERT INTO employees (employee_id, first_name, last_name, email, hire_date, job_id, salary, department_id) VALUES (302, 'Omar', 'Fassi', 'OFASSI', SYSDATE, 'XX_FAKE', 5000, 60);	ERREUR à la ligne 1 : ORA-20201: Job inconnu : XX_FAKE ORA-06512: à "HR.TRIGSALAIREDEPT", ligne 20 ORA-04088: erreur lors d'exécution du déclencheur 'HR.TRIGSALAIREDEPT'
☐ Insertion acceptée ☐ Insertion rejetée par le trigger	

La directive « : NEW »

- Le trigger **BEFORE INSERT** permet de vérifier, avant l'ajout d'un employé, que son salaire appartient bien à la fourchette salariale définie pour son poste.
- On utilise un déclencheur **FOR EACH ROW** car on désire qu'il s'exécute autant de fois qu'il y a de lignes concernées par l'événement déclencheur.
- Chaque enregistrement qui tente d'être ajouté dans la table **employees** est désigné par **:NEW** au niveau du code du déclencheur.

La directive « :OLD »

- Chaque enregistrement qui tente d'être supprimé d'une table qui inclut un déclencheur de type **DELETE FOR EACH ROW**, est désigné par **:OLD** au niveau du code du déclencheur.
- Programmons le déclencheur **TrigDelEmployee** qui surveille les suppressions de la table **Employees** et décrémente de 1 la colonne **nb_employees** pour le département concerné par la suppression.
- L'événement déclencheur est ici **AFTER DELETE** car il faudra s'assurer que la suppression n'est pas entravée par d'éventuelles contraintes référentielles.
- Le trigger ne s'exécute qu'après la confirmation que le **DELETE** a bien réussi. Si le **DELETE** échoue, le trigger ne tourne jamais.

La directive « :OLD »

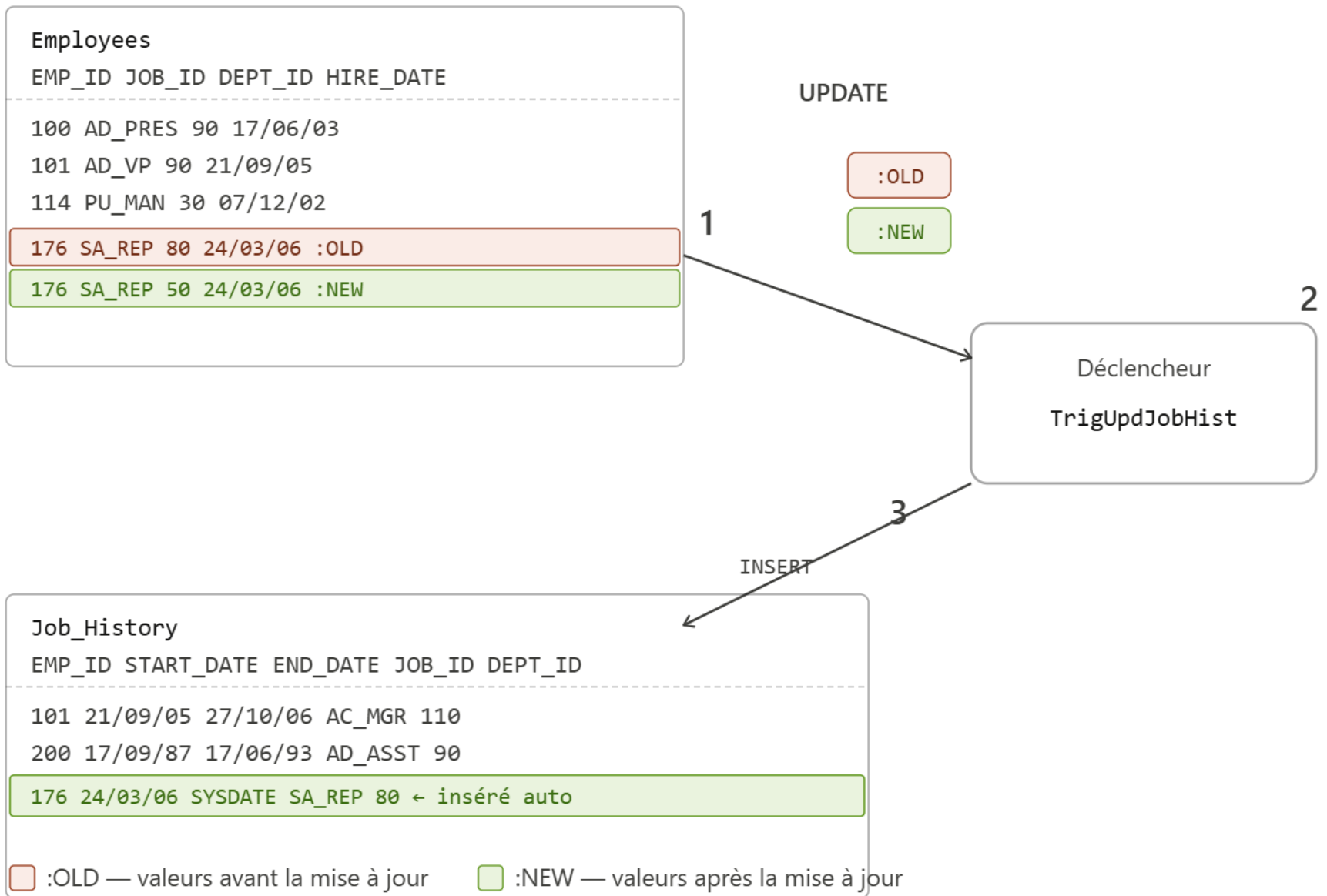
```
CREATE OR REPLACE TRIGGER TrigDelEmployee
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
UPDATE departments SET nb_employees=nb_employees-1
WHERE department_id =:OLD.department_id;
END;
/
```

Quand utiliser à la fois les directives :NEW et :OLD ?

- Seuls les déclencheurs de type **UPDATE FOR EACH ROW** permettent de manipuler à la fois les directives **:NEW** et **:OLD**.
- En effet, la mise à jour d'une ligne dans une table fait intervenir une nouvelle donnée qui en remplace une ancienne.
 - L'accès aux anciennes valeurs se fera par la notation pointée du pseudo-enregistrement :OLD
 - L'accès aux nouvelles valeurs se fera par :NEW

Les directives :NEW et :OLD

```
CREATE OR REPLACE TRIGGER TrigUpdJobHist
AFTER UPDATE OF department_id ON employees
FOR EACH ROW
BEGIN
    INSERT INTO job_history (employee_id, start_date,
end_date, job_id, department_id)
    VALUES (:OLD.employee_id,
            :OLD.hire_date,
            SYSDATE,
            :OLD.job_id,
            :OLD.department_id) ;
END ;
```

Les Déclencheurs de Lignes

Le principe

- ① UPDATE — On change le DEPARTMENT_ID de l'employé 176 (de 80 → 50). Le trigger voit :OLD (ancien département = 80) et :NEW (nouveau département = 50).
- ② Déclencheur *TrigUpdJobHist* — Il s'active automatiquement après chaque UPDATE sur EMPLOYEES.
- ③ INSERT automatique — Il archive l'ancienne affectation dans *JOB_HISTORY* en utilisant :OLD.DEPARTMENT_ID, :OLD.JOB_ID, et SYSDATE comme date de fin.

Gestion des déclencheurs

SQL	Commentaires
<code>ALTER TRIGGER <i>nomDéclencheur</i> COMPILE;</code>	Recompilation d'un déclencheur.
<code>ALTER TRIGGER <i>nomDéclencheur</i> DISABLE;</code>	Désactivation d'un déclencheur.
<code>ALTER TABLE <i>nomTable</i> DISABLE ALL TRIGGERS;</code>	Désactivation de tous les déclencheurs d'une table.
<code>ALTER TRIGGER <i>nomDéclencheur</i> ENABLE;</code>	Réactivation d'un déclencheur.
<code>ALTER TABLE <i>nomTable</i> ENABLE ALL TRIGGERS;</code>	Réactivation de tous les déclencheurs d'une table.
<code>DROP TRIGGER <i>nomDéclencheur</i>;</code>	Suppression d'un déclencheur.

Triggers

Exemple 1 :

- Écrire un trigger ***TrigProtectManager*** qui se déclenche **avant chaque suppression** d'un employé.
- Le trigger doit vérifier si l'employé est référencé comme manager_id dans la table DEPARTMENTS.
- Si c'est le cas, **bloquer la suppression** avec un message d'erreur explicite contenant l'identifiant de l'employé.

Triggers

```
CREATE OR REPLACE TRIGGER TrigProtectManager
BEFORE DELETE ON employees
FOR EACH ROW
DECLARE
v_count NUMBER;
BEGIN
SELECT COUNT(*) INTO v_count FROM departments
WHERE manager_id = :OLD.employee_id;
    IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20400, 'Impossible : employé
        '|| :OLD.employee_id ||' est le manager d'un département);
    END IF;
END;
/
```

Triggers

Exemple 2 :

- Écrire un trigger **TrigDecrEmp** qui se déclenche **après chaque suppression** d'un employé.
- Ce trigger doit **décrémenter de 1** la colonne **nb_employees** du département concerné dans la table DEPARTMENTS.
- Il doit également **archiver** les informations de l'employé supprimé dans la table **employees_archive** avec la date du jour.

Triggers

```
CREATE OR REPLACE TRIGGER TrigDecrEmp
AFTER DELETE ON employees
FOR EACH ROW BEGIN
UPDATE departments SET nb_employees = nb_employees-1
WHERE department_id = :OLD.department_id;
INSERT INTO employees_archive (employee_id,
first_name, last_name, job_id, salary, department_id,
date_suppression)
VALUES (:OLD.employee_id, :OLD.first_name, :OLD.last_n
ame, :OLD.job_id, :OLD.salary, :OLD.department_id,
SYSDATE) ;
END ;
/
```

Triggers

Exemple 3 :

Lorsqu'une augmentation du prix unitaire (PU) d'un produit est tentée, il faut limiter l'augmentation à **10%** du prix en cours.

Triggers

```
UPDATE PRODUIT
SET Prix = 15.99
WHERE Codprod = 10 ;
```

```
CREATE OR REPLACE TRIGGER BORNER_AUGMENTPU
BEFORE UPDATE OF Prix ON PRODUIT
FOR EACH ROW
When (New.Prix > Old.Prix * 1.1)
BEGIN
    : New.Prix := :Old.Prix * 1.1 ;
END ;
/
```

	Codprod	Prix	...
Ligne avant (OLD)	10	11	

	Codprod	Prix	...
Ligne après (NEW)	10	15.99	

	Codprod	Prix	...
Ligne après (NEW)	10	12.1	